

SOCIETY OS

Enterprise Product Blueprint & Functional Specification

For Indian Residential Communities, Standalone Buildings, Apartments, Villas, Complexes and Townships

VERSION 4.0 | Consolidated Enterprise Product Blueprint & Functional Specification

Document Control	Details
Document type	Product blueprint, functional specification, implementation guide and roadmap
Product	Society OS
Target market	Indian residential societies, RWAs, CHS, standalone buildings, gated communities, villas, mixed-use complexes and townships
Commercial direction	Free launch for selected societies; subscription-ready architecture retained privately
Document version	v4.0 — Consolidated Enterprise Production Blueprint
Status	Approved for Engineering Execution, AI Code Generation (Codex/Antigravity), Design & QA

CONFIDENTIAL — PRODUCT & IMPLEMENTATION DOCUMENT

VERSION 4.3 CONSOLIDATION: Existing 19-section structure and v3 content are retained; v4 requirements are integrated as production-level expansions. The blueprint deliberately avoids feature-to-role/application mapping and screen-by-screen UI specification.

VERSION 4.3 SCENARIO-FIRST OBJECTIVE - This master blueprint is designed for Product, Engineering, QA, Management and pitch/demo use. Every major capability is expected to be explained through real-world scenarios, explicit normal/alternate/exception paths, authoritative data behaviour, measurable acceptance criteria, and aligned use-case/data-flow diagrams.

VERSION 4.3 EXPANSION NOTE — Existing v4 content is retained. This revision adds foundational day-to-day workflows, real-world scenarios, exception handling, reusable platform engines, operational test dimensions, migration/recovery detail, and pitch-ready narrative depth. The document may intentionally exceed 109 pages.

Table of Contents

- 1. Executive Summary
- 2. Product Vision and Principles
- 3. User Personas and Applications

4.3 Feature Definition Standard

Each feature in the catalogue should be understood as a capability contract with five mandatory dimensions: trigger, business rules, state/lifecycle, exceptions, and measurable outcome. A feature is not considered fully specified merely because its name appears in the catalogue.

Capability area	Normal scenario	Important exception	Business outcome
Property & resident foundation	New owner registers and is linked to Unit A-1203	Ownership transfer occurs while a tenant is active	Accurate occupancy and historical traceability
Access & gate	Pre-approved visitor arrives and pass is validated	Resident is unreachable and visitor is not pre-approved	Secure admission with auditable exception handling
Finance	Monthly invoice is generated and paid by UPI	Gateway sends duplicate callback after timeout	Exactly-once financial posting
Operations	Lift AMC work order becomes due	Vendor misses SLA and uploads incomplete service evidence	Preventive maintenance and escalation visibility

4. Complete Feature Catalogue

5. Architecture and Data Flow Diagrams

6. Role-based Permission Model

7. Deep Functional Specification by Module

8. Owner, Tenant and Occupancy History Management

9. Biometric Attendance Integration

10. Document Vault, NOC and Secure Access

11. Workflows and User Journeys

12. Data Model and API Design

13. Feature Flags and Configuration

14. Security, Privacy and Audit Logging

15. Reports and Society Health Score

16. Smart Automation and Future Integrations

17. Roadmap and Delivery Priorities

18. Acceptance Criteria and Implementation Backlog

19. Appendix

Document discipline This specification defines product behaviour and engineering intent. It deliberately avoids screen-by-screen UI specifications and avoids assigning individual features to particular apps or roles; those details belong in dedicated UX and authorization specifications.

This blueprint defines behaviour, state machines, contracts, data rules, architecture, acceptance criteria and engineering intent. It intentionally does not prescribe screen-by-screen layouts or a feature-to-role/application visibility matrix; those belong in dedicated UX and authorization specifications.

1.6 Document Discipline

Convenience first; configurable society profiles; India-first payment and operational workflows; DPDP-oriented purpose/consent and retention boundaries; immutable financial history; hardware/offline resilience; and enterprise-grade traceability.

1.5 Core Guarantees

Target runtimes are cross-platform native mobile for iOS and Android, Android Tablet/Android Go for guard operations, and a React/Next.js administrative web console. The distributed backend is API-first and must keep authoritative domain, financial, access, workflow and audit state deterministic.

1.4 Production Runtime Contract

Guard workflows should minimize typing, use large touch targets and high contrast, provide camera/OCR assistance, support local-language/audio prompts and remain usable through network outages.

2.7 Low-Cognitive-Load Guard Operations

Accounting standards, fund treatment, interest rules, statutory exports, regional language, verification and society governance rules must be represented as configurable rule packs. Maharashtra, Karnataka and other state-specific assumptions must not be hard-coded as universal rules.

2.6 India-First Rule Packs

Profiles are configurable presets rather than separate products: Standalone/Small CHS (approximately 8–50 units), Mid-tier Gated Society (approximately 50–500 units), and Integrated Multi-Phase Township (approximately 500–10,000+ units). Individual society settings can enable or disable capabilities subject to dependency and authorization checks.

2.5 Dynamic Society Sizing

The application ecosystem is described for runtime and operational context only. This blueprint deliberately avoids mapping individual features to individual roles or applications.

3.4 Boundary Rule

Mobile clients are battery/network aware; tablets support landscape operational workflows; low-memory Android Go devices suppress nonessential animation and cache pressure; desktop web supports information-dense administrative workflows. All surfaces use the same domain model, configuration and authorization contract.

3.3 Device Classes and Runtime Boundaries

Standalone/Small CHS: core billing, notices, complaints, basic visitor and documents. Mid-tier Gated: pre-approval, delivery handling, facilities, parking, vendor/AMC and dispute workflows. Township: multi-gate ANPR/Rfid, boom barriers, smart sub-metering, biometric shift calculations, multi-committee governance, e-voting and zone-based security. Profiles can be overridden by validated society settings.

4.2 Configuration Profiles

Visitor state machine, resident revocation race handling, dynamic parcel cubby PINs, overstay rules, offline reconciliation, double-entry financial reversals, explainable rental/tenant assessment, owner/tenant historical privacy, biometric log-only integration, cryptographic NOC verification, Smart Poll Engine, Vendor Sinking Fund/Milestone Escrow, acoustic/seepage dispute evidence, Smart Lost & Found, Community Carpool, Festival/Event Automation, Society Micro-Commerce, Domestic Worker Replacement Network, Pet Management, Guest Stay Management, Housekeeping Quality, intercom replacement, payroll readiness, water/electricity/solar/EV operations, utility connectors and enterprise intelligence are included as configurable capabilities.

4.1 v4 Enterprise Additions

PostgreSQL is authoritative for transactions. Redis supports cache/pub-sub/idempotency/session use cases but is not the financial or access source of truth. Object storage is private. Long-running jobs use queues/outbox workers. Integrations are adapter-based and circuit-breaker protected. Audit events are emitted by sensitive domain mutations.

5.9 Architecture Principles

Critical field events are recorded locally with client UUIDs, timestamps and synchronization metadata. On reconnect, the server validates tenant context, deduplicates, applies deterministic ordering/conflict rules and returns per-record outcomes. Third-party hardware failures must degrade to safe manual or queued modes rather than disabling unrelated functions.

5.8 Offline / Hardware Reliability

X-Society-Id is an input, not proof of authorization. The server validates the authenticated session, society context and entity relationship before accessing data. Repository/ORM operations carry societyId, and PostgreSQL RLS reinforces isolation.

Figure 12. Tenant resolution, authorization and PostgreSQL isolation boundary.

5.7 Multi-Tenant Security Boundary

The resident, guard and administrative clients communicate through a secure gateway. Local persistence uses encrypted SQLite/SQLCipher or WatermelonDB-backed SQLite. Secure storage holds tokens and encryption keys. Real-time synchronization may use WebSockets/MQTT with backoff. High-priority approval calling may use platform-supported VoIP/foreground mechanisms where permitted.

Figure 11. Cross-platform runtime, gateway and service architecture.

5.6 Cross-Platform Runtime Architecture

Figure 13. Original blueprint feature-flag and permission control flow retained for continuity.

Current unit association does not grant access to previous occupants' KYC, agreements, phone numbers or sensitive cases. Historical access is separately authorized and audited.

6.6 Historical Data Boundary

Frontend restrictions improve usability only. Backend authorization remains authoritative. A feature being visible does not make it executable, and a hidden feature must still be blocked server-side.

6.5 Authorization Independence

Every protected request evaluates active session, trusted society context, role/permission claims, feature state, entity relationship and sensitivity, valid workflow transition, configured approval requirements and audit requirements.

6.4 Eight Authorization Filters

A later occupant does not inherit prior occupant KYC, rent agreements, phone numbers or sensitive tickets merely because the unit is the same.

8.9 Historical Privacy

The rental/tenant assessment is an explainable operational readiness indicator based on authorized criteria such as verification completeness, agreement validity and configured society requirements. It must not use protected or unrelated personal characteristics and must show the contributing factors.

8.8 Rental / Tenant Assessment

Clearance evaluates authoritative dues, physical access assets, parking, damage, configured complaints/disputes and required documents. A blocked clearance creates an explicit exception; a permitted

override requires a signed reason and audit record. NOC issuance creates a signed PDF and verification digest/code. Access revocation invalidates sessions and configured gate mappings before archival.

Figure 15. Detailed move-out, clearance, NOC, access revocation and archival flow.

8.7 Move-Out Lifecycle

Request → identity/profile capture → KYC/document collection → owner approval where required → society validation → rule acknowledgement → lift/truck logistics where configured → access activation → ACTIVE occupancy.

8.6 Move-In Lifecycle

Corrections preserve old/new values, reason, actor, approval and audit. Daily/monthly summaries support shift adherence and outsourced security/housekeeping invoice verification.

9.6 Attendance Correction and Billing Support

Biometric capture/matching stays in the device/vendor ecosystem. Society OS receives attendance logs such as employee code, device identifier, punch timestamp, punch type, source and synchronization status. Duplicate keys, tenant scope and staff mapping are validated before persistence.

Figure 16. Biometric attendance synchronization without raw biometric storage.

9.5 Production Sync Boundary

No-dues, move-out, tenant, parking, renovation and residence certificates are configurable. Each defines clearance checks, approval chain, document template, verification method and retention policy.

10.6 NOC Catalogue

Sensitive documents store category, owning entity, sensitivity, version, checksum, storage key, expiry and retention. Object storage remains private. Downloads require a fresh authorization decision and a short-lived signed URL/controlled stream. NOCs/certificates can carry SHA-256 verification data and QR codes.

Figure 17. Secure document lifecycle and controlled retrieval.

10.5 Cryptographic Integrity and Retrieval

Trigger → classify → dispatch → acknowledge → respond → escalate → close → post-incident review. Delivery failure uses configured fallback paths.

11.7 Emergency Flow

Request → purpose → recipient decision → persistent channel on acceptance; report routes to audited moderation. Phone numbers remain masked.

11.6 Resident Connect First Contact

Pass revocation versus offline admission: server-authoritative state wins when the revocation is determined to precede the admission; the admission becomes UNAUTHORIZED_ENTRY_FLAGGED. Duplicate punches are silently deduplicated by deterministic key. Concurrent edits use server-monotonic ordering and workflow state; stale clients receive explicit conflict responses.

11.5 Conflict Resolution Rules

Offline records carry client UUID, device identifier, local timestamp and vector/ordering metadata. Reconnect processing authenticates, validates tenant scope, deduplicates and resolves each record independently. One conflict must not fail an entire batch.

Figure 18. Offline queue, idempotency and deterministic conflict resolution.

11.4.1 Import, Export and Data Migration Workflow

Operational deployments often begin with spreadsheets. Import must therefore be a product capability: Upload → Format Validation → Column Mapping → Duplicate Check → Business Validation → Preview → Confirm → Async Processing → Success/Failure Report → Audit. A failed row must be downloadable with an actionable error reason.

Real-world scenario — 5,000-resident migration: a society uploads owners, tenants, vehicles, parking and opening balances from legacy spreadsheets. 4,912 records pass, 88 fail because of duplicate unit associations or invalid payment references. The system imports only the confirmed valid set, exposes the 88 errors, and preserves import history so the same file is not accidentally applied twice.

11.4.2 Backup, Restore and Disaster Recovery Workflow

Production operations require database backups, object/document backups, encryption, retention policies, point-in-time recovery where supported, restore testing, disaster recovery procedures and preservation of audit integrity. Backup success must be monitored; “backup job ran” is not equivalent to “restore is possible”.

Real-world scenario — accidental data corruption: a bad deployment corrupts a configuration table. The recovery process restores the affected state to a known point, validates tenant isolation, checks financial consistency and confirms that audit records remain intact before traffic is returned to the recovered environment.

11.4 Offline Gate Sync

Use versioned endpoints, society-scoped context, idempotency for retryable mutations, signature verification for callbacks, stable identifiers, deterministic error contracts, secure upload/retrieval separation and trace IDs.

11.4.4 Account, Session and Device Lifecycle

Authentication must include login/OTP/password flows as configured, device registration, session issuance, expiry, token revocation, force logout, lost-device handling and suspicious-login controls. Device trust is separate from user identity.

Real-world scenario — lost guard tablet: the tablet is reported lost. Existing sessions/tokens are revoked, the device is marked untrusted, and a replacement device must complete registration before offline operational data can be synchronized.

- Sessions have explicit expiry and revocation state.
- Device identifiers must not be treated as permanent identities for a human.
- Offline capability must use bounded credentials and local data protection.
- Suspicious login signals should be observable and auditable.

11.4.3 File and Media Management

Files should be treated as managed objects rather than arbitrary URLs. The lifecycle includes upload initiation, chunk/resume where supported, validation, malware scanning, checksum, metadata normalization, storage, versioning, access control, download audit and retention.

Real-world scenario — gate photo upload: a guard captures a damaged vehicle photo while offline. The attachment enters a local queue, is retried when connectivity returns, and is attached to the violation record only after server validation confirms integrity.

- Allowed MIME types, size limits and retention are configurable.
- EXIF metadata should be stripped where privacy policy requires it.
- Failed scans quarantine the object and prevent normal retrieval.

- Access URLs should be short-lived/signed where appropriate and must not become authorization bypasses.

12.8 API Rules

POST /api/v1/gate/sync-batch — offline batch reconciliation; per-record COMMITTED/CONFLICT results.

POST /api/v1/gate/visitor-checkin — visitor creation/check-in with secure pass token and server monotonic time.

POST /api/v1/finance/adjustments — compensating/reversal journal linked to original transaction.

POST /api/v1/visitors/preapprovals; POST /api/v1/gate/entries; POST /api/v1/bills/generate; POST /api/v1/payments/callback; POST /api/v1/complaints; POST /api/v1/bookings; POST /api/v1/integrations/biometric/punches/sync; GET /api/v1/attendance/monthly; PATCH /api/v1/attendance/{id}/correction.

12.7 Contract APIs

CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE EXTENSION IF NOT EXISTS "citext";

- Core tables: societies, units, users, occupancies, visitors, financial_accounts,
- ledger_transactions, ledger_lines, document_records, staff_profiles,
- biometric_attendance_punches and disputes.
- Production rule: every tenant-scoped table carries society_id; ledger_lines
- also carries society_id so RLS can be enforced directly.
- Posted financial records are append-only; corrections are reversal journals.

ALTER TABLE units ENABLE ROW LEVEL SECURITY;

ALTER TABLE visitors ENABLE ROW LEVEL SECURITY;

ALTER TABLE ledger_transactions ENABLE ROW LEVEL SECURITY;

ALTER TABLE ledger_lines ENABLE ROW LEVEL SECURITY;

ALTER TABLE document_records ENABLE ROW LEVEL SECURITY;

ALTER TABLE occupancies ENABLE ROW LEVEL SECURITY;

CREATE POLICY society_isolation_units ON units FOR ALL USING

(society_id = NULLIF(current_setting('app.current_society_id', true), '')::UUID);

CREATE POLICY society_isolation_visitors ON visitors FOR ALL USING

(society_id = NULLIF(current_setting('app.current_society_id', true), '')::UUID);

CREATE POLICY society_isolation_ledger_transactions ON ledger_transactions FOR ALL USING

(society_id = NULLIF(current_setting('app.current_society_id', true), '')::UUID);

CREATE POLICY society_isolation_ledger_lines ON ledger_lines FOR ALL USING

(society_id = NULLIF(current_setting('app.current_society_id', true), '')::UUID);

CREATE POLICY society_isolation_documents ON document_records FOR ALL USING

(society_id = NULLIF(current_setting('app.current_society_id', true), '')::UUID);

CREATE POLICY society_isolation_occupancies ON occupancies FOR ALL USING

(society_id = NULLIF(current_setting('app.current_society_id', true), '')::UUID);

12.6 Consolidated Core DDL

The v4 production schema must scope uniqueness and idempotency by society where business identifiers are society-local; carry society_id onto ledger_lines for direct isolation; apply RLS to every sensitive tenant-scoped table; and enforce balanced journal posting in a transaction. Original posted ledger rows are not updated or deleted.

12.5 Production DDL Hardening Notes

POST /api/v1/finance/adjustments — Request contains originalTransactionId, reversalReason, supportingDocumentId and effectiveDate. Response 201 returns reversalTransactionId, referenceNumber, posted debit/credit lines and balanceStatus=BALANCED.

POST /api/v1/gate/visitor-checkin — Request contains unitId, visitorName, phoneNumber, visitorType, optional deliveryBrand, vehicleRegistration, gateId and offlineCreatedTimestamp. Response 201 returns visitorId, status, requiresCallBypass, secure passValidationToken, residentActionTimeoutSeconds, traceId and serverMonotonicTime.

POST /api/v1/gate/sync-batch — Headers: Authorization Bearer <JWT>, X-Society-Id <UUID>, Idempotency-Key <UUID>. Request contains deviceIdentifier, batchTimestamp and records with idempotencyKey, unitId, visitorName, phoneNumber, visitorType, vehicleNumber, action, offlineEventTimestamp and vectorClock. Response 200 returns PROCESSED, processedCount, per-record COMMITTED/CONFLICT resolution, visitorId/currentStatus and serverSyncTimestamp.

12.10 Exact API Contract Examples

```
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
```

```
CREATE EXTENSION IF NOT EXISTS "citext";
```

```
CREATE TABLE societies (
```

```
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), slug CITEXT UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL, registration_number VARCHAR(100), state_code VARCHAR(10) NOT
  NULL,
```

```
  currency_code VARCHAR(3) NOT NULL DEFAULT 'INR', accounting_standard VARCHAR(100),
  settings JSONB NOT NULL DEFAULT '{}::jsonb, is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT
  NOW()
```

```
);
```

```
CREATE TABLE units (
```

```
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID NOT NULL REFERENCES
  societies(id) ON DELETE RESTRICT,
```

```
  phase VARCHAR(100), tower VARCHAR(100) NOT NULL, block VARCHAR(100), wing VARCHAR(100),
  floor_number INT NOT NULL,
```

```
  unit_number VARCHAR(50) NOT NULL, unit_type VARCHAR(50) NOT NULL DEFAULT 'FLAT',
  super_builtup_sqft NUMERIC(10,2), carpet_sqft NUMERIC(10,2), occupancy_status VARCHAR(50) NOT
  NULL DEFAULT 'VACANT',
```

```
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT
  NOW(),
```

```
  CONSTRAINT uk_society_unit UNIQUE (society_id, tower, unit_number)
```

```
);
```

```
CREATE TABLE users (
```

```
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID REFERENCES societies(id) ON
  DELETE RESTRICT,
```

```
  phone_number VARCHAR(20) NOT NULL, email CITEXT, full_name VARCHAR(255) NOT NULL,
  password_hash VARCHAR(255),
```

```
  dpdp_consent_record JSONB NOT NULL DEFAULT '{}::jsonb, is_suspended BOOLEAN NOT NULL DEFAULT
  FALSE,
```

```
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT
  NOW(),
```

```
  CONSTRAINT uk_society_user_phone UNIQUE (society_id, phone_number)
```

```
);
```

```
CREATE TABLE occupancies (
```

```
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID NOT NULL REFERENCES
```

```

societies(id) ON DELETE RESTRICT,
    unit_id UUID NOT NULL REFERENCES units(id) ON DELETE RESTRICT, user_id UUID NOT NULL
REFERENCES users(id) ON DELETE RESTRICT,
    occupant_role VARCHAR(50) NOT NULL, start_date DATE NOT NULL, end_date DATE,
    status VARCHAR(50) NOT NULL DEFAULT 'PENDING_APPROVAL', created_at TIMESTAMPTZ NOT NULL
DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TABLE visitors (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID NOT NULL REFERENCES
societies(id) ON DELETE RESTRICT,
    unit_id UUID NOT NULL REFERENCES units(id) ON DELETE RESTRICT, visitor_name VARCHAR(255) NOT
NULL, phone_number VARCHAR(20),
    visitor_type VARCHAR(50) NOT NULL, vehicle_registration VARCHAR(50), pass_code_hash VARCHAR(255),
    entry_gate_id VARCHAR(50), exit_gate_id VARCHAR(50), check_in_at TIMESTAMPTZ, check_out_at
TIMESTAMPTZ,
    current_status VARCHAR(50) NOT NULL DEFAULT 'EXPECTED', idempotency_key UUID NOT NULL,
sync_vector_clock BIGINT NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), CONSTRAINT uk_visitor_idempotency UNIQUE
(society_id, idempotency_key)
);
CREATE TABLE financial_accounts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID NOT NULL REFERENCES
societies(id) ON DELETE RESTRICT,
    account_code VARCHAR(50) NOT NULL, account_name VARCHAR(255) NOT NULL, account_type
VARCHAR(50) NOT NULL,
    is_statutory_fund BOOLEAN NOT NULL DEFAULT FALSE, CONSTRAINT uk_society_account UNIQUE
(society_id, account_code)
);
CREATE TABLE ledger_transactions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID NOT NULL REFERENCES
societies(id) ON DELETE RESTRICT,
    unit_id UUID REFERENCES units(id) ON DELETE RESTRICT, transaction_date TIMESTAMPTZ NOT NULL
DEFAULT NOW(),
    reference_number VARCHAR(100) NOT NULL, description TEXT NOT NULL, is_reversal BOOLEAN NOT
NULL DEFAULT FALSE,
    reversed_transaction_id UUID REFERENCES ledger_transactions(id), created_by UUID NOT NULL
REFERENCES users(id), created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT uk_society_reference UNIQUE (society_id, reference_number)
);
CREATE TABLE ledger_lines (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID NOT NULL REFERENCES
societies(id) ON DELETE RESTRICT,
    transaction_id UUID NOT NULL REFERENCES ledger_transactions(id) ON DELETE RESTRICT, account_id
UUID NOT NULL REFERENCES financial_accounts(id) ON DELETE RESTRICT,
    amount NUMERIC(14,2) NOT NULL CHECK (amount > 0), entry_type VARCHAR(10) NOT NULL CHECK
(entry_type IN ('DEBIT','CREDIT'))
);
CREATE TABLE document_records (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID NOT NULL REFERENCES
societies(id) ON DELETE RESTRICT,
    unit_id UUID REFERENCES units(id) ON DELETE RESTRICT, category VARCHAR(50) NOT NULL,
file_storage_key VARCHAR(512) NOT NULL,
    file_sha256_checksum CHAR(64) NOT NULL, mime_type VARCHAR(100) NOT NULL, is_encrypted

```

```

BOOLEAN NOT NULL DEFAULT TRUE,
  access_level VARCHAR(50) NOT NULL DEFAULT 'RESTRICTED', retention_valid_until DATE, created_at
TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TABLE staff_profiles (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID NOT NULL REFERENCES
societies(id) ON DELETE RESTRICT,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT, staff_type VARCHAR(50) NOT
NULL, employee_code VARCHAR(100) NOT NULL,
  is_verified BOOLEAN NOT NULL DEFAULT FALSE, created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
CONSTRAINT uk_society_employee UNIQUE (society_id, employee_code)
);
CREATE TABLE biometric_attendance_punches (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID NOT NULL REFERENCES
societies(id) ON DELETE RESTRICT,
  staff_id UUID NOT NULL REFERENCES staff_profiles(id) ON DELETE RESTRICT, device_identifier
VARCHAR(100) NOT NULL,
  punch_timestamp TIMESTAMPTZ NOT NULL, punch_type VARCHAR(20) NOT NULL CHECK (punch_type
IN ('IN','OUT')),
  punch_hash CHAR(64) NOT NULL, created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), CONSTRAINT
uk_society_punch_hash UNIQUE (society_id, punch_hash)
);
CREATE TABLE disputes (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), society_id UUID NOT NULL REFERENCES
societies(id) ON DELETE RESTRICT,
  initiator_unit_id UUID NOT NULL REFERENCES units(id) ON DELETE RESTRICT, target_unit_id UUID NOT
NULL REFERENCES units(id) ON DELETE RESTRICT,
  category VARCHAR(50) NOT NULL, status VARCHAR(50) NOT NULL DEFAULT 'OPEN', evidence_payload
JSONB NOT NULL DEFAULT '{}::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(), updated_at TIMESTAMPTZ NOT NULL DEFAULT
NOW()
);
ALTER TABLE units ENABLE ROW LEVEL SECURITY; ALTER TABLE visitors ENABLE ROW LEVEL SECURITY;
ALTER TABLE ledger_transactions ENABLE ROW LEVEL SECURITY; ALTER TABLE ledger_lines ENABLE
ROW LEVEL SECURITY;
ALTER TABLE document_records ENABLE ROW LEVEL SECURITY; ALTER TABLE occupancies ENABLE ROW
LEVEL SECURITY;
CREATE POLICY society_isolation_units ON units FOR ALL USING (society_id =
NULLIF(current_setting('app.current_society_id', true), '')::UUID);
CREATE POLICY society_isolation_visitors ON visitors FOR ALL USING (society_id =
NULLIF(current_setting('app.current_society_id', true), '')::UUID);
CREATE POLICY society_isolation_ledger_transactions ON ledger_transactions FOR ALL USING (society_id =
NULLIF(current_setting('app.current_society_id', true), '')::UUID);
CREATE POLICY society_isolation_ledger_lines ON ledger_lines FOR ALL USING (society_id =
NULLIF(current_setting('app.current_society_id', true), '')::UUID);
CREATE POLICY society_isolation_documents ON document_records FOR ALL USING (society_id =
NULLIF(current_setting('app.current_society_id', true), '')::UUID);
CREATE POLICY society_isolation_occupancies ON occupancies FOR ALL USING (society_id =
NULLIF(current_setting('app.current_society_id', true), '')::UUID);

```

12.9 Full v4 DDL Reference

```

{"tier":"GATED_APARTMENT_COMPLEX","modules":{"visitorManagement":
{"enabled":true,"enforceVehicleOCR":true,"requireResidentApprovalOverstayMinutes":120,"allowGuardMan

```

```

ualOverride":false},"financialLedger":
{"enabled":true,"lateFeeAnnualInterestPercent":21.0,"lateFeeType":"SIMPLE_INTEREST","reversalRequiresD
ualSignoff":true},"hardwareGateway":
{"biometricSyncEnabled":false,"anprBarrierEnabled":false},"interFlatChat":
{"enabled":true,"maskContactNumbers":true,"requireFirstContactApproval":true}}}

```

13.7 Example JSON Configuration

Figure 19. Feature flag evaluation followed by independent authorization and workflow validation.

13.6 Runtime Control Flow

Enablement requires dependency validation. Disabling a module blocks new business actions but does not delete historical records. Configuration changes are audited with old value, new value, actor and timestamp. Feature flags never replace authorization.

13.5 Dependency and Runtime Rules

Core launch flags remain visitorManagement, maintenanceBilling, complaints, documentVault, ownerTenantHistory, moveOutNocMandatory and residentMessenger enabled; public directory, biometric and hardware integrations, subscription UI and smart automation remain disabled by default. Water, energy, EV, governance voting, marketplace, carpool, lost & found, festival automation, guest stays, housekeeping, payroll, utility integrations and advanced intelligence are configurable.

13.4 v4 Configuration Contract

Sensitive gate/security events can be chained in a Merkle-style append-only audit structure. Document issuance can store SHA-256 of serialized bytes. Audit records include tenant, actor, action, target, timestamp, result and trace context without unnecessary duplication of personal content.

14.6 Cryptographic Audit

Policy baselines such as 180-day retention for selected transient visitor/security PII and approximately 8-year retention for selected financial/audit records may be configured, but production values must be validated against the applicable legal, contractual and society policy requirements.

14.7 Backup, Disaster Recovery and Business Continuity

Backup and disaster recovery are production capabilities. The platform should protect relational data, document/object storage and critical configuration with encrypted backups and defined retention. Recovery procedures must be rehearsed rather than assumed.

Real-world scenario — regional outage: the primary hosting region becomes unavailable. The recovery plan restores services according to defined RPO/RTO targets, validates tenant isolation and financial consistency, and records the recovery event. The target is controlled continuity, not an untested promise of zero downtime.

- Define RPO/RTO per environment and criticality tier.
- Perform periodic restore verification.
- Preserve audit integrity during restoration.
- Have explicit procedures for corruption, accidental deletion, credential compromise and infrastructure loss.

14.5 Retention Baselines

The system records purpose/consent context where required and evaluates retention before deletion/anonymisation. An erasure request does not automatically remove records that must legitimately be retained; instead the retention/legal-hold decision is recorded and personal fields are minimized or scrubbed where eligible.

Figure 20. Purpose, retention, erasure/anonymisation and retained-record separation.

14.4 DPDP-Oriented Retention Architecture

Every score movement must be traceable to source metrics, time window, formula/version and weighting. AI may summarize but cannot invent the underlying metric.

15.5 Explainability

Collection efficiency, complaint SLA, security discipline, compliance health, operational efficiency, resident satisfaction, workforce health and utility/sustainability health may contribute to a transparent society-level score.

15.5.1 Health Score Calculation and Guardrails

The Society Health Score should combine measurable dimensions such as collections, complaint SLA, preventive maintenance, vendor performance, facility utilization, incident history, document/compliance status and operational responsiveness. It should explain which factors drove the score.

Real-world scenario — score falls from 82 to 68: the explanation identifies increased overdue dues, two missed critical maintenance tasks and a complaint SLA breach. Management can drill into source records instead of receiving an opaque AI judgement.

- Weights are configurable and versioned.
- Low data quality reduces confidence rather than being silently treated as a zero score.
- AI-generated narratives must link back to source metrics and should not invent causes.

15.4 Expanded Health Dimensions

Payment, messaging, biometric and utility integrations use circuit breakers, queues and retry state. A baseline of three consecutive failures may open a circuit; synchronous unavailable services return controlled 503/Retry-After behaviour where applicable.

16.9 Integration Failure Isolation

AI never becomes the source of truth. Permissions are evaluated before retrieval. Uploaded documents/messages are untrusted. Low-confidence results require uncertainty/manual review. Financial, access, approval and compliance mutations continue through deterministic workflows. Core operations work if the AI provider is unavailable.

16.9.2 API and Schema Versioning

API contracts must support backward compatibility and explicit deprecation. Database migrations must be forward-safe where possible, and mobile/offline clients must tolerate server evolution.

Real-world scenario — offline app after release: a guard tablet reconnects two weeks after an application update. The server receives an older sync schema. Compatibility logic validates and migrates the payload or rejects it with a recoverable error; it must not partially apply unknown fields.

- Use explicit API versions for breaking changes.
- Contract tests protect mobile/server compatibility.
- Database migrations are ordered, observable and reversible where technically possible.
- Feature-flag/configuration migrations must not leave orphaned states.

16.9.1 Observability and Operational Control Plane

Production observability must cover API health, database health, queues, scheduled jobs, notification delivery, payment callbacks, sync conflicts, biometric failures, storage, integration latency, suspicious activity and feature-flag/configuration changes.

Real-world scenario — notification queue backlog: payment reminders are delayed because a provider is throttling requests. Operations should see queue depth, retry rate, dead-letter volume and affected notification types without confusing this infrastructure problem with payment failure.

- Every cross-service operation carries a trace/correlation identifier.
- Metrics must be tied to business outcomes where possible, not only CPU/memory.
- Dead-letter items require retry/replay controls with authorization and audit.
- Alert thresholds should distinguish transient spikes from sustained incidents.

16.8 AI Guardrails

Smart Lost & Found, Community Carpool, Festival/Event Automation, Society Micro-Commerce, Domestic Worker Replacement Network, Pet Management, Classifieds, Guest Stay Management, Housekeeping Quality, Emergency Local Network, intercom replacement, water, electricity, solar, EV charging and utility connectors are configurable extensions.

16.7 Community Intelligence and Utility Extensions

AI Society Manager; Society Intelligence Engine; Society Digital Twin; Society Memory/Knowledge Graph; Predictive Maintenance; AI Financial Advisor; Vendor Intelligence; Smart Complaint/Incident Intelligence; Community Health/Anomaly Detection; No-Code Automation; Smart Security; Emergency Command Center; and resident-facing timeline/personal-assistant capabilities are planned as downstream intelligence services.

16.6 Enterprise Intelligence Portfolio

A feature is production-ready only after data model, authorization, migrations, observability, failure handling, feature-flag paths, acceptance tests, security review and operational documentation are complete.

17.4 Production Activation Gate

MVP: setup, units, onboarding, visitor/gate, billing, complaints, notices, staff, documents, notifications, feature flags and offline foundations. Phase 1A: Resident Connect, history, move lifecycle, archive, parcel handover, inter-flat issues, lost & found. Phase 2: NOC, parking, facilities, vendors/AMC, assets, inventory, renovation, governance, compliance, housekeeping and advanced reporting. Phase 3: biometric, workforce automation, senior care, emergency volunteers, marketplace, commerce, carpool, events, payroll readiness and Health Score. Phase 4: RFID, ANPR, boom barriers, CCTV, smart meters, solar/energy, EV, smart automation, e-voting and advanced connected devices.

17.3 Consolidated v4 Roadmap

Test concurrency, idempotency, offline replay, RLS isolation, backup/restore, queue recovery, circuit-breaker opening/closing, document checksum verification, audit-chain continuity, low-memory devices, regional formatting and deterministic report reconciliation.

18.3 Additional Non-Functional Acceptance

TC-SEC-01 resident denial locks admission; TC-FIN-01 reversal leaves original ledger unchanged and balances; TC-NOC-01 outstanding dues return deterministic 422 and block NOC; TC-SYNC-01 200 offline visitors reconcile with zero loss/duplicates; TC-DPDP-01 eligible erasure scrubs personal data while preserving legitimately retained records; TC-HARD-01 low-memory device avoids OOM; TC-SEC-02 revocation race flags unauthorized entry; TC-CHAT-01 first-contact approval masks phone; TC-PRIV-01 later occupant cannot retrieve prior occupant KYC/private history; TC-FLAG-01 disabled feature rejects backend call; TC-AI-01 AI recommendation is permission-filtered and cannot mutate authoritative state.

18.2 Expanded End-to-End Tests

Society OS should make a society easier to operate, easier to govern and easier to live in without sacrificing privacy, financial integrity, physical security, transparency or traceability. The moat is one trusted

operational system in which people, units, money, documents, security, services, governance, assets, utilities and intelligence share a consistent foundation.

19.7 Final Product Principle

Provision PostgreSQL with RLS and migrations; scaffold Flutter/React Native with encrypted local storage; implement NestJS or Go API/service boundaries; publish OpenAPI contracts; implement queues/outbox and connector circuit breakers; create state-machine tests; run cross-society isolation tests; test low-end tablets; validate privacy/retention policies; complete end-to-end acceptance matrix.

19.6 Implementation Checklist

Never execute an unscoped tenant mutation. Reject null societyId. Never UPDATE/DELETE posted ledger_transactions or ledger_lines. Use client UUID idempotency for gate/attendance events. Put external integrations behind circuit breakers. Keep offline local writes transactional. Hash document bytes where cryptographic proof is required. Never expose historical tenant KYC through current occupancy. AI cannot bypass permissions, workflow state, approvals or financial controls.

19.5 Master AI / Developer Guardrails

1. Executive Summary

Society OS is a configurable operating platform for Indian residential communities. It is designed to support the full lifecycle of a society—from initial setup and resident onboarding through daily security, billing, complaints, facilities, staff operations, governance, compliance, asset management, communication and community services.

The product is intentionally broader than a basic visitor-and-maintenance application. It is designed to become the operational backbone of a society while remaining modular enough for a small standalone building and powerful enough for a large township. The same core architecture should support different society structures, policies, workflows, charge models, approval chains and integrations without requiring a separate product codebase.

The platform combines structured operational records with configurable workflows, secure documents, notifications, auditability, analytics and an intelligent automation layer. The intelligence layer is intended to understand patterns, explain information, assist with drafting and recommend actions; authoritative financial, access, approval and audit state remains controlled by deterministic application logic.

1.1.1 What “Complete” Means in This Blueprint

Completeness is defined operationally, not cosmetically. Every major capability is described through its business purpose, normal path, exception path, data that changes, audit implications, user-visible outcome, and testable acceptance behaviour. This makes the document usable by engineering, product, QA, implementation teams and management without relying on unstated assumptions.

Real-world scenario — Monday morning society operations: at 8:45 AM, a resident reports a water leak, a tenant has a move-in scheduled, a delivery rider arrives at the gate, an AMC vendor misses a scheduled visit, and a monthly maintenance payment fails. Society OS must treat these as separate domain events while keeping the shared unit, person, access, finance, notification and audit records consistent.

1.1.2 Pitch Narrative: Why Society OS Is More Than a Resident App

Society OS is positioned as an operating system for residential communities: a common operational record for property structure, people, access, money, work, documents, communication, compliance and community services. The pitch is therefore not “another society app”; it is a configurable operational platform that reduces manual coordination and fragmented records.

1.1 Product Scope

- Society and property master data, units, occupancy, ownership and resident lifecycle.
- Visitor, gate, delivery, vendor, staff and material movement management.
- Maintenance billing, collections, receipts, accounting records, budgets and reconciliation readiness.
- Complaints, helpdesk, SLA management, escalation, evidence and resolution workflows.
- Facilities, amenities, reservations, deposits, inspections and usage management.
- Parking, vehicles, violations, visitor parking and integration readiness.
- Resident communication, controlled messaging, announcements, community groups and privacy controls.
- Inter-flat issues, mediation, evidence and resolution tracking.
- Governance, meetings, notices, resolutions, polls, elections and e-voting where configured.
- Safety, emergency response, senior citizen support, incident management and local emergency coordination.
- Staff, domestic help, attendance, shift management, verification, payroll readiness and workforce operations.
- Vendors, AMC, assets, preventive maintenance, inventory and procurement workflows.
- Document vault, compliance records, certificates, NOCs and secure access.
- Marketplace, community classifieds, local services, carpool and community commerce.
- Water, electricity, energy, solar, EV charging and utility-management readiness.
- Smart automation, analytics, intelligence, knowledge retrieval and future connected-device integrations.

1.2 Supported Society Models

Society model	Product approach
Standalone building	Simple configuration with the same core data model; unnecessary modules remain disabled.
Cooperative housing society / CHS	Governance, billing, records, notices, compliance and committee workflows are supported.
Gated apartment community	Multiple towers, gates, parking areas, amenities, vendors and operational teams are supported.
Villa / bungalow community	Unit hierarchy, visitor management, utilities, security and community services remain configurable.
Mixed-use complex	Residential and commercial units can coexist with separate rules and charge structures.
Large township	Multiple phases, towers, zones, gates, amenities, staff teams and operational dependencies can be represented.

1.3 Core Product Promise

Promise	Meaning
Simple	Frequent society tasks should be quick, understandable and consistent.
Configurable	The product adapts to society-specific rules instead of forcing one operating model.
India-first	UPI, cash/cheque, regional language readiness, local verification workflows and Indian society processes are native considerations.
Privacy-first	Sensitive resident, tenant, financial, communication and historical data is protected by default.
Traceable	Important transactions and decisions are auditable from initiation through closure.
Operationally complete	The product should reduce dependency on paper registers, fragmented spreadsheets and unmanaged messaging groups.
Future-ready	Integrations and smart capabilities are added through controlled interfaces rather than hard-coded dependencies.

2. Product Vision and Principles

2.1 Mission

To simplify, digitize and continuously improve the daily operations of Indian residential communities through a configurable, trustworthy and easy-to-use platform.

2.2 Vision

To become the operating system for Indian housing communities—covering people, property, money, security, services, governance, compliance, assets and community life—while creating a strong foundation for automation, connected devices and intelligent decision support.

2.3 Product Philosophy

Principle	Product interpretation
Convenience first	High-frequency actions should require minimal effort and provide clear confirmation.
Configurable, not forced	Societies should enable only what they need and define their own rules.
Transparency by default	Bills, complaints, approvals, attendance, NOCs, governance decisions and operational changes should leave a traceable record.
Privacy by design	Access is granted by explicit permission and data sensitivity rather than by convenience.
Deterministic core	Money, permissions, access, workflow state and audit records are controlled by application logic.
Automation with control	Automation can reduce repetitive work but must respect approvals, exceptions and audit requirements.
Low-connectivity readiness	Critical operational workflows should degrade gracefully and synchronise when connectivity returns.
India-first operations	Payment methods, society practices, verification, local vendors and regional language considerations are treated as first-class requirements.
Modular engineering	Domains remain independently configurable with clear interfaces and bounded responsibilities.
Data as a product asset	Clean historical records enable reporting, automation, forecasting and future intelligence.

2.4 Product Differentiation

- Complete operational coverage rather than a single-purpose visitor or billing tool.
- Strong owner/tenant lifecycle history and controlled historical document archive.
- Mandatory move-out clearance and NOC framework with configurable exceptions.
- Privacy-first resident communication that does not require exposing personal phone numbers.
- Workforce and biometric attendance readiness for outsourced security and housekeeping operations.
- Society-wide asset, AMC, inventory and preventive-maintenance management.
- Community utility layer covering marketplace, classifieds, carpool, events and local services.
- Utility and sustainability layer covering water, electricity, solar, EV and energy intelligence.
- Intelligence layer designed around the society's own structured records, rules and history.
- Subscription-ready architecture without forcing monetisation into the initial product experience.

2.4.1 Localization and Accessibility Principles

Society OS should be usable by residents, senior citizens, guards and staff with different language, literacy, device and accessibility needs. Localization must cover English, Hindi, Marathi, Kannada, Tamil, Telugu, Bengali, Gujarati, Malayalam, Punjabi and future languages through externalized content.

Real-world scenario — senior citizen receives an emergency notice: the notification should be understandable without relying on tiny text or color-only indicators. Where supported, audio/local-language assistance can improve comprehension.

- Use scalable typography and large touch targets.
- Provide high contrast and non-color-only status indicators.
- Support screen-reader semantics and keyboard navigation on web.
- Externalize notification/message templates and terminology for localization.
- Use reduced motion preferences where supported.

3. User Personas and Applications

Society OS is a multi-party operational platform. Different participants have different responsibilities, but the product is designed around a single shared society data model and consistent governance, security and audit principles.

3.1 Personas

Persona	Primary context
Resident owner	Lives in or owns a unit and participates in day-to-day society life, payments and governance.
Tenant	Occupies a unit under an authorised tenancy and interacts with society processes during occupancy.
Family member	Participates in household activities under controlled association with the unit.
Security guard / supervisor	Performs gate, visitor, incident and operational security activities.
Facility manager / supervisor	Coordinates complaints, staff, vendors, assets, inspections and day-to-day operations.
Committee / office bearer	Oversees governance, approvals, notices, compliance and society administration.
Treasurer / accountant	Maintains financial operations, billing, collections, expenses, budgets and records.
Vendor / service workforce	Performs authorised society services, contracts, maintenance and operational work.
Super administrator	Maintains platform-level configuration, onboarding, support and subscription-ready controls.

3.2 Application Ecosystem

Application surface	Purpose
Resident mobile experience	Primary digital interaction layer for residents and household members.
Security / gate experience	Fast operational interface for physical entry, visitor and incident workflows.
Administrative web experience	Central operational workspace for configuration, records, approvals, reporting and governance.
Facility operations experience	Focused operational workspace for maintenance, assets, workforce, vendors and incidents.
Platform administration experience	Cross-society configuration, onboarding, support and platform controls.

Scope boundary This section intentionally defines the product ecosystem without prescribing which individual feature belongs to which application or role, and without defining screen-by-screen layouts.

4. Complete Feature Catalogue

The catalogue below consolidates the product scope into a delivery-oriented feature inventory. Phases indicate intended maturity rather than a promise that every feature must be enabled for every society.

Feature area	Functional scope	Phase
Rental / Tenant Assessment	Explainable rental/tenant readiness assessment using configurable verification, document completeness, agreement and operational criteria; no opaque profiling.	1A
Society & Unit Setup	Society hierarchy, phases, towers, blocks, wings, floors, units, unit types, areas, occupancy, billing attributes and configuration.	MVP
Owner & Tenant Management	Current ownership, tenancy, family members, KYC, agreements, vehicles, pets, approvals and lifecycle status.	MVP
Owner/Tenant History	Historical ownership and occupancy timeline, previous resident records, archive and controlled historical references.	1A
Document Vault	Secure files, metadata, versions, expiry, verification, controlled access and audit history.	MVP
Move-in / Move-out	Onboarding, document collection, approvals, access activation/revocation, move logistics and clearance.	1A
Digital NOC & Certificates	No-dues, move-out, tenant, parking, renovation, residence and other configurable certificates.	2
Visitor & Gate Management	Guest, delivery, cab, vendor, staff, material passes, QR/OTP, gate logs and offline operation.	MVP
Staff & Domestic Help	Guards, housekeeping, maids, drivers, cooks, technicians and other workforce records, verification and shifts.	MVP
Biometric Attendance	Device integration, punch sync, shift calculation, corrections, reports and vendor invoice verification.	3
Maintenance Billing & Accounting	Charge heads, recurring bills, penalties, UPI, cash/cheque, receipts, ledger, reconciliation, budgets and exports.	MVP/2
Complaints & Helpdesk	Tickets, categories, priorities, evidence, SLA, assignment, escalation, vendor work and feedback.	MVP
Facility & Amenity Booking	Clubhouse, gym, pool, sports courts, halls, guest rooms, deposits, QR check-in, inspection and refunds.	2
Parking & Vehicles	Vehicle master, allocation, visitor parking, violations, blocking workflow and RFID/ANPR readiness.	2
Resident Connect	Privacy-first messaging, first-contact approval, controlled group communication, announcements and moderation.	1A
Privacy Directory	Configurable visibility, opt-out and privacy-aware resident discovery.	1A
Inter-flat Issues	Leakage, noise, renovation disturbance, parking, pets, damage claims, evidence and mediation.	1A
Governance	AGM/SGM, agenda, RSVP, proxies, minutes, resolutions, polls, elections and optional e-voting.	2/4
Safety & Emergency	SOS, medical, fire, lift, water/electrical	2/3

	incidents, volunteers, senior support and incident timeline.	
Vendor, AMC & Assets	Vendor records, contracts, AMC, warranties, assets, service history, inspections and scorecards.	2
Inventory & Procurement	Consumables, stock levels, issue/return, purchase requests, approvals, reorder levels and vendor linkage.	2
Marketplace & Community Commerce	Approved service directory, local services, micro-commerce, community deals and future verified partners.	3
Community Services	Lost & found, carpool, classifieds, domestic-worker replacement network, pet services and local utility services.	3
Events & Festival Automation	Events, RSVPs, contributions, volunteer coordination, reminders, notices and recurring festival workflows.	3
Water Management	Meter/readings readiness, tanker tracking, consumption, leakage incidents, maintenance and conservation reporting.	3/4
Energy & Sustainability	Electricity tracking, solar generation, common-area consumption, EV charging and sustainability metrics.	4
Legal & Compliance	Compliance register, document expiry, statutory records, notices, action tracking and renewal reminders.	2/3
Security Workforce Management	Shift rosters, attendance, post/beat assignment, deployment records, incident logs and vendor verification.	2/3
Payroll Readiness	Staff/vendor workforce attendance inputs, pay-period summaries, deductions/adjustments readiness and exports.	3
Intercom Replacement	In-app emergency and service communication workflows replacing fragmented intercom dependency.	3
Guest Stay Management	Guest room/stay requests, approvals, deposits, occupancy records and checkout.	2
Housekeeping Quality	Checklists, inspection scores, recurring tasks, issue logging and vendor/service quality trends.	2/3
Regional Localization	Language-ready content, configurable labels, date/time/number formats and region-specific communication.	MVP/1A
Offline Support	Offline-safe critical operations with queued actions, sync status and conflict handling.	MVP/2
Smart Automation	Notice drafting, document search, bill explanation, complaint routing, meeting summaries and operational recommendations.	4
Intelligent Society Insights	Pattern detection, anomaly identification, risk alerts, forecasts and explainable recommendations.	4
Hardware Integration Layer	Biometric devices, RFID, ANPR, CCTV, smart meters, EV chargers, boom barriers and IoT-ready connectors.	3/4

Commercial direction The launch experience remains free for selected societies. Subscription, pricing, trial and upgrade mechanics remain hidden until monetisation is intentionally activated.

5. Architecture and Data Flow Diagrams

The architecture is intentionally layered. Society OS should separate presentation, domain services, persistence, integrations, notifications, document storage, analytics and future intelligence. This keeps operational workflows reliable while allowing advanced capabilities to evolve independently.

5.1 High-level Architecture

Figure 1. High-level Society OS architecture.

5.2 Context-level Data Flow

Figure 2. Context-level data flow and external integration boundaries.

5.3 Level-1 Data Flow

Figure 3. Level-1 data flow showing core processing and data stores.

5.4 Architectural Principles

- API-first domain services with clear ownership of business rules.
- Multi-tenant isolation enforced server-side on every business request.
- Relational transactional storage for authoritative operational records.
- Object storage for documents with controlled access and signed/authorised retrieval.
- Event/outbox pattern for reliable notifications and external integrations.
- Background workers for large imports, document indexing, attendance synchronisation, reports and scheduled automation.
- Integration adapters for payment gateways, messaging providers, biometric devices, RFID/ANPR, utility data and future IoT.
- Feature flags and configuration evaluated consistently across the product and backend.
- Audit events generated for sensitive mutations and configuration changes.
- Intelligence services remain downstream of authoritative data and cannot bypass domain authorization.

5.5 Reliability Model

Concern	Design requirement
Transaction integrity	Critical changes are committed atomically and remain traceable.
Idempotency	Callbacks, imports and retried commands use deterministic idempotency keys.
Async processing	Long-running work is queued and monitored rather than blocking transactional APIs.
Offline operation	Critical field workflows can queue events locally and reconcile later.
Failure isolation	An unavailable third-party provider must not make unrelated core functions unavailable.
Observability	Every integration and scheduled job exposes status, failure reason, retry state and audit context.

6. Role-based Permission Model

Role-based access control is a platform security foundation. The purpose of the model is not to document screen visibility; it is to establish a consistent authorization boundary around society data, actions and sensitive records.

6.1 Authorization Layers

1. Authenticate the requesting identity and validate the active session.
2. Resolve the society and business context from trusted server-side data.
3. Evaluate role and granular permission claims.
4. Evaluate feature flag and society configuration.
5. Evaluate entity relationship and data sensitivity.
6. Validate workflow state and whether the requested transition is legal.
7. Apply approval requirements where configured.
8. Record an audit event for sensitive operations.

6.2 Permission Dimensions

Dimension	Examples
Tenant / society scope	Society, phase, tower, unit and operational entity boundaries.
Action permission	Create, view, update, approve, reject, publish, export, correct, close or administer.
Data sensitivity	Public, resident-visible, unit-restricted, staff-restricted, financial, security, personal-sensitive and historical.
Workflow state	A request may be editable in draft but immutable after approval or accounting lock.
Relationship	Ownership, occupancy, assignment, operational responsibility or explicit administrative authority.
Feature state	Disabled capability must be blocked regardless of UI visibility.
Approval authority	Actions may require one or more configured approvals.
Audit requirement	Sensitive actions must produce immutable audit records.

6.3 Sensitive Data Classes

- Identity and contact information.
- Owner, tenant and historical occupancy records.
- Agreements, KYC, police verification and sensitive documents.
- Financial transactions, bills, receipts and ledgers.
- Private communication metadata and reported content.
- Security incidents, watchlists and controlled camera evidence.
- Attendance and workforce records.
- Emergency and senior-support information.
- Legal and compliance records.

Security rule Frontend restrictions are for usability only. The backend remains the authoritative enforcement point for society scope, role, permission, feature state, data sensitivity and workflow state.

7. Deep Functional Specification by Module

Each module below defines business purpose, functional scope, business rules and implementation guidance. The intent is to provide a stable product contract without coupling the specification to a particular screen design.

7.1 Society, Unit and Resident Foundation

This is the master-data backbone. Every operational record must ultimately resolve to a society and, where applicable, a unit or property entity.

Key capabilities

- Society hierarchy: society, phase, tower, block, floor, wing, flat, villa, shop or office.
- Unit attributes: area, unit type, occupancy, billing category, parking linkage and status.

- Resident profile with ownership/tenancy relationship, family association and lifecycle state.
- Vehicle, pet, emergency contact, KYC and rule-acknowledgement records.
- Bulk import with validation, duplicate detection, preview, error reporting and rollback.
- Configurable unit numbering and hierarchy rules.

Implementation guidance

Unit is the central business entity. All applicable transactions should carry a strong society scope and unit relationship so reporting, security and lifecycle history remain consistent.

7.1.4 Family and Household Management

Household membership is a dated relationship model. Support owner, co-owner, tenant, spouse, child, parent, dependent and other authorized member relationships, along with emergency contact, vehicle, pet, visitor and facility associations where configured.

Real-world scenario — elderly parent moves in: the parent is added as an authorized household member with an effective date. Emergency and communication preferences can be configured without changing the legal ownership of the unit.

- Every relationship has start/end semantics.
- Household membership must not automatically imply access to financial or sensitive records.
- Changes to household composition should trigger only the notifications and access recalculations defined by policy.

7.1.3 Resident Registration, Verification and Account Recovery

Resident onboarding must support invitation, self-registration where enabled, OTP/mobile or email verification, profile completion, unit association, owner/tenant/family relationship, document submission, verification, approval, activation, suspension and recovery. Registration state must be separate from occupancy state.

Real-world scenario — duplicate registration: a tenant receives an invitation and also registers manually using the same phone number. The system should identify a possible duplicate, require confirmation/verification, and avoid creating two active identities for the same person without an explicit business reason.

- Verification attempts and failures are rate-limited and auditable.
- Account recovery must revoke compromised sessions/tokens where appropriate.
- Suspension disables configured access without deleting historical relationships.
- Identity matching must be conservative; false-positive merges are more damaging than a duplicate review case.

7.1.2 Property Master: Full Hierarchy and Lifecycle

The property master is the backbone for every other domain. It must model Society → Phase → Tower/Wing → Floor → Unit and independently track parking, storage, common areas, facilities, utility areas, meters and shared infrastructure. A unit may be a flat, villa, plot, shop or another configured property type.

Real-world scenario — redevelopment or renumbering: a society changes Tower B unit numbering during redevelopment. Historical invoices, complaints and visitor records must continue to resolve to the original identity while the current display name reflects the new numbering. The system must use stable identifiers rather than relying only on human-readable unit numbers.

- Never use a mutable unit label as the primary identity.
- Property hierarchy changes require effective dates and audit history.
- Common assets and facilities may belong to the society or to a defined property node.
- Property deactivation must not cascade-delete financial, legal or historical records.

7.1.1 Society Onboarding and Initial Configuration

Onboarding is the first production workflow and must be treated as a controlled configuration process rather than a simple registration form. A society record becomes operational only after its legal identity, geographic settings, property structure, billing rules, operating policies and initial data have passed validation.

Real-world scenario — a 6-tower apartment society: the implementation team creates the society, selects Maharashtra and IST, configures the financial year, adds six towers with 24 floors each, defines flat sizes, imports 1,152 units, maps parking slots, configures maintenance charge heads and late-fee rules, loads emergency contacts, and validates imported owners before activation.

Failure scenario — incomplete property master: if 1,152 units are imported but 37 units have duplicate numbers within the same tower, activation must stop or require an explicit correction workflow. The platform must never silently overwrite one unit with another.

- Capture society name, legal/registration identifiers, address hierarchy, pincode, timezone, currency, financial year and society type.
- Define property hierarchy: Society → Phase → Tower/Wing → Floor → Unit, with optional villas, plots, shops and shared/common infrastructure.
- Configure parking inventory, common areas, facilities, meters, charge heads, billing frequency, late fees, payment methods, notification defaults and operational policies.
- Support initial import with validation, preview, duplicate detection, business-rule validation, error report and controlled activation.
- Preserve configuration history so changes to numbering, charges or structure never destroy historical references.

7.2 Visitor and Gate Management

Digitises physical entry operations while keeping the process fast and auditable.

Key capabilities

- Guest pre-approval with QR/OTP.
- Walk-in visitor registration and resident verification.
- Delivery, cab, vendor, staff and material movement records.
- Gate-wise logs, entry/exit timestamps and purpose.
- Watchlist/blacklist with controlled escalation.
- Offline entry and later synchronisation.
- Visitor parking and temporary vehicle association.

Implementation guidance

The entry workflow must remain resilient on low-end devices and unstable networks. Duplicate events and sync conflicts must be safely handled.

7.2.1 Everyday Gate Scenarios and Exception Handling

Gate management must work for the messy reality of residential security: pre-approved guests, unknown visitors, delivery agents, domestic workers, service technicians, cabs, school transport, emergency responders, recurring visitors and residents returning after a long absence.

Real-world scenario — pre-approved guest: a resident creates a visitor invitation for Saturday 7:00–10:00 PM. The visitor arrives at 7:35 PM, the guard verifies the pass and identity, records the vehicle, and admission is logged. If the visitor leaves at 9:10 PM, the visit is closed and the system can show who is currently inside the premises.

Real-world scenario — resident unreachable: a plumber arrives for Unit B-804 without pre-approval. The resident does not answer. The system must not convert “resident did not answer” into automatic admission. The configured exception path may allow a temporary request/approval process, but final admission remains auditable.

Real-world scenario — emergency: an ambulance arrives without normal visitor credentials. Emergency policy may bypass ordinary approval checks, but the bypass reason, gate, time, operator/device and resulting access event must be captured.

- Visitor types should be configurable and not hard-coded.
- Every visit must have explicit entry and exit semantics; current occupancy must be derivable from valid events.
- Offline gate events require device timestamps, idempotency keys and deterministic reconciliation when connectivity returns.
- Vehicle details, recurring visitor patterns, blacklists and exceptions must be handled as policy-controlled data rather than ad hoc notes.
- Gate workflows should support camera/OCR assistance without making OCR correctness a prerequisite for safe operation.

7.3 Maintenance Billing, Payments and Accounting

Provides an auditable financial foundation for recurring society operations.

Key capabilities

- Recurring monthly bills and ad-hoc charges.
- Fixed, area-based, meter-based, parking, facility and penalty charge heads.
- UPI/payment gateway support and manual cash/cheque recording.
- Receipts, unit ledger, outstanding balance and defaulter reporting.
- Bank reconciliation readiness and accounting exports.
- Budget versus actual and fund-wise reporting.
- Correction/reversal instead of destructive deletion.

Implementation guidance

Resident maintenance billing must remain logically separate from SaaS subscription billing. Financial records should be append-corrected and never silently deleted.

7.3.2 Financial Exception Catalogue

Financial workflows must explicitly define exceptions instead of treating them as generic errors. Important cases include partial payments, advance payments, credit balances, waivers, discounts, late fees, interest, bounced cheques, failed UPI payments, duplicate callbacks, refunds, reversals, charge corrections and unmatched bank transactions.

Real-world scenario — advance payment: a resident pays ₹20,000 when only ₹15,000 is due. The ₹5,000 surplus becomes a controlled credit/advance according to configuration; it is not silently discarded or incorrectly posted as revenue.

Real-world scenario — bounced cheque: a cheque initially marked received later bounces. The system reverses the settlement effect through an auditable financial transaction and restores the outstanding balance plus configured charges.

- Never edit a posted financial journal to correct history.
- All adjustments require reason, actor, timestamp and supporting evidence where policy requires.
- Reconciliation exceptions must be visible until resolved.
- Receipt numbering and reversal relationships remain traceable.

7.3.1 Complete Billing-to-Ledger Business Lifecycle

Finance is the system-of-record domain and must be described at transaction level. The product must distinguish an invoice, a payment attempt, a settled payment, a receipt, an allocation to an invoice, a ledger posting, an adjustment and a refund. These are related but not interchangeable concepts.

Real-world scenario — normal monthly billing: on the first day of the billing cycle, the engine calculates maintenance, parking, water and sinking-fund charges for each unit. Validation completes, invoices are

generated and published, residents are notified, payments are received, receipts are issued and balanced ledger entries are posted.

Real-world scenario — partial payment: a unit owes ₹12,500 and pays ₹7,500. The system records the actual payment, allocates it according to configured allocation rules, leaves the remaining ₹5,000 outstanding, and recalculates ageing without pretending the invoice was fully settled.

Real-world scenario — duplicate gateway callback: the payment provider retries the same callback after a network timeout. Signature verification and idempotency must cause the second callback to become a no-op from the ledger perspective while retaining an audit record of the duplicate event.

Real-world scenario — refund after reconciliation: a ₹10,000 payment is later refunded by ₹2,000. The original posted transaction remains immutable; a compensating/refund transaction records the reversal and the unit balance changes deterministically.

- Support fixed, area-based, usage-based and configurable charge heads, including maintenance, parking, water, electricity/common-area recovery, funds, penalties, interest, amenity charges, move-in/out and miscellaneous charges.
- Support partial payments, advances, credits, waivers, discounts, penalties, failed payments, bounced cheques, refunds, reversals and manual adjustments with approvals where configured.
- Use transaction serialization and balanced journal posting; never resolve financial conflicts with last-write-wins.
- Bank reconciliation must distinguish unmatched, partially matched, matched, reversed and exception transactions.
- Invoice numbering, tax configuration, rounding and currency precision must be configurable and jurisdiction-aware rather than hard-coded.

7.3.7 Financial Reporting and Audit Traceability

Every financial KPI, statement and report must trace to source transactions, period, account and unit where applicable. Users must be able to explain why an amount changed without relying on manually edited history.

7.3.6 Financial Integrity Invariants

Posted financial journals are balanced and immutable; corrections are represented by reversal/adjustment transactions; callbacks are idempotent; refunds reference their source payment; partial payments retain residual outstanding; credits retain source provenance; effective-dated configuration determines calculations; reconciliation cannot duplicate settlement; month-end controls prevent uncontrolled back-posting.

7.3.5 Financial Exception Catalogue

The product must explicitly distinguish failed payment, unpaid invoice, duplicate callback, partial payment, advance/credit, refund, reversal, bounced cheque, late fee, interest, waiver, discount, credit/debit adjustment, unmatched bank movement, disputed transaction, failed refund and period-close exception.

7.3.4 Financial End-to-End Lifecycle

Billing configuration -> billing run -> charge calculation -> validation -> invoice generation -> publication -> notification -> payment initiation -> provider confirmation -> signature/idempotency validation -> allocation -> receipt -> ledger posting -> reconciliation -> ageing and collections -> adjustments/refunds/reversals -> period close.

7.3.3 Financial Product as the System of Record

Finance is a first-class operational subsystem. Charges, invoices, payments, receipts, credits, waivers, adjustments, refunds, reversals, payables, reconciliation and close controls must be explainable from source records and ledger effects.

7.4 Complaints and Helpdesk

Converts informal complaints into structured, owned and measurable work.

Key capabilities

- Complaint categories, priorities, evidence and affected location.
- Assignment to internal teams or approved vendors.
- SLA start, pause, escalation, breach and resolution.
- Resident confirmation, feedback and reopen.
- Private/sensitive complaint mode with restricted visibility.
- Recurring issue correlation and parent incident capability.

Implementation guidance

Every complaint should have status, owner, timestamps, SLA, audit trail and notification rules. Sensitive complaints require stricter data visibility.

7.4.2 Complaint Intake Channels and Duplicate Handling

Complaints may originate from resident apps, staff, imports, integrations or operational teams. Regardless of channel, the resulting record follows one canonical lifecycle and source attribution.

Real-world scenario — duplicate complaint: a resident reports a broken streetlight at 8:00 PM and another resident reports the same pole at 8:05 PM. The second ticket can be linked to the parent incident while retaining its own reporter, acknowledgement and communication history.

- Preserve original source and submitted timestamp.
- Duplicate linking must never delete the reporter's original evidence.
- Parent incidents may broadcast common updates while individual records retain unit-specific communication.

7.4.1 Complaint Lifecycle, SLA and Escalation

A complaint is a service workflow, not merely a message. It must carry enough context to identify what failed, where it failed, how urgent it is, who owns it, what evidence exists, what SLA applies, what actions were taken and why it was finally closed.

Real-world scenario — lift complaint: a resident reports that the lift stops at the 7th floor and uploads a video. The ticket is classified as Lift → Operational Failure, marked high priority, assigned to the lift AMC workflow, acknowledged, worked, placed on hold while a spare part is sourced, resumed after the part arrives, resolved with service evidence and finally closed after the configured closure rule.

Real-world scenario — recurring complaint: ten residents report “water pressure low” in the same tower within 20 minutes. The system should detect possible correlation and allow a parent incident/outage record, preventing ten independent technician visits while preserving each resident complaint for communication and history.

Real-world scenario — reopen: a ticket is closed but the resident reports that the issue persists the next morning. Reopening must create a new state transition and preserve the original closure evidence rather than editing history.

- Lifecycle: CREATED → CLASSIFIED → ASSIGNED → ACKNOWLEDGED → IN_PROGRESS → WAITING/HOLD → RESOLVED → REOPENED → CLOSED.
- Capture category, subcategory, location, evidence, priority, SLA, assignment, escalation, comments, internal notes, root cause and closure reason.
- Support reassignment, duplicate detection, parent incidents, recurring complaints and SLA breach escalation.
- Keep resident-facing communication separate from internal operational notes while preserving auditability.
- Closure should be evidence-based and measurable; “technician marked done” is not automatically equivalent to “service outcome verified”.

7.5 Facility and Amenity Booking

Manages shared resources with capacity, schedule and financial controls.

Key capabilities

- Configurable facilities, operating hours and blackout periods.
- Slot availability and capacity rules.
- Free/paid bookings, deposits and refunds.
- Approval workflows where required.
- QR check-in/check-out.
- Damage deposit and post-use inspection.

Implementation guidance

Database-level constraints must prevent double booking. Booking state must drive payment, deposit and refund behaviour.

7.5.1 Booking Engine and Conflict Rules

Facility booking must be treated as a reservation engine with concurrency controls, not a calendar of informal requests.

Real-world scenario — clubhouse booking: a resident requests the clubhouse from 6:00–8:00 PM. The facility has a 30-minute cleanup buffer, a maximum capacity of 50 and a cancellation cutoff of 24 hours. A second request for 7:30–9:00 PM must be rejected or placed on a waitlist if overlap is prohibited by policy.

Real-world scenario — blackout period: the pool is unavailable for maintenance every Monday 6:00–10:00 AM. Booking requests during the blackout are rejected with a policy reason, not treated as an arbitrary application error.

Real-world scenario — no-show: a resident reserves a court but does not check in. The system records the no-show and can apply a configurable penalty or booking restriction without altering unrelated bookings.

- Facility master must define operating hours, holidays, capacity, slot duration, buffers, booking window, cancellation window, deposits, charges, penalties, approval requirements and blackout periods.
- Confirmed reservations must use atomic conflict prevention where overlap is prohibited.
- Support recurring bookings, waitlists, cancellation, refund, no-show, check-in, usage and damage inspection where applicable.
- Booking changes must be versioned so concurrent updates cannot silently overwrite each other.

7.6 Parking and Vehicle Management

Maintains vehicle ownership and allocation while supporting future access automation.

Key capabilities

- Vehicle registration and verification.
- Parking slot allocation and reassignment.
- Visitor and temporary parking passes.
- Wrong parking, blocking and violation workflows.
- Repeat-offence escalation.
- RFID/ANPR readiness.

Implementation guidance

Parking slots are treated as managed assets linked to units and vehicles. Violation records remain controlled operational data.

7.6.2 Parking Master, Allocation and Enforcement Rules

Parking allocation must distinguish ownership/entitlement, physical slot, temporary use, vehicle association and actual occupancy. A slot can have a historical allocation chain while only one valid active allocation applies at a given time.

Real-world scenario — EV conversion: a resident requests an EV charging-enabled parking arrangement. The system records the vehicle's EV status, the slot capability and any required approval/charging configuration without changing the legal identity of the parking slot.

- Detect conflicting active allocations.
- Track temporary and visitor allocations with explicit expiry.
- Keep violation evidence separate from the permanent parking master.
- Support future RFID/ANPR integration without making hardware availability a prerequisite for manual operation.

7.6.1 Parking, Vehicle and Violation Operations

Parking is both a property master and an operational enforcement domain. The platform must distinguish a parking slot from the vehicle currently using it and from the historical allocation of that slot.

Real-world scenario — vehicle replacement: a resident sells MH-12-AB-1234 and buys MH-14-CD-5678. The new vehicle is linked to the same unit and parking slot while the old vehicle remains historically recorded as inactive; historical gate and parking events must continue to resolve correctly.

Real-world scenario — visitor parking abuse: a visitor vehicle occupies a reserved resident slot. Security records the incident with slot, vehicle, evidence and time. The system can trigger a violation workflow and configured penalty/towing readiness without changing the permanent parking allocation.

Real-world scenario — temporary allocation: a resident has a second vehicle for two weeks. The society grants temporary parking with a start/end date. The temporary allocation must automatically expire and must not overwrite the permanent slot ownership.

- Support covered/open, two/four-wheeler, reserved, visitor, EV, accessibility and temporary slots.
- Vehicle master should include registration, type, make/model, unit/person association, parking, RFID/ANPR identifiers, EV flag and active/inactive state.
- Prevent duplicate active vehicle registration where policy requires uniqueness; allow legitimate historical duplicates across time.
- Capture violations with evidence, status, resolution and optional penalty/towing workflow.

7.7 Resident Connect and Controlled Communication

Provides community communication without forcing residents to expose personal phone numbers.

Key capabilities

- First-contact request with subject and reason.
- Accept, reject, block and report controls.
- Persistent chat after acceptance.
- Controlled group communication.
- Announcements and notice acknowledgement.
- Moderation workflow for reported content.

Implementation guidance

Private communication should remain private by default. Moderation access must be exceptional, policy-driven and audited.

7.7.1 Communication as a Controlled Operational System

Communication must cover both broad announcements and privacy-sensitive person-to-person contact. The platform should make the safe path the easy path: users can communicate without exposing private phone numbers by default.

Real-world scenario — first-contact request: a committee member wants to contact a resident about a parking dispute. Instead of exposing the resident's phone number, the platform creates a contact request. The resident approves or declines; only then does the permitted communication channel open.

Real-world scenario — emergency broadcast: a major water pipeline failure affects Tower C. The society publishes a high-priority notice, pushes it to affected residents, records delivery/acknowledgement where supported and keeps the notice available for the duration of the incident.

Real-world scenario — moderation: a community message contains abusive content. Moderation actions must preserve evidence, identify the action taken and follow configurable retention and escalation rules.

- Support announcements, direct communication, groups, emergency broadcasts, delivery/read/acknowledgement states and channel fallback.
- Provide templates, localization, scheduling, expiry, withdrawal and revision history.
- Apply privacy boundaries to discovery, contact initiation, exports and notification payloads.
- Prepare integration boundaries for push, SMS, email and WhatsApp-like channels without coupling the domain model to a single provider.

7.8 Inter-flat Issues and Dispute Mediation

Creates a calm, evidence-based process for disputes that involve more than one property or resident.

Key capabilities

- Leakage, noise, renovation, parking, pets and damage cases.
- Evidence upload and affected-property tagging.
- Inspection and mediation assignment.
- Resolution proposal, acceptance and closure proof.
- Escalation and documented exceptions.

Implementation guidance

Dispute cases remain separate from normal helpdesk tickets because visibility, evidence and multi-party handling are more sensitive.

7.8.1 Dispute Case Lifecycle

Disputes require neutral case management rather than becoming unstructured chat threads. A case should capture subject, parties, property/context, evidence, chronology, communications, actions, mediation status, resolution and closure reason.

Real-world scenario — water seepage between flats: Unit A-701 reports seepage believed to originate from A-801. The case records evidence and inspection findings, creates tasks for maintenance/vendor inspection, allows controlled communication, and closes only after a documented resolution or formal outcome.

- Support evidence, comments, internal notes and controlled participant communication.
- Preserve an immutable case timeline.
- Allow escalation to configured governance/legal processes without exposing unrelated resident information.
- Dispute closure should record whether the outcome was resolved, mediated, rejected, withdrawn or unresolved.

7.9 Governance, AGM and Elections

Supports formal society governance and auditable decisions.

Key capabilities

- AGM/SGM notices, agendas, attendance and RSVP.
- Proxy authorisation and question submission.
- Minutes and resolution tracker.
- Polls and voting where enabled.
- Election nomination, candidate information and result publication.
- Decision archive and acknowledgement.

Implementation guidance

E-voting remains optional. Eligibility, quorum, voting rules and compliance assumptions must be configured before activation.

7.9.1 Governance and Decision Lifecycle

Governance features must support agenda preparation, meeting scheduling, notices, quorum/attendance capture, resolutions, voting, minutes, action items and archival of approved records.

Real-world scenario — AGM resolution: an agenda item proposes a major repair fund. The system publishes the notice, records eligible participation, captures the configured voting process, records the resolution outcome and links resulting actions to finance/procurement workflows.

- Governance records must be versioned and tamper-evident.
- Voting eligibility and quorum rules are configuration-driven and jurisdiction-aware.
- Approved minutes are not silently edited; corrections use versioned amendments.

7.10 Safety, Emergency and Senior Citizen Care

Provides a unified operational framework for urgent events.

Key capabilities

- SOS, medical, fire and lift-stuck alerts.
- Water/electrical emergency categories.
- Emergency volunteer coordination.
- Senior citizen support and check-in workflows.
- Incident timeline, acknowledgement and escalation.
- Post-incident review and evidence.

Implementation guidance

Critical alerts must not depend on ordinary notification preferences. Emergency workflows require reliable acknowledgement and escalation.

7.10.2 Incident, Evacuation and Post-Incident Review

Safety operations should support incident declaration, severity, affected zones, responders, tasks, evacuation/check-in information where applicable, communication, status updates and post-incident corrective actions.

Real-world scenario — basement flooding: water ingress is detected in the basement. The incident is declared, affected parking is marked unavailable, residents receive a notice, maintenance creates work orders for pumps and electrical inspection, and the final incident record contains response duration and corrective actions.

- Incident status must be independent from individual complaints that may be linked to it.
- Emergency instructions are versioned so later edits do not erase what was communicated during the incident.
- Critical actions require audit evidence.

7.10.1 Emergency Operations and Incident Command

Emergency handling must prioritize speed while preserving an auditable chain of events. Incidents should support declaration, severity, affected area, responders, communication, tasks, status updates, resolution and post-incident review.

Real-world scenario — fire alarm: an alarm is triggered in Tower D. The system records the incident, broadcasts an emergency notice, surfaces emergency contacts and operational tasks, tracks updates, and later records the post-incident report. Normal notification quiet hours must not suppress critical alerts when policy permits override.

- Emergency events must have higher delivery priority than routine notifications.

- Critical access bypasses must be explicitly reason-coded.
- Post-incident review should capture root cause, response time and corrective actions.
- Senior-citizen/emergency-care data must be purpose-limited and protected as sensitive information.

7.11 Vendor, AMC, Asset and Inventory Management

Manages the physical and contractual resources that keep the society running.

Key capabilities

- Vendor master and contract records.
- AMC dates, renewals, warranties and service history.
- Asset register with location, category, condition and lifecycle.
- Preventive-maintenance schedules and inspection checklists.
- Inventory stock, issue/return and purchase requests.
- Vendor performance scorecards.

Implementation guidance

Asset records should preserve lifecycle history. Vendor scoring should use objective operational data rather than opaque ratings.

7.11.3 Procurement Lifecycle and Three-Way Verification

Procurement should connect demand, approval, purchase order, receipt, service/inspection and invoice verification. Where applicable, finance should compare ordered quantity/value, received quantity/value and invoiced quantity/value before payment.

Real-world scenario — plumbing spares: a PO orders 100 valves. Only 90 arrive. The goods receipt records 90, inventory increases by 90, and an invoice for 100 cannot be treated as fully received without an exception or authorized variance.

- Support purchase requests, approvals, PO revisions, partial receipts, returns and cancellations.
- Link procurement lines to inventory items, assets/work orders and financial commitments.
- Keep supplier and contract history even after vendor deactivation.

7.11.2 Preventive Maintenance Engine

Preventive maintenance converts asset registers into planned work. Each asset may have one or more maintenance plans with frequency, due-date calculation, inspection checklist, SLA, assigned vendor/team, parts and cost capture.

Real-world scenario — diesel generator: the generator requires weekly visual inspection, monthly test run and annual servicing. The scheduler creates the correct work orders, tracks missed tasks, records readings and service evidence, and calculates the next due date from the completed event.

- Assets should support criticality, location, installation date, warranty, AMC, meter/readings and lifecycle state.
- Maintenance frequency can be calendar-based, usage-based or rule-driven.
- Missed maintenance must trigger escalation according to configured SLA.
- Completion must record evidence and next-due calculation.

7.11.1 Vendor-to-Work-Order-to-Payment Lifecycle

Vendor management must connect commercial records with operational service delivery. A vendor is not “active” merely because a profile exists; contracts, insurance, KYC, AMC dates, work orders, service evidence, invoices and performance history form one lifecycle.

Real-world scenario — lift AMC: an AMC expires on 31 March. Before expiry, the system creates a renewal reminder. After renewal, a scheduled preventive maintenance work order is created. The technician/vendor records inspection findings, replaces a component, uploads the service report, and submits an invoice. Finance verifies the work completion before payment.

Real-world scenario — stock-out during repair: a pump repair requires a spare that is below reorder level. The work order can create a material request, procurement can raise a PO, goods receipt increases stock, the spare is issued to the work order, and consumption is reflected in inventory and cost records.

Real-world scenario — poor vendor performance: a vendor repeatedly misses SLA. Performance scoring aggregates breach history, unresolved defects and response time, enabling renewal/penalty/termination decisions based on evidence.

- Vendor lifecycle: onboarding → KYC/docs → contract → AMC/PO → service → work order → inspection → invoice → verification → payment → performance → renew/terminate.
- Inventory lifecycle: item master → opening stock → purchase request → approval → PO → receipt → stock-in → issue → consumption → return/adjustment → reorder.
- Support batch, serial, expiry, damaged stock, stock locations, min/reorder levels and stock audit.
- Link asset, preventive maintenance plan, work order, spare consumption, vendor service and financial cost for traceability.

7.12 Marketplace and External Services

Creates a controlled community utility layer around approved local services.

Key capabilities

- Society-approved service directory.
- Cleaning, car wash, pest control, laundry, appliance services, pet care, tutors and elder support.
- Packers/movers, interiors and renovation services.
- Community classifieds and local service discovery.
- Micro-commerce / society marketplace for permitted products and services.
- Verified partner and payment readiness.

Implementation guidance

The marketplace should feel like a useful community utility rather than an intrusive advertising surface. Partner onboarding and commercial controls must remain configurable.

Society is the tenant root; unit identifiers are unique within the society hierarchy; occupancy periods are time-bounded; active overlap is rejected unless explicitly supported; unit status derives from authoritative occupancy. Bulk import follows validate → normalize → deduplicate → preview → transactional commit → audit/rollback.

Low-memory guard mode: release image buffers, suppress blur/shadows/animations, throttle camera capture and preserve text/OCR operations.

Tailgating/expiry: a cleared gate pass has a short configurable opening window (baseline example 90 seconds); if passage sensors fail, the pass does not remain indefinitely valid.

Delivery cubby protocol: “Drop at Gate” allocates a physical bin and a single-use pickup PIN delivered only to the intended unit recipient.

Resident revocation race: an offline guard admission is provisional; on reconciliation, authoritative server state and event ordering determine the outcome. If revocation precedes admission, the event becomes UNAUTHORIZED_ENTRY_FLAGGED and triggers configured security/resident alerts.

Figure 14. Deterministic visitor entry state machine.

7.2.1 Visitor State Machine

Every posted journal must balance debits and credits. Corrections use new compensating/reversal entries linked to the original. Charge rules store calculation inputs for reproducibility and bill explanation. Payment callbacks require signature verification and idempotency; cash/cheque entries remain traceable. Statutory fund/interest/export rules are configurable.

Complaint lifecycle supports created/classified/assigned/in-progress/resolved/confirmed/closed plus reopen and escalation exceptions. Duplicate/parent incident correlation can be intelligence-assisted but final state remains deterministic.

Database/service constraints prevent overlapping bookings. Deposits, payments, cancellations, refunds, inspection and damage claims follow booking state. Guest rooms are included alongside clubhouse, gym, pool, sports courts and halls.

Vehicle and slot records support temporary passes, wrong-parking/blocking incidents, repeat-offence escalation and RFID/ANPR adapters. Hardware absence must not stop manual parking operations.

First-contact requests capture subject/reason. Accept/reject/block/report controls are explicit. Phone numbers remain masked. Reported content can enter audited moderation; ordinary private communication is not exposed through administrative browsing.

Disputes are separate from ordinary complaints. Seepage, noise/vibration, renovation, parking, pets and damage can carry photos, documents, inspection notes and configured measurement evidence. Mediation proposals, acknowledgements and legal escalation remain time-stamped.

Governance covers AGM/SGM, agenda, RSVP, proxies, minutes, resolutions, polls, elections and decision archive. Smart Poll Engine may assist with poll drafting and response summarization, but eligibility, quorum, counting and final results are deterministic.

SOS, medical, fire, lift, water and electrical incidents use urgent acknowledgement/escalation. Emergency alerts bypass ordinary notification preferences. Senior support can include simple mode and configurable check-ins while preserving data minimization.

Vendor lifecycle includes verification, contract/AMC, service, inspection, invoice, scorecard and renewal. Assets preserve lifecycle history. Vendor Sinking Fund/Milestone Escrow links capital-project milestones, supporting documents, approvals and financial movements. Inventory supports reorder levels, purchase requests and issue/return.

Community services include cleaning, car wash, pest control, laundry, appliance service, pet care, tutors, elder support, packers/movers and renovation. Society Micro-Commerce supports approved products/services and payment readiness without turning the product into an intrusive ad surface.

7.12.3 Service Requests, Quotes, Orders and Refunds

External service transactions should use an explicit state model such as REQUESTED → QUOTED → ACCEPTED → SCHEDULED → IN_PROGRESS → COMPLETED → CANCELLED/DISPUTED. Commercial settlement should remain distinct from service completion.

Real-world scenario — appliance repair: a resident requests a service, receives a quote, accepts it, schedules a visit, the provider completes the work, the resident confirms completion and a payment is settled. If the provider does not arrive, the order enters an exception state rather than being marked completed.

- Cancellation windows, provider no-show, resident no-show and refund rules are configurable.
- Ratings are linked to completed orders and cannot be manipulated by creating fictitious service completions.
- Provider failures must not corrupt society-level finance or resident records.

7.12.2 Notice, Announcement and Emergency Communication Lifecycle

Announcements are governed content objects. They require authoring, audience targeting, priority, effective time, expiry, attachments, localization, scheduled publication, withdrawal, revision history and delivery/acknowledgement tracking where supported.

Real-world scenario — planned water shutdown: the society schedules a notice for Tuesday 9:00 AM explaining that Tower B water supply will stop from 11:00 AM to 2:00 PM. At 10:55 AM the notice becomes active, affected residents receive it, and after restoration a follow-up notice closes the incident. The original notice remains historically available.

Real-world scenario — emergency override: a critical fire-related notice must be delivered immediately even when a resident has enabled quiet hours. The emergency policy explicitly permits the override and the audit record captures why it occurred.

- Separate draft, scheduled, published, expired, withdrawn and superseded states.
- Target by configured audience criteria without leaking the audience definition to recipients who should not see it.
- Attachments inherit document security and retention rules.
- Localized templates must preserve meaning rather than simply translating labels independently.

7.12.1 Marketplace and Service Order Lifecycle

Marketplace functionality should separate discovery from contracting and fulfilment. A resident may request a service, receive a quote, place an order, track fulfilment, provide feedback and raise a dispute. Society-sponsored vendors may have different commercial rules from external providers.

Real-world scenario — electrician booking: a resident requests an electrician for a Saturday slot. The service provider confirms availability, the job is completed, the resident approves the outcome, payment is captured and the service history is retained without exposing unrelated resident data.

- Provider onboarding, availability, service area, pricing/quote, order state, cancellation and refund rules should be configurable.
- External service data must remain logically separated from society financial records unless a formal settlement relationship exists.
- Provider ratings should be traceable to actual completed orders.

8. Owner, Tenant and Occupancy History Management

Owner and tenant lifecycle history is a core differentiator. The system should preserve a reliable historical timeline for each unit rather than overwriting current resident information.

8.1 Business Purpose

For every unit, the platform should be able to establish the current ownership/occupancy state, previous ownership and tenancy periods, relevant documents, move-in and move-out events, NOC history, clearance status and access lifecycle.

8.2.1 Occupancy and Household Model

Occupancy is time-bound. The same unit may have an owner, co-owner, tenant, family members, authorized occupants and historical occupants at different times. The model must therefore use dated relationships rather than a single mutable “current resident” field.

Real-world scenario — owner becomes landlord: Owner A purchases Unit A-1203, later rents it to Tenant B, then terminates the tenancy and moves back in. Historical records must show all three periods. Visitor records, invoices, documents and NOC decisions must resolve to the correct period without exposing unrelated historical data.

Real-world scenario — family member exits: a child previously associated with the household moves out. The relationship ends on a date; historical facility or visitor events remain intact, while future household permissions no longer inherit from the previous association.

8.2.2 Resident Registration and Verification Lifecycle

Registration should support invitation, mobile/email verification, identity/profile completion, unit association, owner/tenant/family relationship, document submission, verification, approval where configured, activation, suspension, recovery and eventual exit. Duplicate-account detection must be based on configured identity signals and must never merge records blindly.

8.2 Functional Scope

Capability	Expected behaviour
Current ownership	Maintain active owner/co-owner relationship with effective dates.
Current tenancy	Maintain tenant, agreement metadata, move-in date, verification status and owner approval context.
Previous ownership	Preserve historical owners, transfer dates, transfer reason and linked documents.
Previous tenancy	Preserve previous tenants, occupancy period, exit status, NOC and clearance context.
Document archive	Retain historical agreements, KYC, verification and handover documents without exposing them by default to later occupants.
Unit timeline	Provide chronological lifecycle events including ownership, move-in, move-out, NOC and access activation/revocation.
Mandatory move-out clearance	Do not close the lifecycle until configured clearances are complete or a permitted override is recorded with reason.

8.3 Lifecycle Rules

9. Create or update the occupancy relationship with effective dates.
10. Collect required onboarding records and approvals.
11. Activate the occupancy only after required validation.
12. Record operational events against the current occupancy period.
13. Initiate move-out with a defined exit date and clearance checklist.
14. Resolve dues, access items, parking, damage, complaints and required documents.
15. Generate the applicable NOC/certificate.
16. Close the occupancy period and revoke applicable access.

8.4 Rental / Tenant Assessment

The platform may provide a configurable, explainable assessment of tenancy readiness for society operations. It should be based only on authorised, relevant operational criteria such as verification status, required document completeness, agreement validity and configured society requirements.

The assessment must not use protected or unrelated personal characteristics, must show the factors contributing to the result, and must not be presented as an infallible prediction of a person's character or future behaviour.

17. 8.5 Data Entities

8.4 Data Entities

Entity	Purpose	Important fields
UnitOccupancyHistory	Historical occupancy period.	unitId, ownerId, tenantId, startDate, endDate, status, moveInRequestId, moveOutRequestId
OwnerHistory	Ownership timeline.	unitId, ownerProfileId, ownershipStart, ownershipEnd, transferReason, documentIds
TenantHistory	Tenant occupancy and exit context.	unitId, tenantProfileId, agreementStart, agreementEnd, verificationStatus, nocStatus
MoveInRequest	Onboarding workflow.	unitId, requestedBy, approvals, liftSlot, truckPass, accessStatus
MoveOutRequest	Exit and clearance workflow.	unitId, requestedBy, checklist, approvalStatus, exitDate
NocRecord	Issued certificate metadata.	requestId, nocType, issuedBy, issuedAt, documentId, verificationCode
DocumentAccessLog	Sensitive document access trail.	documentId, actorUserId, action, timestamp, ipAddress, deviceId

Historical privacy rule A new occupant of a unit must not automatically inherit access to documents or sensitive records belonging to previous occupants.

Figure 6. Move-out and mandatory NOC lifecycle flow.

9. Biometric Attendance Integration

The platform should support biometric/fingerprint attendance devices primarily for operational attendance, workforce verification and vendor invoice reconciliation.

9.1 Privacy Boundary

Society OS should not store raw fingerprint images or biometric templates. Registration and biometric matching should remain inside the device/vendor ecosystem. Society OS should receive attendance events and associated operational metadata.

9.2 Functional Scope

Capability	Detailed behaviour
Device registration	Register device code, location, society, operational context and synchronisation mode.
Staff mapping	Map device employee code to a workforce profile, vendor and shift context.
Punch synchronisation	Import punch events through API, file import, sync agent or vendor connector.
Duplicate prevention	Use a deterministic duplicate key such as employeeCode + deviceId + punchTime.
Shift calculation	Compare attendance with roster to derive present, late, early checkout, half day or absent.
Manual correction	Allow controlled correction of missed or invalid punches with reason, approval where configured and audit.
Reporting	Generate daily, monthly, staff-wise, vendor-wise and location-wise attendance summaries.
Vendor billing support	Provide attendance summaries that can support outsourced workforce invoice verification.

9.3 Data Entities

Entity	Purpose	Important fields
BiometricDevice	Device metadata.	id, societyId, locationId, deviceCode, vendorName, syncType, status
BiometricStaffMapping	Employee-code mapping.	deviceId, employeeCode, staffProfileId, effectiveFrom, effectiveTo
AttendancePunch	Imported attendance event.	staffId, deviceId, punchTime, punchType, source, syncJobId, duplicateKey
AttendanceDailySummary	Computed attendance.	staffId, date, shiftId, status, firstIn, lastOut, lateMinutes, earlyMinutes
AttendanceCorrectionRequest	Correction workflow.	attendanceId, requestedBy, reason, oldValue, newValue, approvalStatus

9.4 Sync Flow

Figure 7. Biometric attendance synchronisation flow.

10. Document Vault, NOC and Secure Access

Documents are among the most sensitive records in Society OS. The document architecture must treat storage, metadata, access, versioning, expiry and audit as one controlled lifecycle.

10.1 Document Categories

Category	Examples	Control requirements
Society documents	Registration, bylaws, AGM minutes, fire NOC, lift certificate.	Controlled publication, versioning and expiry.
Owner documents	Sale deed, share certificate, identity/address proof.	Sensitive and explicitly authorised access.
Tenant documents	Rent agreement, KYC, police verification, owner approval.	Sensitive; isolated from unrelated residents and later occupants.
Move-in documents	Move-in form, vehicle details, rule acknowledgement, truck pass.	Linked to move-in lifecycle.
Move-out documents	No-dues, move-out NOC, handover, damage inspection.	Required for lifecycle closure.
Staff/vendor documents	Verification, contracts, licences, insurance, AMC.	Controlled operational access and expiry reminders.
Compliance documents	Fire, lift, insurance, legal notices and certificates.	Expiry tracking and compliance register.

10.2 NOC and Certificate Catalogue

NOC / certificate	Purpose	Typical clearance
No-dues certificate	Financial clearance.	Maintenance, penalties, facility charges and deposits.
Move-out NOC	Formal exit clearance.	Dues, access cards, parking, complaints, damage and documents.
Tenant NOC	Tenancy onboarding/exit validation.	Owner approval, KYC, agreement and verification.
Parking NOC	Parking/vehicle clearance.	Parking dues, sticker and violations.
Renovation NOC	Controlled renovation completion.	Damage inspection, debris clearance and contractor closure.
Residence certificate	Society address/occupancy proof.	Active occupancy and document verification.

10.3 Secure Document Access Flow

Figure 8. Secure document access flow.

10.4.1 Document Upload, Versioning and Retrieval

Document handling is a security boundary. Uploads should support configured file types and limits, checksum calculation, malware scanning, optional image compression/thumbnails, metadata minimization, encryption, access control, versioning, retention and download audit.

Real-world scenario — rental agreement: a tenant uploads a new agreement. The previous agreement remains immutable as version 1; the new document becomes version 2 with effective date and uploader recorded. A user without permission cannot retrieve either document merely by guessing its URL.

Failure scenario — interrupted upload: a mobile user loses connectivity at 80%. The client may resume or retry safely. The backend must not create a half-valid document record that appears complete.

10.4.2 NOC as a Business Workflow

NOC issuance must be treated as a controlled clearance process. A move-out request can require financial clearance, document/asset return, parking status, access revocation, meter readings, pending complaint/vendor obligations and configurable approvals before an NOC is issued.

Real-world scenario — move-out with outstanding dues: the resident requests NOC while ₹18,000 is outstanding. The system calculates the current ledger position, flags the clearance condition, allows settlement, posts the payment, verifies all required checks, and only then issues the NOC. The original debt and clearance evidence remain auditable.

10.4 Document Rules

- Never expose raw object-storage paths publicly.
- Use signed URLs or controlled download APIs with short-lived access.
- Maintain document type, owner entity, visibility, version and expiry metadata.
- Record sensitive view/download events.
- Retain historical versions where required by the document lifecycle.
- Support controlled verification of issued NOCs and certificates.
- Apply retention and deletion policies without breaking legal/audit requirements.

11. Workflows and User Journeys

Workflows should be implemented through configurable state machines or workflow services wherever practical. The objective is to support different society policies without duplicating business logic.

11.1.1 Reusable Workflow Engines

The platform should avoid implementing approval, SLA, notification and scheduling logic separately inside every module. Reusable engines reduce inconsistent behaviour and make society-specific policy configuration possible.

11.1.2 Reusable Approval Engine

Generic flow: Request → Rule Evaluation → Approver Resolution → Pending → Approve/Reject/Return → Multi-level Approval where required → Final State → Audit. Examples include tenant onboarding, move-out/NOC, procurement, vendor onboarding, facility exceptions, financial adjustments and governance decisions.

11.1.3 Reusable SLA and Escalation Engine

Generic flow: Domain Event → SLA Policy → Due Time → Reminder → Escalation 1 → Escalation 2 → Breach → Audit. The same engine can serve complaints, vendor AMC, preventive maintenance, compliance expiry, document expiry and operational exceptions.

11.1.4 Reusable Rules Engine

Generic flow: Input → Ruleset → Conditions → Action → Exception → Audit. Rules should drive late fees, booking constraints, tenant requirements, parking rules, move-out clearance, notification policies and approval routing without hard-coding society-specific logic into application services.

11.1.5 Calendar and Scheduling Engine

A shared scheduling capability should handle facility bookings, inspections, maintenance, vendor visits, meetings, events, move-in/out appointments, drills and shifts. Recurrence, blackout periods, buffers and conflict detection should be first-class concepts.

11.1 Workflow Principles

- Every workflow has a clear start state, valid transitions, terminal states and exception handling.
- Approval requirements are configuration-driven.
- Every state change is traceable.
- Notifications are derived from state changes rather than hard-coded UI actions.

- Long-running work is asynchronous and observable.
- Exceptions require an explicit reason and, where configured, approval.
- Closed financial and compliance records are corrected through controlled adjustments rather than deletion.

Workflow	Main steps	Important edge cases
Monthly maintenance billing	Prepare charge heads → generate draft → validate → approve → publish → collect → receipt → reconcile.	Duplicate payment, partial payment, late penalty, manual cheque/cash, accounting lock.
Complaint lifecycle	Create → classify → assign → SLA → work → resolve → confirm/reopen → close.	Repeated complaint, private complaint, escalation, vendor delay, wrong category.
Move-in	Submit → validate → upload documents → approvals → rule acknowledgement → activate occupancy/access.	Incomplete KYC, missing verification, slot conflict, missing document.
Move-out	Request → calculate clearances → resolve dues/assets/access → approvals → issue NOC → revoke access → close occupancy.	Pending dues, damage claim, lost access card, override, missing NOC.
Biometric attendance	Device punch → sync → validate → deduplicate → map → calculate shift → summarise.	Duplicate punch, missing checkout, offline device, wrong employee code, correction.
Wrong parking	Report → identify vehicle → notify → investigate → resolve → record violation.	Unregistered vehicle, emergency blocking, repeat violation, false report.
Visitor entry	Pre-approval or walk-in → validation → entry → optional exit → close record.	Network failure, expired pass, duplicate entry, invalid QR/OTP.
NOC issuance	Request → checklist → approvals → document generation → verification code → issue.	Incomplete clearance, approval rejection, document error.
Vendor AMC	Contract register → reminder → service → inspection → invoice → performance update → renewal/closure.	Expired contract, repeated failure, disputed invoice, emergency service.
Emergency incident	Trigger → acknowledge → classify → notify → respond → escalate if needed → close → review.	No acknowledgement, simultaneous incidents, communication failure.

11.1.7 Calendar, Recurrence and Conflict Management

The scheduling engine should represent events independently from calendars and views. A recurring maintenance schedule, facility booking, vendor visit and move-in appointment are all time-bound domain commitments with different conflict rules.

Real-world scenario — move-in versus maintenance: a move-in appointment requires loading access on Saturday, but a lift maintenance window is also scheduled. The scheduling engine identifies the conflict and allows policy-driven rescheduling rather than leaving operations to discover the collision at the gate.

- Use explicit timezone and local-date semantics.
- Recurrence rules must have end conditions and exception dates.
- Buffers and blackout windows participate in conflict evaluation.
- Changing an event must preserve its history and source of change.

11.1.6 Notification Engine

Notifications are a reusable event-driven capability. Domain event → notification rule → audience resolution → channel selection → delivery → retry/fallback → status/audit. The engine should support user preferences, quiet hours, critical overrides, templates and localization.

Real-world scenario — invoice reminder: an invoice becomes overdue. The rules engine sends a push notification first, then a configured fallback channel if delivery fails. The system records the attempt and final delivery state without creating a second invoice reminder event for the same rule occurrence.

- Notifications must be idempotent at the business-event/rule level.
- Critical notifications may override quiet hours according to explicit policy.
- Delivery failure must not be confused with business failure; the invoice remains overdue regardless of notification status.

- Templates must be versioned.

11.2 Visitor Pre-approval Flow

Figure 9. Visitor pre-approval workflow.

11.3 Resident Communication Approval Flow

Figure 10. Privacy-first first-contact communication flow.

12. Data Model and API Design

The data model must be multi-tenant from day one. Every business record must be directly scoped by `societyId` or strongly linked through a parent entity whose society boundary is validated server-side.

12.1.1 Master Data vs Transactional Data

Master/reference data changes relatively slowly and defines allowed vocabulary: complaint categories, visitor types, facility types, charge heads, payment methods, document categories, NOC types, vehicle types, vendor categories, asset categories, inventory categories, templates, SLA policies and escalation rules. Transactional data records what actually happened: visits, invoices, payments, complaints, bookings, work orders, messages and attendance events.

12.1.2 Entity Timeline and Historical Boundary

A unit timeline may include creation, ownership, parking, tenant move-in, visitors, complaints, payments, documents, move-out and NOC events. This timeline is an operational reconstruction aid, not unrestricted access to every historical record. Privacy and authorization boundaries remain applicable at every lookup.

12.1.3 Time, Timezone, Financial Year and Rounding

Store authoritative timestamps in UTC where appropriate and render them in the society/user context. Business rules must explicitly handle IST/local midnight, month-end, leap years, financial-year boundaries, billing cut-offs, INR precision and configurable rounding. Date-only business events must not be accidentally shifted by timezone conversion.

12.1 Core Data Entities

Entity	Purpose
Society	Tenant/account root containing hierarchy, configuration, policies and subscription-ready metadata.
Unit	Flat/villa/shop/office entity linked to residents, bills, visitors, documents, parking and complaints.
ResidentProfile	Owner, tenant or family profile linked to a user and unit relationship.
UnitOccupancyHistory	Historical ownership and tenancy periods.
Document	Secure file metadata, storage reference, visibility, verification and expiry.
NocRequest / NocRecord	Certificate request workflow and issued-document metadata.
StaffProfile	Guard, housekeeping, facility, domestic-help or vendor workforce record.
AttendancePunch / AttendanceDailySummary	Raw imported event and calculated attendance state.
Bill / Payment / LedgerEntry	Maintenance billing and financial transaction model.
Complaint	Structured issue with SLA, assignment, evidence and status.
Facility / Booking	Amenity definition, schedule and reservation lifecycle.
Vehicle / ParkingSlot	Vehicle and allocation model.
Visitor / GateEntry	Visitor identity, pass and physical entry record.

Vendor / Contract / AMC	Supplier and contractual service lifecycle.
Asset / MaintenanceRecord	Asset lifecycle, warranty, AMC and service history.
InventoryItem / StockTransaction	Consumable inventory and movement.
GovernanceMeeting / Resolution / Poll / Election	Formal governance records.
Incident / EmergencyEvent	Operational and emergency incident lifecycle.
AuditLog	Immutable sensitive-action trail.

12.1.4 Master and Reference Data Governance

Master data requires ownership, effective dates, validation and dependency awareness. Examples include complaint categories, visitor types, facility types, charge heads, payment methods, document categories, NOC types, vehicle types, staff types, vendor categories, asset categories, inventory categories, templates and escalation/SLA definitions.

Real-world scenario — complaint category rename: “Lift Problem” becomes “Vertical Transport”. Historical tickets retain their original classification semantics while new tickets use the new category. Reports must be able to group across versions when configured.

- Do not hard-delete referenced master values.
- Use inactive/effective-dated states for changed vocabulary.
- Configuration changes require audit and, where necessary, approval.
- Dependencies must be validated before a master record is retired.

12.2 API Design Principles

- Version APIs explicitly.
- Use consistent resource naming and HTTP semantics.
- Scope requests to the authenticated society context server-side.
- Use idempotency keys for retryable mutations and callbacks.
- Return stable identifiers for long-running operations.
- Keep payment, billing, NOC and workflow transitions behind domain services.
- Separate file upload initiation from secure file retrieval.
- Expose audit and operational status where appropriate without exposing sensitive internal metadata.

12.2.1 Error Contract and Validation Semantics

APIs must distinguish validation failure, authentication/authorization failure, business-rule rejection, concurrency conflict, idempotent replay, integration failure and unexpected server failure. Clients and testers need deterministic categories to handle these states correctly.

Real-world scenario — booking conflict: two users submit the same facility slot. One succeeds; the other receives a deterministic business conflict response with a reference to the failed reservation attempt. The second request must not partially create a booking.

- Use stable error codes in addition to human-readable messages.
- Validation errors identify the affected field or business rule where possible.
- Concurrency failures expose retry-safe semantics.
- Do not leak sensitive information through error messages.

12.3 Representative API Catalogue

Endpoint	Purpose
POST /api/v1/societies	Create society.
PATCH /api/v1/societies/{id}/configuration	Update society configuration.
POST /api/v1/units/import	Bulk import units/residents.
GET /api/v1/units/{id}/occupancy-history	Retrieve unit lifecycle history.
POST /api/v1/move-in-requests	Create move-in workflow.
POST /api/v1/move-out-requests	Create move-out workflow.
POST /api/v1/noc-requests	Request NOC/certificate.
POST /api/v1/documents	Upload document metadata/file reference.
GET /api/v1/documents/{id}/download	Controlled document retrieval.

POST /api/v1/visitors/preapprovals	Create visitor pass.
POST /api/v1/gate/entries	Record gate entry.
POST /api/v1/bills/generate	Generate maintenance bills.
POST /api/v1/payments/callback	Receive payment callback.
POST /api/v1/complaints	Create complaint.
POST /api/v1/bookings	Create facility booking.
POST /api/v1/integrations/biometric/devices	Register biometric device.
POST /api/v1/integrations/biometric/punches/sync	Import attendance logs.
GET /api/v1/attendance/monthly	Attendance reporting.
PATCH /api/v1/attendance/{id}/correction	Attendance correction workflow.

12.4.1 Domain-Specific Concurrency Strategy

Conflict resolution must be selected by business semantics rather than applying one universal CRDT or last-write-wins rule. Financial ledgers require transaction serialization; payment callbacks require idempotency and signature verification; occupancy requires validated state transitions; facility booking requires atomic reservation; attendance requires idempotent event ingestion; documents require version conflict detection; configuration uses optimistic concurrency; chat is append-oriented; audit is append-only.

12.4.2 Idempotency and Exactly-Once Business Effects

API retries are normal in mobile and distributed systems. Every externally retried command that can create a business effect should have an idempotency strategy. A repeated request may produce the same business result, but the system must not create duplicate invoices, payments, visitor admissions, stock issues or attendance events.

12.4 Data Integrity Rules

- Do not hard-delete financial transactions.
- Do not overwrite historical occupancy periods.
- Do not expose previous resident documents through current occupancy relationships.
- Do not permit duplicate booking or duplicate attendance events.
- Do not accept payment callbacks without signature/verification and idempotency handling.
- Do not allow feature-disabled APIs to execute business actions.

12.4.3 Financial, Reservation and Occupancy Invariants

Critical domains require explicit invariants. Financial journals must balance; active occupancy periods must not overlap when prohibited; confirmed reservations must not conflict; active parking allocations must respect slot constraints; visitor state transitions must be valid; attendance events must be deduplicated; audit events must not be overwritten.

These invariants should be enforced at the strongest practical layer—database constraints for structural rules, transactional services for cross-record rules, and workflow/rules engines for configurable policy.

- Every invariant has at least one automated regression test.
- A failed invariant leaves no partial business effect.
- Constraint violations are observable and diagnosable.

13. Feature Flags and Configuration

Feature flags allow one codebase to support societies with different operational complexity. Flags should control availability without creating inconsistent backend behaviour.

13.1 Core Feature Flags

Feature flag	Purpose	Initial direction
visitorManagement	Visitor passes, gate entries and gate logs.	true

maintenanceBilling	Maintenance billing and payment workflows.	true
complaints	Complaint/helpdesk workflows.	true
documentVault	Secure document management.	true
ownerTenantHistory	Historical occupancy and document archive.	true
moveOutNocMandatory	Require NOC/clearance before move-out closure.	true
residentMessenger	Privacy-first messaging.	true
publicResidentDirectory	Optional public directory.	false
biometricAttendanceIntegration	Biometric synchronisation.	false at launch
hardwareIntegration	Common integration framework.	false at launch
subscriptionBillingUI	Pricing/subscription UI.	false during free launch
smartAutomation	Smart automation layer.	false until maturity/readiness criteria are met
waterManagement	Water operational module.	configurable
energyManagement	Energy/solar/utility module.	configurable
evCharging	EV charging management.	false until enabled
governanceVoting	Configured voting workflows.	configurable
marketplace	Community services and commerce.	configurable
carpool	Community carpool.	configurable
lostAndFound	Community lost & found.	configurable
festivalAutomation	Event and festival workflows.	configurable

13.2 Configuration Principles

- Disabled modules must not appear as active capabilities in the product experience.
- Backend APIs must reject access to disabled capabilities.
- Configuration changes must be audited with old value, new value, actor and timestamp.
- Configuration must be tenant/society scoped.
- Dependencies must be validated before enabling a feature.
- Feature flags must not be used as a substitute for authorization.
- Commercial flags must remain hidden until monetisation is activated.

13.2.2 Configuration Safety and Dependency Checks

A feature flag or policy cannot be considered safe merely because it is a boolean. Dependencies must be validated before enabling a capability. For example, enabling automated payment reminders requires notification templates and channel configuration; enabling preventive maintenance requires asset and schedule structures.

Real-world scenario — partial rollout: a society enables biometric attendance but one device is not configured. The feature remains available for configured devices while the missing device is surfaced as an operational readiness exception.

- Feature enablement must validate prerequisites.
- Configuration changes must be reversible or have a documented migration path.
- Runtime evaluation is separate from authorization.
- Flag state changes are observable and auditable.

13.2.1 Configuration Governance and Effective Dating

Configuration is production data. Changes to charge heads, SLA values, booking windows, notification templates, approval rules, retention periods and feature flags should support versioning/effective dates where historical interpretation matters.

Real-world scenario — late-fee change: the society changes late fees from 1% to 1.5% starting 1 October. September invoices retain the old rule; October calculations use the new rule. Historical invoices must not be silently recalculated.

- Do not apply future configuration retroactively unless an explicit migration is executed.

- Configuration changes are audited.
- Dependent configurations are validated before activation.
- A configuration rollback must have defined semantics.

13.3 Example Configuration Structure

features: { visitorManagement: true, maintenanceBilling: true, complaints: true, documentVault: true, ownerTenantHistory: true, moveOutNocMandatory: true, residentMessenger: true, publicResidentDirectory: false, biometricAttendanceIntegration: false, hardwareIntegration: false, subscriptionBillingUI: false, smartAutomation: false, waterManagement: true, energyManagement: false, marketplace: false, carpool: false, lostAndFound: true, festivalAutomation: false }

14. Security, Privacy and Audit Logging

Society OS contains identity, financial, resident, tenant, staff, security and communication data. Security and privacy therefore form part of the product definition rather than an implementation afterthought.

14.1 Security Controls

Area	Required control
Tenant isolation	Every request is scoped to societyId and validated server-side.
RBAC	Role and permission checks apply to sensitive operations.
Document security	Use controlled storage and signed/authorised retrieval; never expose public document paths.
Audit logging	Record sensitive access, approvals, corrections, configuration changes and financial controls.
Privacy settings	Directory visibility, contact exposure and communication preferences are server-enforced.
Chat privacy	Private communication is not readable by administrators by default; moderation requires an explicit workflow.
Biometric privacy	Do not store raw fingerprint templates or images.
Data retention	Retention periods are configurable by data class and operational/legal requirement.
Session security	Secure tokens, expiry, device/session management and forced logout support.
Financial integrity	Use correction/reversal entries; do not delete posted financial history.
Secrets	Keep credentials and integration secrets outside source code and user-facing configuration.
Operational monitoring	Security events, failed authentication, unusual access and integration failures are observable.

14.2 Privacy Principles

- Collect only information necessary for a defined business purpose.
- Separate sensitive data from general profile information where practical.
- Make privacy defaults restrictive.
- Prevent historical data from leaking through shared unit relationships.
- Treat uploaded documents and user-generated content as untrusted input.
- Maintain access logs for sensitive documents and administrative operations.
- Provide configurable retention and controlled deletion where legally and operationally appropriate.

14.2.1 Privacy-by-Design Scenarios

Privacy controls should be tested against realistic misuse, not only documented permissions. Search, exports, notifications, communication, documents, historical timelines and offline caches can all become data-leak channels.

Real-world scenario — shared device: a guard logs out after using a shared tablet. The next guard must not see the previous operator's resident searches, visitor history or cached personal information.

Real-world scenario — notification preview: a lock-screen notification should not reveal sensitive document or financial details if the configured privacy level forbids it.

- Minimize sensitive data in push/SMS/email payloads.
- Encrypt sensitive local/offline data.
- Log access to high-sensitivity documents and records.
- Deletion/anonymization must respect legitimate retention requirements.

14.3.1 Operational Audit vs Business History

Audit answers “who did what, when, from where, through which channel, and with what result?”. Business history answers “what was the valid state over time?”. These should be correlated but not conflated. A correction to a financial record is a new compensating transaction, not an edit that erases the original event.

14.3.2 Data Lifecycle and Retention

A generic lifecycle is CREATED → ACTIVE → INACTIVE → ARCHIVED → RETENTION → ANONYMIZED/PURGED, subject to legal, contractual and operational requirements. The retention engine should be configurable by record category and purpose.

Real-world scenario — former tenant: a tenant leaves in June. Their operational association becomes inactive, but required financial records and legally necessary audit evidence may remain retained. Personal information that no longer has a valid purpose may be anonymized or purged according to policy; the platform must not treat all records as either “delete everything” or “keep forever”.

14.3 Audit Model

Event category	Examples
Identity	Login, logout, forced logout, password/credential changes.
Authorization	Permission changes, role changes, access denial.
Financial	Bill lock, correction, reversal, payment reconciliation, refund.
Document	View, download, upload, version, expiry, permission change.
Lifecycle	Move-in approval, move-out closure, NOC issue, access revocation.
Attendance	Correction, manual entry, mapping change.
Governance	Resolution approval, poll configuration, election result publication.
Security	Watchlist change, incident escalation, emergency action.
Configuration	Feature flag or society policy change.

15. Reports and Society Health Score

Reporting should support decisions, not merely display counts. The reporting layer should combine current status, trends, exceptions and historical comparisons.

15.1.1 Operational Dashboard Model

Dashboards should be derived from authoritative source records and should answer operational questions, not merely display attractive charts. Typical domains include dues and collections, outstanding ageing, complaints and SLA, current visitors, incidents, facility utilization, parking, attendance, assets, preventive maintenance, inventory, vendors, compliance and emergencies.

Real-world scenario — committee meeting: the committee wants to know why expenses increased. The dashboard should allow the aggregate number to be traced to invoices, vendor work orders, inventory consumption and ledger postings rather than producing an unexplained AI-generated narrative.

15.1.2 Search and Global Retrieval

Global search should cover units, residents, vehicles, visitors, complaints, invoices, payments, vendors, assets, documents, parking and bookings with filters, date/status/category/unit/reference, sorting, pagination and export. Search results must respect privacy and authorization boundaries; discoverability is not permission.

15.1 Report Catalogue

Category	Example reports
Financial	Collection, defaulters, expenses, budget vs actual, fund balances, reconciliation.
Operations	Complaint SLA, vendor performance, housekeeping quality, asset maintenance, recurring issues.
Security	Visitor trends, gate load, incident counts, watchlist events, workforce attendance.
Staff	Daily/monthly attendance, shift adherence, absence, vendor-wise summaries.
Owner/Tenant lifecycle	Move-in status, pending move-out clearance, agreement expiry, verification pending.
Compliance	Expired documents, fire/lift renewals, legal actions, governance records, rule acknowledgement.
Community	Notice acknowledgement, event participation, poll response, complaint feedback.
Utilities	Water consumption, tanker usage, electricity consumption, solar generation, EV usage where enabled.

15.2 Society Health Score

The Society Health Score is a composite operational indicator. It should be transparent, trend-based and explainable. It must not become an opaque personal score for residents.

Component	Example metrics
Collection efficiency	Percentage collected within due date, ageing and trend.
Complaint resolution	SLA compliance, average resolution time, reopen rate and feedback.
Security discipline	Gate process adherence, incidents, workforce attendance and unresolved security tasks.
Compliance health	Validity of mandatory documents and renewal timeliness.
Operational efficiency	Vendor performance, asset uptime, repeated issue reduction and checklist completion.
Resident satisfaction	Feedback, response time, participation and complaint sentiment where appropriate.
Workforce health	Attendance adherence, shift coverage and repeated absence patterns.
Utility health	Water/energy variance, leakage events, consumption trends and sustainability indicators where enabled.

15.3 Intelligence-Ready Reporting

- Explain material changes rather than only showing the new value.
- Highlight exceptions requiring attention.
- Allow historical comparison against the society's own baseline.
- Keep financial calculations deterministic.
- Show the source records behind derived indicators.
- Retain report generation metadata for audit and reproducibility.

15.3.1 KPI Definitions and Source-of-Truth Traceability

Every executive KPI should have a written definition, calculation period, source entities, exclusions, refresh behaviour and drill-down path. This is essential for trust in committee, management and investor-facing conversations.

Real-world scenario — collection efficiency: the KPI must distinguish billed amount, collectible amount, settled amount, reversed/refunded amount and outstanding amount. A dashboard showing one “collection %” without these definitions can create misleading conclusions.

- KPI definitions are versioned when business formulas change.
- Aggregations must reconcile to source records within documented tolerances.
- Exports and dashboards use the same authoritative metric definitions.

16. Smart Automation and Future Integrations

Smart automation is a controlled extension of the operational platform. It should be introduced after the core data model is clean, permissions are stable and historical records are reliable.

16.1 Smart Capabilities

Capability	Functional purpose	Dependency
Smart Complaint Routing	Suggest category, priority and operational routing using complaint context and rules.	Complaint history and category rules.
Automated Notice Drafting	Draft clear, multilingual society notices from structured context and approved intent.	Communication module.
Smart Document Search	Answer questions over permitted society documents and structured records.	Document vault and permission model.
Bill Explanation Assistant	Explain changes in a maintenance bill using authoritative charge data.	Structured billing model.
Meeting Summary	Summarise meeting notes/transcripts into actions and decisions.	Governance records.
Maintenance Risk Alerts	Identify assets with rising operational risk.	Asset and maintenance history.
Duplicate Incident Detection	Correlate similar complaints into a common operational incident.	Complaint history and location/time data.
Vendor Intelligence	Summarise SLA, cost, complaint and service performance.	Vendor and work-order history.
Collection Intelligence	Explain collection trends and highlight unusual variance.	Billing and payment history.
Community Operations Insights	Surface recurring issues, bottlenecks and recommended actions.	Cross-module operational data.

16.1.1 AI/Automation Decision Boundaries

AI may assist with classification, summarization, anomaly detection, forecasting and recommendations, but authoritative business state remains deterministic. AI output must never silently create a payment, approve a resident, issue an NOC, admit a visitor or alter an audit record.

Real-world scenario — complaint classification: AI predicts “water leakage” with 91% confidence. The prediction pre-fills classification, but the operational workflow still validates the category and permits correction. The final ticket state remains governed by deterministic rules.

- Store model/version and confidence metadata where AI materially affects a workflow.
- Provide human-review paths for consequential recommendations.
- Never fabricate missing source data in summaries.
- AI-generated explanations must link to authoritative records.

16.2 Intelligence Guardrails

- Authoritative values come from structured application data, not model invention.

- Permissions are applied before sensitive information is retrieved.
- Sensitive state changes require the existing deterministic workflow and appropriate approval.
- Uploaded documents and messages are treated as untrusted content.
- Every important recommendation includes evidence, confidence or source context where applicable.
- Low-confidence output should be presented as uncertain or routed for manual review.
- Core society operations must continue if an external intelligence provider is unavailable.

16.3.1 Integration Reliability Contract

Every external integration should define authentication, request/response contract, timeout, retry policy, idempotency, rate limits, failure states, reconciliation path, observability and versioning. Integrations must fail closed where security requires it and fail gracefully where availability is more important.

Real-world scenario — payment provider outage: residents can initiate payments but the provider is temporarily unavailable. Society OS must avoid marking invoices paid based on client-side success alone. Pending payment attempts remain pending until verified evidence arrives, while reconciliation exposes exceptions for operations/finance.

Real-world scenario — biometric device offline: a biometric terminal stores punches locally while connectivity is unavailable. When the connection returns, events are uploaded with device identifiers and event timestamps. Duplicate sync attempts do not create duplicate attendance records.

16.3.2 Localization and Accessibility as Platform Capabilities

Localization should cover English, Hindi, Marathi, Kannada, Tamil, Telugu, Bengali, Gujarati, Malayalam, Punjabi and future languages through externalized labels/templates rather than duplicated business logic. Accessibility should include scalable text, high contrast, screen-reader semantics, keyboard navigation on web, adequate touch targets, voice/audio assistance where appropriate, reduced motion and non-color-only status communication.

16.3 Integration Catalogue

Integration	Purpose	Design direction
Biometric devices	Attendance synchronisation.	Attendance logs only; no raw biometric templates.
RFID	Vehicle/staff access automation.	Map tags to authorised records.
ANPR	Automatic vehicle recognition.	Optional integration with explicit privacy controls.
CCTV	Restricted incident evidence.	Highly controlled and audited access.
Smart meters	Water/electricity/gas data.	Sensor-ready schema with manual fallback.
EV charging	Slot, usage and billing.	Integrate with facility and financial workflows.
Payment gateways	Digital collections.	Verified callbacks and idempotency.
Messaging providers	SMS/WhatsApp-style fallback and notifications.	Provider abstraction and delivery tracking.
Utility providers	Electricity/water/energy data where APIs are available.	Connector-based integration with graceful fallback.
Property-tax services	Property-related payment/status integration where available.	External-service adapter; do not make core operations dependent on it.

16.4 Community Intelligence Extensions

- Smart Lost & Found: structured item registration, matching, verification and closure.
- Community Carpool: ride requests, recurring routes, matching and controlled contact exchange.
- Festival & Event Automation: planning templates, contributions, volunteers, reminders and closure.
- Society Micro-Commerce: controlled local commerce with order/payment readiness.
- Domestic Worker Replacement Network: availability and verified-service discovery without exposing unnecessary personal information.
- Pet Management: pet registry, service discovery and incident workflows.
- Classifieds: controlled community listings with reporting and moderation.

- Guest Stay Management: stay records, approvals, deposits and checkout.
- Housekeeping Quality: recurring checklists, inspection scores and corrective actions.
- Emergency Local Network: configurable local contacts, volunteers and response coordination.

16.5 Water, Energy and Sustainability

Area	Scope
Water	Meter/readings, tanker logs, consumption, leakage, maintenance, shortage alerts and conservation reporting.
Electricity	Common-area consumption, meter data readiness, billing inputs and anomaly reporting.
Solar	Generation, usage, savings and maintenance records.
EV	Charging points, slot booking, usage, billing and load management readiness.
Energy analytics	Trend, variance, efficiency and sustainability reporting.

16.5.1 Utility and Meter Operations

Utility management should support meter masters, readings, reading source, validation, anomalies, billing/recovery and historical trends. Water, electricity, solar generation, diesel and common-area consumption can be modelled as related but distinct utility domains.

Real-world scenario — abnormal water usage: Tower A consumption doubles compared with its baseline. The anomaly engine flags the change, maintenance receives an investigation task, and the final outcome records whether it was a leak, meter fault, genuine usage spike or data error.

- Meter readings require timestamp/source semantics.
- Outliers should be reviewable before becoming financial charges where policy requires.
- Utility corrections should be versioned and auditable.
- Energy data can feed sustainability and cost reporting.

17. Roadmap and Delivery Priorities

The roadmap is designed to establish a dependable operational core before introducing advanced intelligence and hardware dependencies.

Phase	Feature set	Outcome
MVP	Society setup, unit master, resident onboarding, visitor/gate, billing, complaints, notices, staff management, document vault, feature flags, core notifications and low-connectivity foundations.	Launch a dependable society platform with the essential daily workflows.
Phase 1A	Resident Connect, privacy directory, owner/tenant history, smart move-in/move-out, document archive, parcel handover, inter-flat issues, lost & found and basic community services.	Differentiate the product through privacy, lifecycle history and daily resident utility.
Phase 2	Digital NOC, parking, facility booking, vendor/AMC, assets, inventory, renovation, governance, compliance, housekeeping quality and advanced reporting.	Strengthen operational maturity and administrative control.
Phase 3	Biometric attendance, workforce automation, senior care, emergency volunteers, marketplace, community commerce, carpool, events, staff/payroll readiness and Society Health Score.	Add measurable operational intelligence and broader community utility.
Phase 4	RFID, ANPR, boom barrier, CCTV integrations, smart meters, solar/energy, EV charging, smart automation, e-voting and advanced connected-device integrations.	Make the platform enterprise- and premium-community ready.

17.1 Delivery Priorities

18. Establish clean society, unit, resident and financial master data.
19. Make security, tenant isolation and auditability production-grade before expanding integrations.
20. Complete core visitor, billing, complaint, document and staff workflows.
21. Introduce lifecycle history and NOC capabilities without compromising privacy.
22. Expand operational modules such as parking, facilities, vendors, assets and governance.
23. Add workforce, attendance, compliance and emergency capabilities.
24. Introduce community services and commerce after operational foundations are stable.
25. Introduce smart automation only after sufficient structured data and evaluation criteria exist.
26. Add hardware and utility integrations through stable connector interfaces.
27. Continuously measure adoption, reliability, operational savings and data quality.

17.2.1 Delivery Slice Definition

A delivery slice is complete only when the normal scenario, important exceptions, persistence rules, audit behaviour, notifications, failure handling, observability and acceptance tests are covered. “API implemented” or “screen completed” is not a production-complete slice.

Example slice — visitor admission: the slice includes pre-approval, gate validation, resident action, offline capture, duplicate prevention, exit recording, notification, audit, emergency exception and reconciliation—not just a visitor create endpoint.

17.2.2 Versioning and Migration Discipline

Database migrations, API versions, backward compatibility, feature-flag migrations, mobile/server compatibility and offline schema compatibility must be planned before release. A mobile client may remain offline across a deployment boundary, so sync payloads require compatibility and migration strategy.

17.2 MVP Discipline

Build principle Do not overload the first release with every advanced capability. Build the operational backbone well, keep advanced modules configurable, and add complexity only when the data, workflow and support maturity justify it.

18. Acceptance Criteria and Implementation Backlog

Acceptance criteria must cover positive cases, negative cases, permissions, audit behaviour, edge cases, data integrity and recovery from failure.

Epic	User story / requirement	Acceptance criteria
Resident onboarding	Import residents and units in bulk.	Duplicates detected; validation preview shown; valid records imported; errors reported; import job logged; rollback available.
Visitor management	Create and validate a visitor pass.	QR/OTP or walk-in validation works; entry is logged; invalid/expired passes are rejected; duplicate entry is handled.
Billing	Generate monthly maintenance bills.	Correct charge heads applied; draft validation available; publication is controlled; payments update ledger; receipts are generated.
Complaints	Create and resolve a complaint.	Ticket identifier created; SLA starts; assignment is traceable; evidence retained; resolution and reopen are auditable.
Owner/tenant history	Preserve historical occupancy.	Occupancy periods do not overlap incorrectly; previous records remain available under controlled access; current occupant cannot inherit sensitive historical documents.

Move-out NOC	Prevent premature lifecycle closure.	Clearance checklist evaluated; NOC not issued when mandatory conditions fail; authorised override requires reason and audit.
Document vault	Secure a sensitive document.	Metadata required; storage is controlled; access is authorised; download is logged; expiry/version are retained.
Biometric sync	Import attendance punches.	Duplicate events rejected; employee mapping validated; daily summary calculated; sync failures visible; job is traceable.
Attendance correction	Correct a missed punch.	Reason required; approval rules applied; old/new values retained; audit event created.
Resident communication	Initiate controlled contact.	First-contact request captures purpose; acceptance/rejection/block/report works; phone exposure is not required.
Private communication	Protect private content.	Unreported private conversations are not exposed through ordinary administrative access; moderation follows an audited workflow.
Parking	Record a violation.	Vehicle/property association validated; evidence can be attached; status and history are retained; repeat behaviour can be identified.
Facility booking	Prevent double booking.	Database constraint prevents overlap; payment/deposit behaviour follows booking state; cancellation/refund rules are deterministic.
Emergency	Trigger an urgent incident.	Alert is generated quickly; acknowledgement is tracked; escalation works when unacknowledged; incident history is retained.
Feature flags	Disable a module.	Capability is unavailable in the product experience; backend rejects calls; configuration change is audited.
Financial integrity	Correct an existing transaction.	Original record remains; correction/reversal is linked; audit trail identifies the reason and actor.
Security	Prevent cross-society access.	Requests using another society's identifiers fail even if the resource ID is known.
Offline operation	Continue critical field workflow without network.	Event is queued safely; user sees sync state; reconciliation prevents duplicates after reconnect.
Smart automation	Provide a recommendation or draft.	Source context is valid; permissions are enforced before retrieval; low-confidence cases are handled safely; sensitive state changes still require normal authorization.
Reporting	Produce repeatable operational reports.	Filters are deterministic; totals reconcile with source records; report generation metadata is retained.

18.1.1 Test Scenario Taxonomy

QA should test at five levels: happy path, validation/negative path, concurrency/idempotency, failure/recovery, and audit/privacy. For major workflows, tests should also cover offline behaviour, localization, boundary dates and configuration variants.

Test dimension	Example	Expected result	Evidence
Happy path	Resident pays full invoice	Invoice settled and receipt issued	Ledger + receipt + audit
Negative	Visitor arrives with invalid/expired pass	Admission blocked with reason	Gate event + audit

Concurrency	Two users reserve same slot simultaneously	At most one confirmed reservation	Reservation constraint/log
Idempotency	Same payment callback sent twice	One financial effect	Idempotency record
Failure recovery	Network fails during gate sync	Event retained and reconciled later	Sync log
Privacy	Unauthorized user searches resident	Sensitive data not exposed	Authorization/audit evidence

18.1 Definition of Done

- Business rules documented and implemented.
- Positive and negative cases tested.
- Permission and tenant-isolation tests completed.
- Audit events implemented for sensitive mutations.
- Feature-flag behaviour tested in both enabled and disabled states.
- Failure/retry behaviour documented for integrations.
- Data migration and rollback strategy documented where applicable.
- Operational metrics and logging available.
- User-facing terminology consistent with the product glossary.
- Security review completed for sensitive modules.

19. Appendix

19.1 Example Feature Flag Structure

features: {

```

    visitorManagement: true,
    maintenanceBilling: true,
    complaints: true,
    documentVault: true,
    ownerTenantHistory: true,
    occupancyHistory: true,
    moveOutNocMandatory: true,
    residentDocumentArchive: true,
    residentMessenger: true,
    residentDirectory: true,
    publicResidentDirectory: false,
    controlledGroupChat: true,
    digitalNocRequests: true,
    societyRulesAcknowledgement: true,
    biometricAttendanceIntegration: false,
    attendanceDeviceSync: false,
    staffShiftRoster: true,
    attendanceCorrectionWorkflow: true,
    cctvAccess: false,
    anprRfidIntegration: false,
    smartMeters: false,
    waterManagement: true,
    energyManagement: false,
    evCharging: false,
    marketplace: false,
    carpool: false,
    lostAndFound: true,
    festivalAutomation: false,
    subscriptionBillingUI: false,
    pricingPage: false,
    upgradePrompts: false
  
```

}

19.2 Developer Rules

- Never hardcode visible pricing in the user-facing product during the free-launch period.
- All modules must enforce feature flags on both frontend and backend.
- Every sensitive administrative action must create an audit record.
- Do not delete posted financial records; use reversal or correction entries.
- Documents must have access control, expiry metadata and access logs.
- Critical gate workflows must support low connectivity and synchronisation.
- Keep SaaS subscription billing separate from resident maintenance payments.
- Do not store raw fingerprint data in Society OS; synchronise attendance events only.
- Previous owner/tenant documents must never be visible to later occupants by default.
- Move-out closure requires the configured NOC/clearance unless an authorised override with reason is permitted.
- All external integrations must have retry, failure and observability handling.
- AI/smart capabilities must not bypass deterministic permissions, workflow rules or financial controls.

19.3 Product Glossary

Term	Definition
Society	The top-level tenant/account representing a residential community.
Unit	A flat, villa, shop, office or other occupiable property within a society.
Occupancy	A time-bound relationship between a unit and its resident/tenant context.
NOC	No-objection/no-dues or other configured certificate generated through an approval workflow.
AMC	Annual maintenance contract or similar recurring service agreement.
Feature flag	Configuration switch controlling whether a capability is active.
Audit log	Immutable record of a sensitive action, decision or configuration change.
Smart automation	Rule- or intelligence-assisted functionality that reduces repetitive work or provides recommendations.
Society Health Score	Explainable composite indicator of operational health.
Integration layer	Connector architecture used to communicate with external systems and devices.

19.5 Real-World Scenario Library

The following scenarios should remain part of product discovery, demos, QA regression and implementation workshops because they demonstrate the difference between a feature list and a real operating platform.

Scenario A — New Resident Move-In

28. Society creates or imports the unit and parking master.
29. Owner/tenant receives an invitation and completes verification.
30. Occupancy record becomes active for a defined period.
31. Required documents are uploaded and versioned.
32. Move-in appointment is scheduled and gate instructions are generated.
33. Vehicle and parking are linked.
34. Notifications and audit events are recorded.
35. Any configured deposit/charge is posted to the financial workflow.

Scenario B — Tenant Move-Out with NOC

36. Tenant submits move-out request with target date.
37. Rules engine evaluates notice period and required documents.
38. Finance calculates outstanding balance and pending adjustments.
39. Parking, assets, access and document clearances are checked.

40. Required approvals complete.
41. Final dues/refund are posted through immutable financial transactions.
42. NOC is generated as a controlled document version.
43. Access is revoked at the effective time and occupancy is closed.

Scenario C — Lift Breakdown During Peak Hours

44. Resident reports failure with evidence.
45. Ticket is classified and SLA starts.
46. Correlation engine identifies whether related complaints exist.
47. AMC vendor/work order is assigned.
48. Residents receive an incident update.
49. Vendor diagnoses, requests spare parts and records service evidence.
50. SLA/escalation engine tracks delay.
51. Resolution is verified and cost/inventory records are linked.
52. Ticket closes while the asset maintenance history is updated.

Scenario D — Monthly Billing and Payment Failure

53. Billing period closes and charge calculation runs.
54. Validation detects missing configuration before invoices are published.
55. Invoices are generated and notified.
56. Resident initiates UPI payment.
57. Gateway callback is verified and posted idempotently.
58. A second callback is ignored as a duplicate business effect.
59. A failed/partial payment remains accurately represented.
60. Reconciliation and ageing dashboards update from source records.

19.6 Product Pitch Message

Society OS brings the society's fragmented operational world into one configurable system of record. It connects the property hierarchy with people, access, money, work, assets, documents, communication, governance and intelligence—while preserving auditability, privacy, offline resilience and India-first operational realities. The platform is designed so a small standalone building and a multi-tower township can use the same core model with different configuration, rules and scale.

19.9 How This Blueprint Is Used by Each Discipline

Engineering uses the specification as the domain and implementation contract: states, entities, invariants, API/event semantics, idempotency, concurrency, failure handling and migration rules. Product owners use it to validate business scope, policy options, user outcomes and future roadmap decisions. QA uses it to derive happy-path, negative, concurrency, recovery, privacy, offline, boundary-date and configuration-variant tests. Managers use it to understand delivery scope, operational dependencies, risk, readiness and measurable outcomes. Pitch/demo teams use the real-world scenarios to explain value through complete stories rather than isolated feature names.

A capability should not be considered “specified” until the relevant scenario, exception path, data behaviour, security/privacy boundary and acceptance criteria can be traced back to this blueprint or to an explicitly referenced detailed specification.

19.8 Foundational Capability Coverage Index

This index is a completeness control for future product reviews. It does not assign features to roles or applications; it identifies where a foundational capability is specified and what lifecycle it must cover.

Capability	Primary specification area	Minimum lifecycle / scenario
Society onboarding	1, 7.1	Create → configure → validate → import → activate
Property/unit master	7.1, 8, 12	Create → change → historical

		state → archive
Resident onboarding	7.1, 8	Invite → verify → associate → approve → active → exit
Household/family	7.1, 8	Add → effective period → update → end
Visitor/gate	7.2, 11	Pre-approve → admit → exit → exception → audit
Billing/payments	7.3, 12	Calculate → invoice → pay → reconcile → adjust/refund
Complaints	7.4	Create → assign → work → resolve → reopen/close
Facility booking	7.5	Request → conflict check → confirm → use → cancel/refund
Parking/vehicles	7.6	Register → allocate → operate → violate → reassign/history
Communication	7.7, 11	Compose → target → deliver → acknowledge/moderate
Disputes	7.8	Open → investigate → mediate → resolve → close
Governance	7.9	Agenda → meeting → vote → resolution → minutes/action
Emergency	7.10	Detect → declare → respond → communicate → recover/review
Vendor/AMC/assets	7.11	Onboard → contract → work → evidence → invoice → performance
Inventory/procurement	7.11	Request → approve → PO → receive → issue → reconcile
Documents/NOC	10	Upload → scan → version → access → retain → NOC
Attendance	9	Ingest → validate → dedupe → correct → report
Imports/exports	11, 12	Upload → validate → preview → process → report
Notifications	11, 16	Event → rule → channel → retry/fallback → audit
Search	15	Query → filter → authorize → retrieve → export
Backup/DR	14	Backup → verify → restore → validate → resume
Sessions/devices	11, 14	Authenticate → trust → expire/revoke → recover
SLA/escalation	11, 16	Event → due → remind → escalate → breach
Rules/approval	11, 13	Request → evaluate →

		approve/reject → audit
Calendar/scheduling	11	Schedule → detect conflict → execute → reschedule
Localization/accessibility	2, 16	Configure language/accessibility → render → validate
Data lifecycle	14	Active → inactive → archive → retain → anonymize/purge
Observability	16	Emit → collect → alert → diagnose → recover
Versioning/migration	16, 17	Change → migrate → compatibility → deprecate

19.7 Master Documentation Rule

Whenever a new capability is added to Society OS, its specification should include: business problem, real-world scenario, normal workflow, alternate workflow, exception cases, state model, business rules, data entities, API/event contract, permissions boundary, notifications, audit requirements, offline/failure behaviour where relevant, reporting impact, acceptance criteria, security/privacy impact and operational observability. This rule prevents future versions from becoming feature-name catalogues without implementation depth.

19.4 Final Product Recommendation

Society OS should launch as a clean, free and configurable society management platform while keeping its architecture enterprise-ready. The product should solve standard society needs exceptionally well—visitors, billing, complaints, documents and staff operations—while progressively adding deeper capabilities such as owner/tenant lifecycle history, mandatory move-out NOC, privacy-first communication, workforce attendance, governance, safety, compliance, assets, utilities, community services and smart automation.

The strongest product strategy is not to compete by accumulating isolated features. It is to create one trusted operational system in which people, units, money, documents, security, services, assets, governance and history are connected. Once that foundation is reliable, automation and intelligence can deliver meaningful value because they operate on structured, permission-safe society data.

Final principle Society OS should make the society easier to operate, easier to govern and easier to live in—without sacrificing privacy, financial integrity, security or transparency.

19.10 Scenario-First Functional & Complete Diagram Compendium

Purpose: This section is the authoritative scenario and diagram companion for Product, Engineering, QA, Management and pitch/demo teams. Every scenario contains both a complete use-case diagram and a complete data flow diagram. Diagrams explicitly include authenticated entry points for protected human actions, approval/signature steps where required, external participants, processing services, authoritative storage, notifications and exception paths.

Diagram completeness standard: A diagram is considered complete only when every participant that initiates, authorizes, approves, signs, supplies external evidence, or receives an outcome is represented; every protected administrative action has an explicit sign-in/authentication path; every authoritative state change has a processing component and destination store; every external callback/retry has a verification/idempotency boundary; and exception/retry/conflict paths are shown where the written scenario requires them.

Source-of-truth rule: The written scenario, state progression, use-case diagram and DFD must describe the same business outcome. If a diagram omits a participant or business step, it is considered incomplete even if the narrative mentions it.

Human / External Actor	Application / Process	Authoritative Data Store	Exception / Risk Path
------------------------	-----------------------	--------------------------	-----------------------

F01 - Monthly Maintenance Billing

Domain: Finance / Billing

Primary participants: Billing Administrator, Resident, Notification Service

Business objective: Generate a reproducible billing cycle from effective-dated rules and post receivables once.

Real-world scenario: A 180-unit society closes September billing. Maintenance, parking, water and sinking-fund charges are calculated; anomalous bills are held for review before invoices are published.

Preconditions: Unit/property master is active; billing rules and effective dates are configured; the financial period is open.

Normal flow: Admin signs in → selects billing period → engine snapshots applicable rules → calculates charges → validates exceptions → admin reviews exceptions → invoices are generated → receivables are posted → residents are notified → run is closed.

Exception / alternate flow: Missing rate, duplicate charge, negative/zero result, mid-cycle configuration change or partial run failure must produce a recoverable exception without duplicate posting.

State progression: DRAFT → CALCULATING → VALIDATING → REVIEW → PUBLISHED → POSTED → CLOSED

Key invariant: A billing run is idempotent for the same society/period/rule-version; posted invoices are immutable except through controlled adjustments.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F01-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

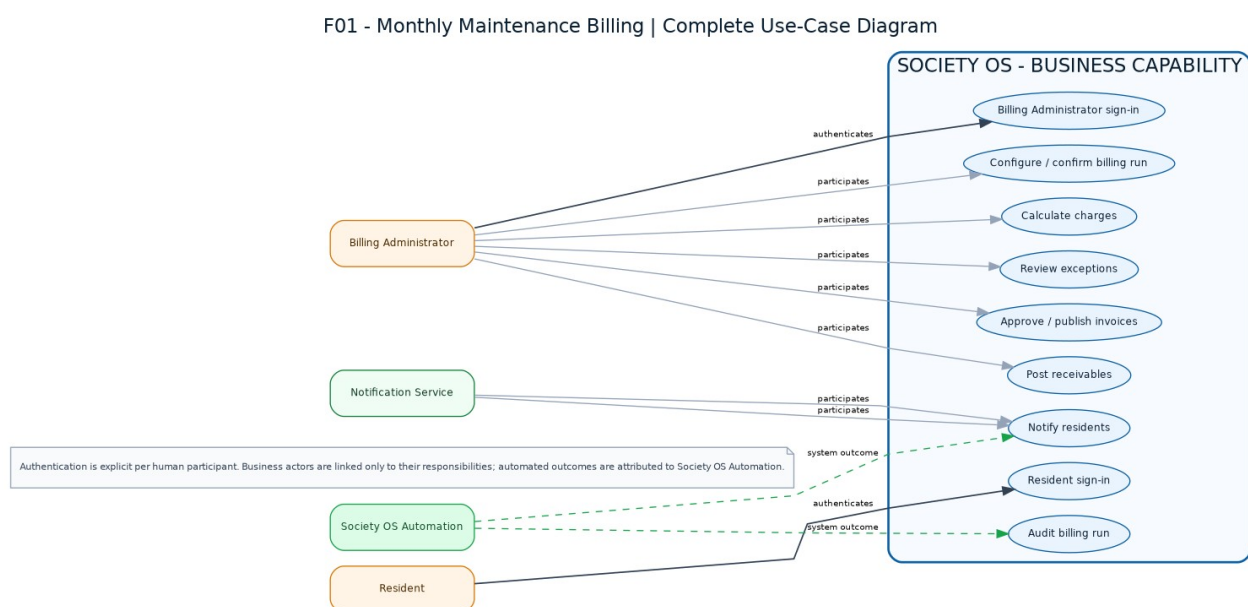
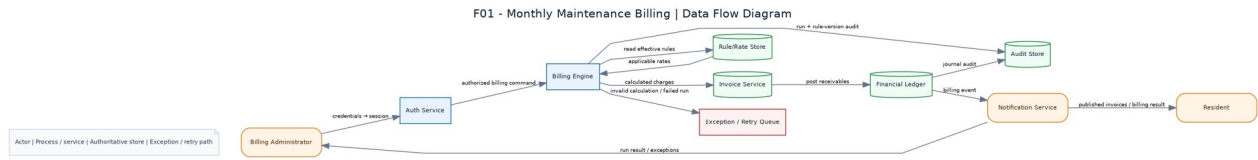


Figure F01-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F02 - Normal UPI Payment

Domain: Finance / Payments

Primary participants: Resident, Payment Gateway, Finance Administrator

Business objective: Turn a verified UPI payment into exactly one allocation, ledger posting and receipt.

Real-world scenario: A resident pays a ₹12,500 maintenance invoice using UPI. The gateway callback is signature-checked and the payment is matched before the ledger is updated.

Preconditions: An invoice is payable; payment gateway and callback credentials are configured; resident session is valid.

Normal flow: Resident signs in → views invoice → initiates UPI → gateway processes payment → callback reaches API → signature and amount/reference are validated → payment is allocated → ledger posts → receipt is generated → resident is notified.

Exception / alternate flow: Gateway timeout, unmatched reference, amount mismatch or provider outage creates a pending/exception record; no payment is posted on client-side success alone. If an exception remains unresolved, the Finance Administrator signs in to review the exception and retry/reconcile it.

State progression: REQUESTED → GATEWAY_PENDING → VERIFIED → ALLOCATED → POSTED → RECEIPTED / EXCEPTION

Key invariant: Callback verification and idempotency are required before financial posting.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F02-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

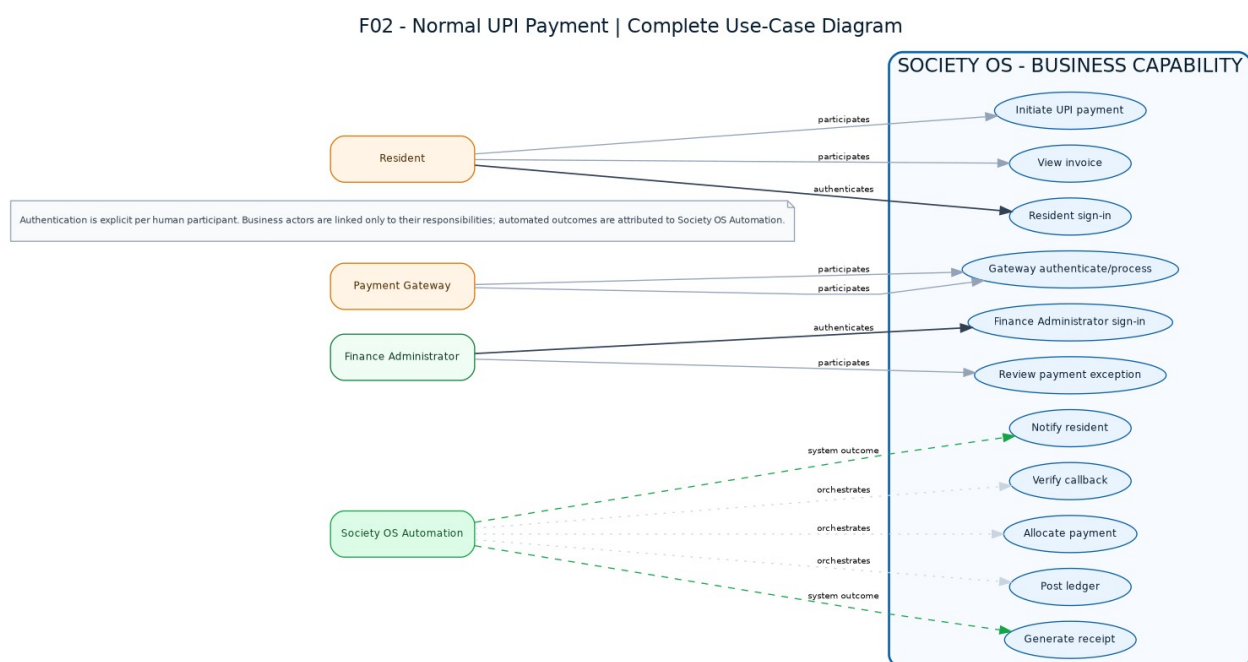
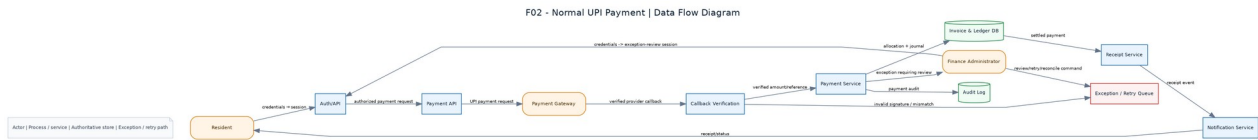


Figure F02-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F03 - Partial Payment

Domain: Finance / Payments

Primary participants: Resident, Finance Administrator

Business objective: Accept a payment lower than the outstanding amount while preserving the remaining balance.

Real-world scenario: A unit owes ₹12,500 and pays ₹7,500. The system allocates the payment using configured priority rules and leaves ₹5,000 outstanding.

Preconditions: Open invoice and allocation policy exist.

Normal flow: Resident signs in → selects amount → payment succeeds → system allocates ₹7,500 → ledger posts settled portion → balance remains ₹5,000 → receipt reflects partial settlement → ageing continues on remaining amount.

Exception / alternate flow: Amount exceeds balance, allocation rule unavailable, duplicate callback or payment reversal must not distort outstanding.

State progression: OPEN → PARTIALLY_PAID → PARTIALLY_RECONCILED → FULLY_PAID or OVERDUE

Key invariant: Allocated amount + remaining outstanding must equal the authoritative invoice balance after all approved adjustments.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F03-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

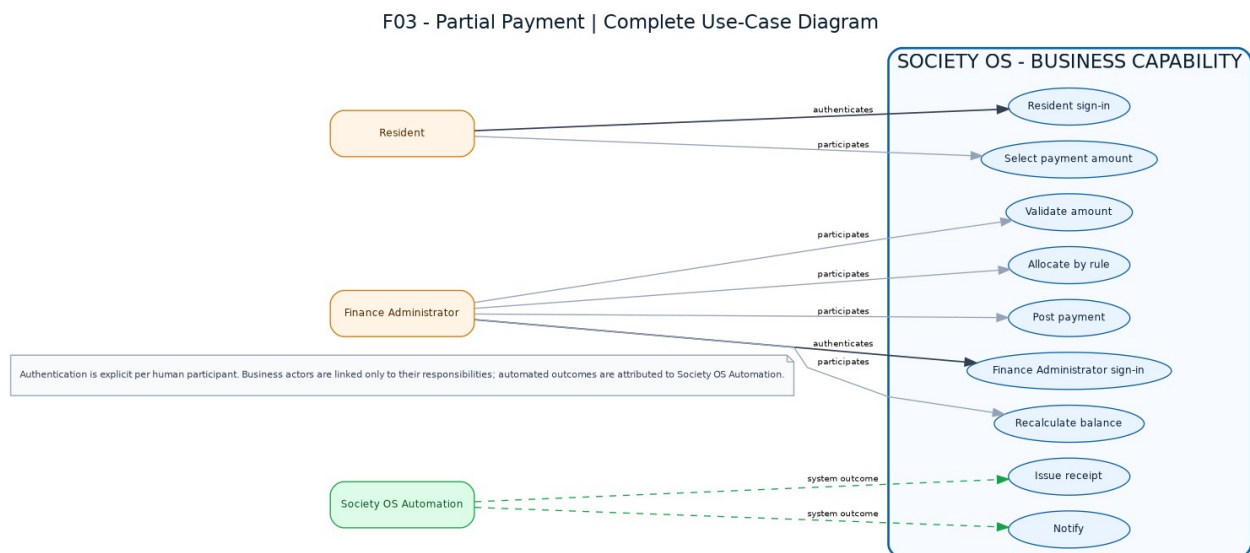
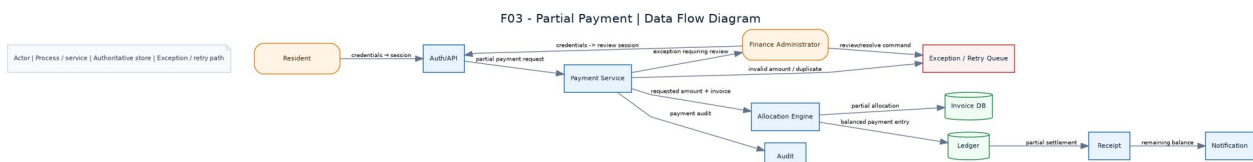


Figure F03-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F04 - Advance / Credit Payment

Domain: Finance / Payments

Primary participants: Resident, Finance Administrator

Business objective: Store excess payment as controlled resident credit rather than unclassified cash.

Real-world scenario: A resident pays ₹20,000 against ₹15,000 due. ₹15,000 settles current dues and ₹5,000 becomes a traceable credit balance for future allocation or refund.

Preconditions: Payment exceeds current collectible balance; credit policy is configured.

Normal flow: Resident signs in → payment verified → current dues allocated → excess identified → credit account created/updated → ledger records source transaction → resident receives receipt showing settlement and credit.

Exception / alternate flow: Credit expiry, restricted refund, duplicate credit or invalid source must be handled by explicit policy.

State progression: REQUESTED → VERIFIED → DUES_ALLOCATED → CREDIT_CREATED → RECONCILED

Key invariant: Every credit must point to the source payment and may not be created by overwriting the ledger.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F04-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

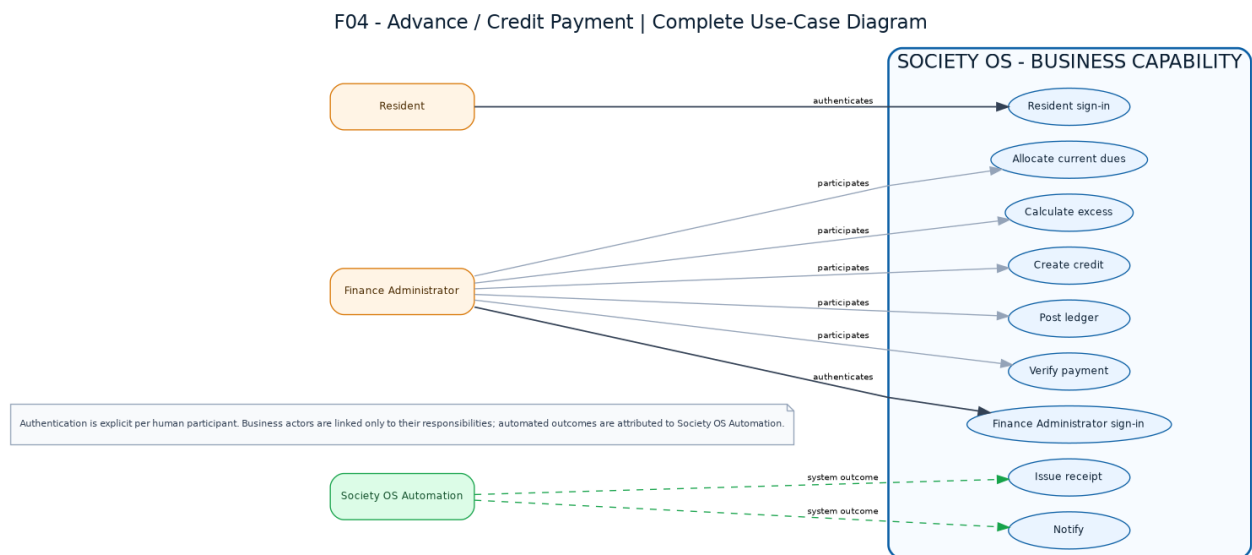
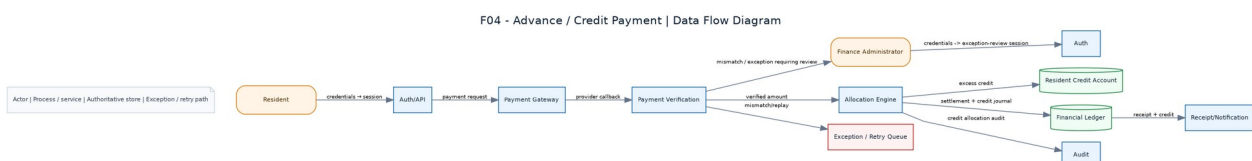


Figure F04-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F05 - Failed Payment and Retry

Domain: Finance / Payments

Primary participants: Resident, Payment Gateway, Finance Administrator

Business objective: Make payment failure recoverable without creating a false paid status.

Real-world scenario: A UPI attempt fails because the bank declines it. The invoice remains outstanding and the resident can retry or use another method.

Preconditions: Invoice is open; provider is reachable enough to accept the request.

Normal flow: Resident signs in → attempts payment → provider returns failure/timeout → payment attempt is marked failed/pending as appropriate → no ledger posting occurs → retry remains available → successful retry creates a new verified transaction.

Exception / alternate flow: Provider callback arrives after client timeout; an earlier attempt later succeeds; repeated retries must not double-settle.

State progression: INITIATED → PROCESSING → FAILED/PENDING → RETRY or CANCELLED → SUCCESS

Key invariant: Failed client attempts never become ledger postings unless an authoritative verified success is received.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F05-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

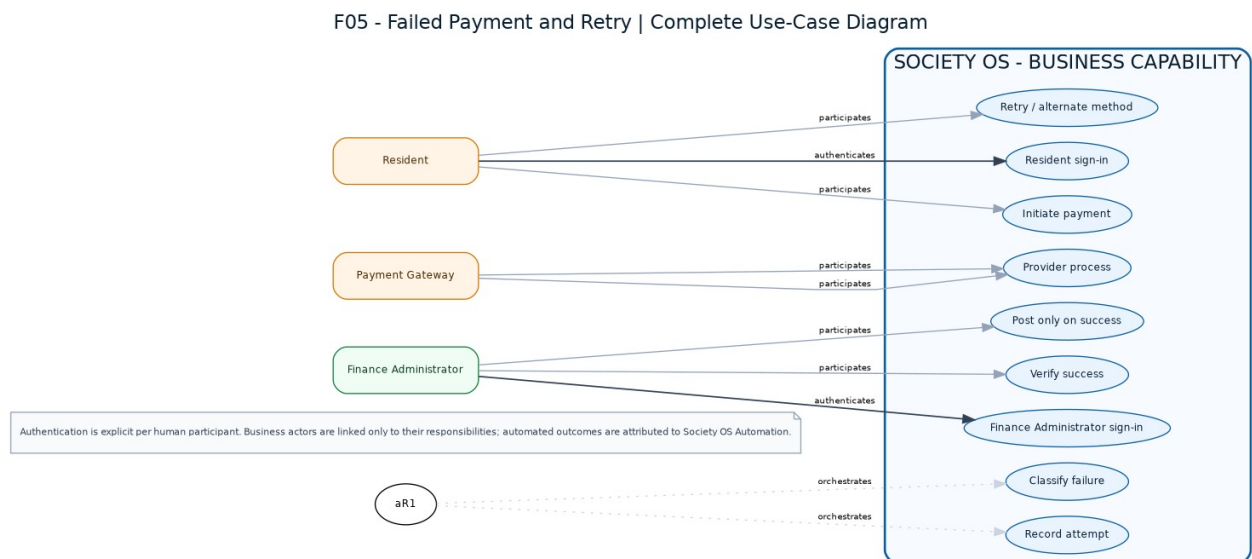
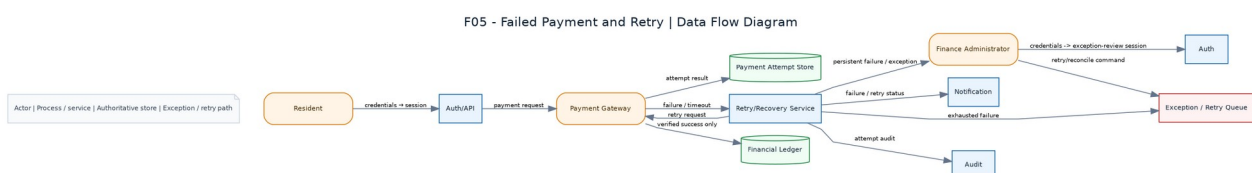


Figure F05-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F06 - Duplicate Payment Callback

Domain: Finance / Payments

Primary participants: Payment Gateway, Finance Administrator

Business objective: Make provider retries harmless to financial state.

Real-world scenario: The gateway retries the same callback after a network timeout. The second callback returns a safe replay result and creates no second receipt or journal.

Preconditions: An earlier callback with the same provider reference/idempotency key was processed or is in-flight.

Normal flow: Gateway sends callback → authentication/signature validation → idempotency lookup → existing transaction found → return prior outcome → no duplicate allocation/journal/receipt → audit replay.

Exception / alternate flow: Same reference with different amount/signature is treated as suspicious and rejected/escalated rather than merged.

State progression: RECEIVED → VERIFYING → FIRST_POST / DUPLICATE_REPLAY or SUSPICIOUS_REJECT

Key invariant: Provider reference and idempotency key uniqueness is scoped to the society/provider integration.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F06-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

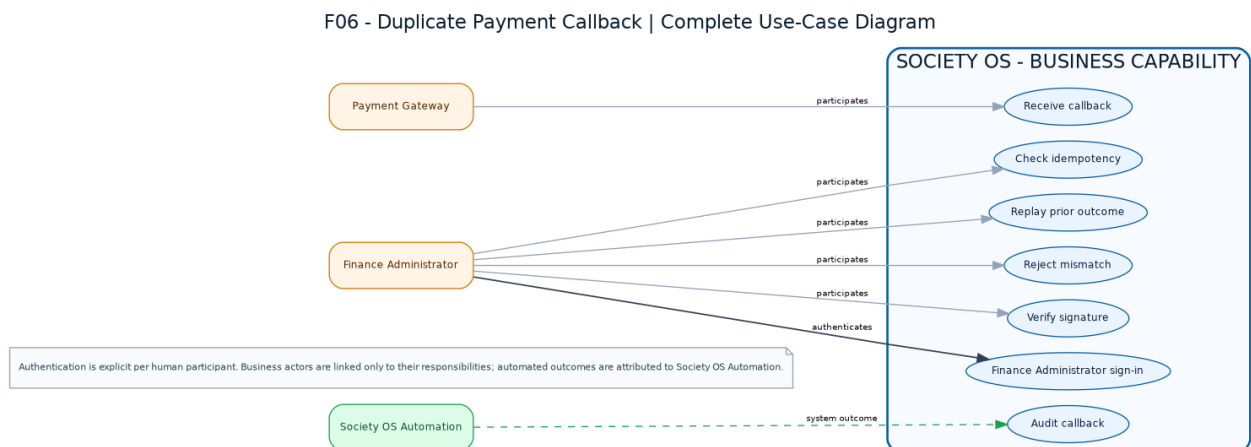
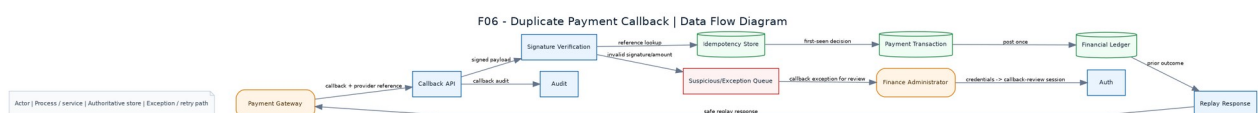


Figure F06-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F07 - Refund

Domain: Finance / Payments

Primary participants: Resident, Finance Administrator, Payment Gateway

Business objective: Return money without mutating the original posted payment history.

Real-world scenario: A ₹10,000 payment is refunded by ₹2,000 after an approved service cancellation. The original transaction remains intact and the refund is a linked compensating transaction.

Preconditions: Original payment exists; refund policy and available refundable amount are validated.

Normal flow: Resident/admin raises refund → admin signs in → eligibility checked → amount approved → gateway refund initiated → confirmation received → refund transaction posted → linked balance recalculated → receipt/notification issued.

Exception / alternate flow: Provider timeout, partial refund, repeated refund request or refund rejection must remain traceable and retry-safe.

State progression: REQUESTED → APPROVED → PROCESSING → REFUNDED/PARTIALLY_REFUNDED → RECONCILED or EXCEPTION

Key invariant: Refund amount cannot exceed refundable balance and must reference the originating payment.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F07-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

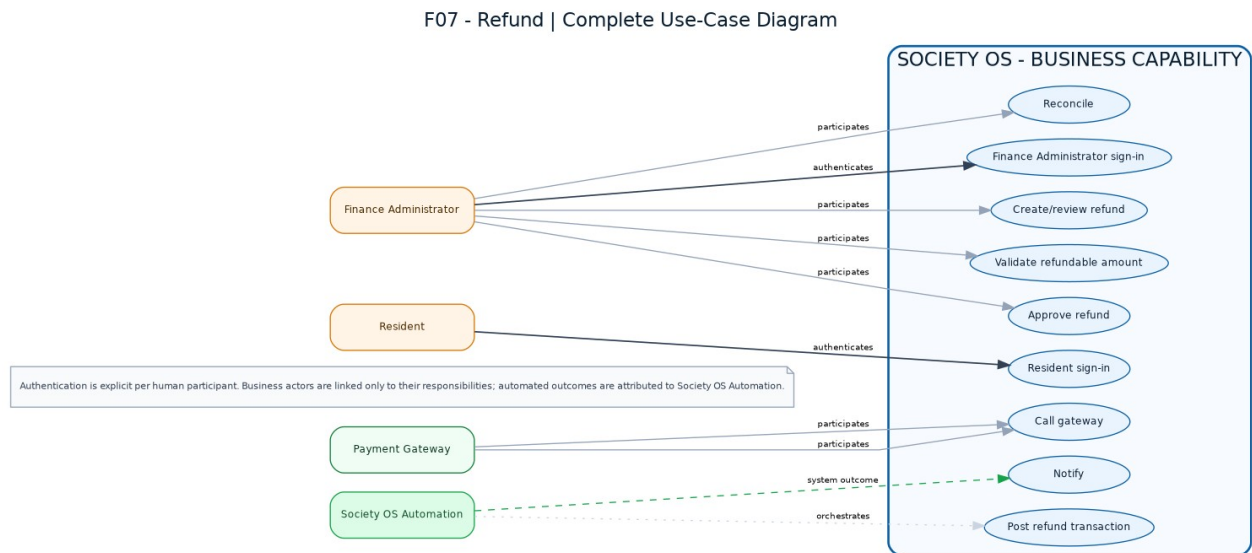
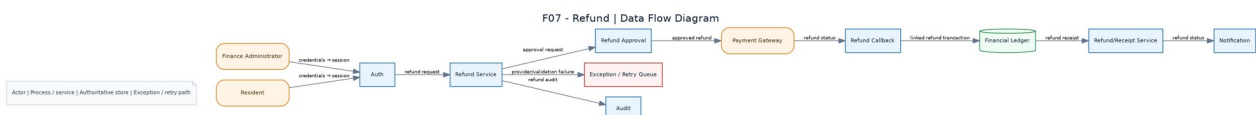


Figure F07-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F08 - Payment Reversal

Domain: Finance / Accounting

Primary participants: Finance Administrator, Resident

Business objective: Reverse an incorrect posted payment through a controlled compensating transaction.

Real-world scenario: A payment was posted to the wrong unit. Finance discovers the issue and posts a reversal, then the correct payment is allocated separately.

Preconditions: Original transaction is posted and identifiable; reversal reason is mandatory.

Normal flow: Admin signs in → selects transaction → validates reversible status → enters reason/support → approval if configured → balanced reversal journal posted → original remains unchanged → affected balances recalculate → audit captures linkage.

Exception / alternate flow: Already reversed, period closed, partial downstream allocation or missing support document requires an exception workflow.

State progression: POSTED → REVERSAL_REQUESTED → APPROVED → REVERSED

Key invariant: Original journal rows are never edited or deleted; reversal must balance and link to original transaction.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F08-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

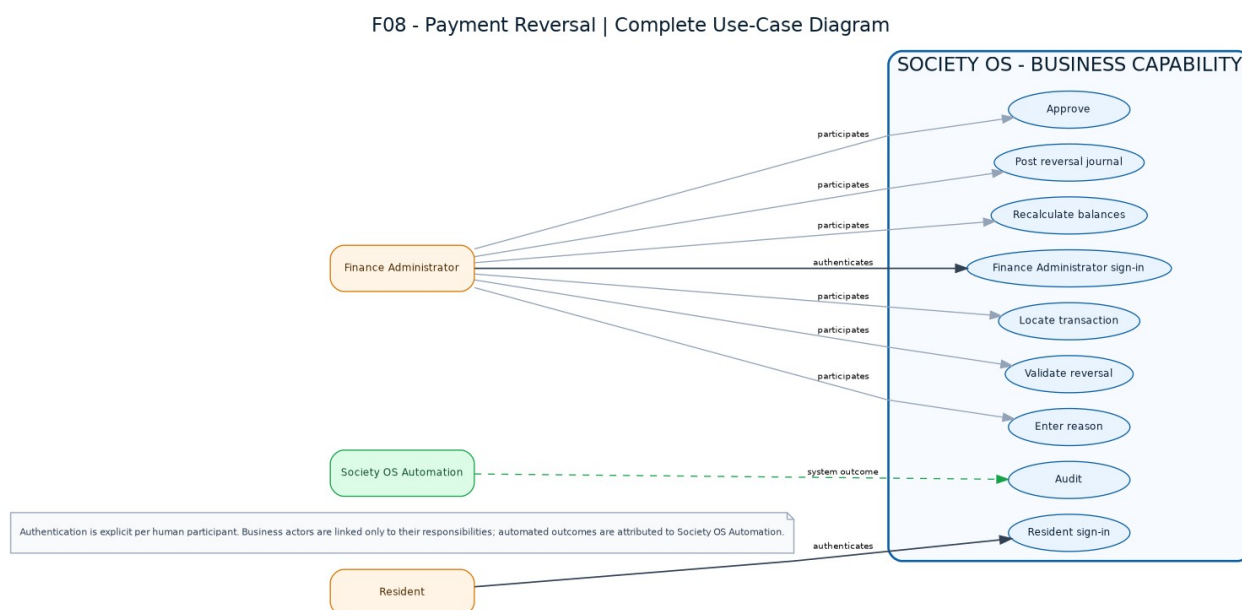
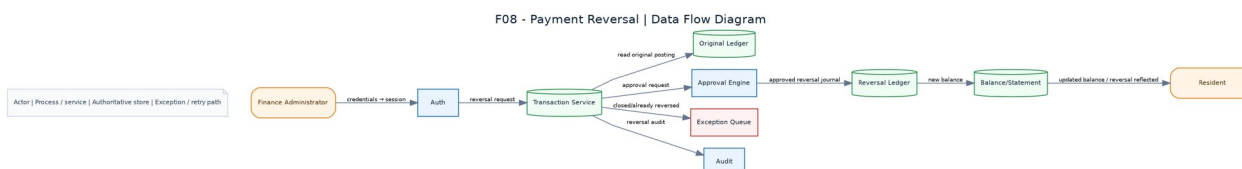


Figure F08-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F09 - Cheque Bounce

Domain: Finance / Payments

Primary participants: Finance Administrator, Resident, Bank

Business objective: Handle a cheque that was initially recorded as received but later rejected by the bank.

Real-world scenario: A ₹25,000 cheque is recorded and later bounces. The settlement effect is reversed, the outstanding amount returns, and configured bounce charges may be added.

Preconditions: Cheque payment exists in pending/received state with bank reference.

Normal flow: Bank/finance records bounce → admin signs in → system matches cheque → payment status becomes bounced → compensating entry reverses settlement → charges applied if configured → outstanding recalculated → resident notified.

Exception / alternate flow: Unknown cheque, duplicate bounce notice, already-refunded cheque or closed period require controlled exception.

State progression: RECEIVED → PRESENTED → CLEARED or BOUNCED → REVERSED/CHARGED → OUTSTANDING

Key invariant: A bounce must not silently delete the original receipt or create an unbalanced journal.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F09-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

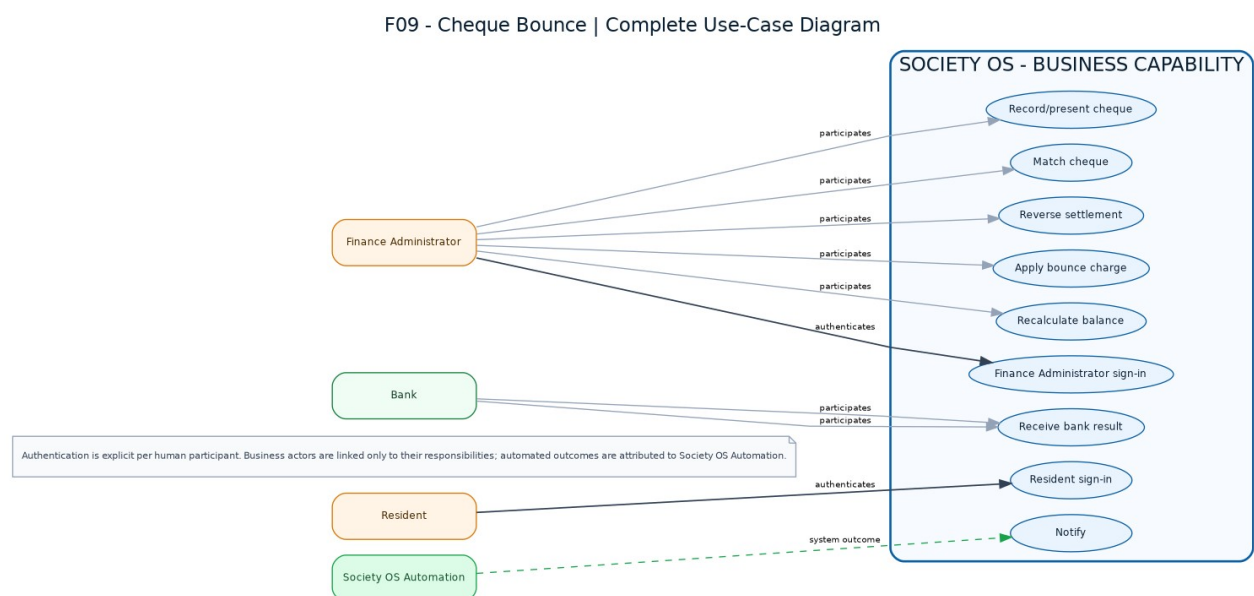
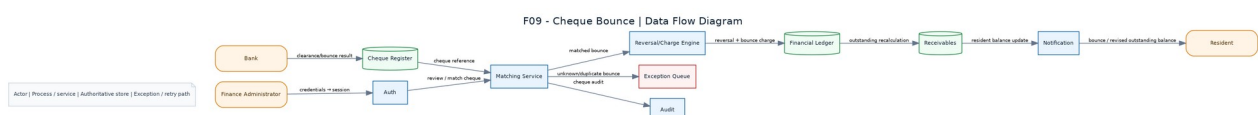


Figure F09-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F10 - Late Fee / Interest

Domain: Finance / Billing

Primary participants: Billing Administrator, Resident

Business objective: Apply overdue charges using configured effective-dated rules and transparent calculation evidence.

Real-world scenario: An invoice due on 5 October remains unpaid after the configured grace period. Late fee or interest is calculated according to the active policy.

Preconditions: Invoice is overdue; grace period and calculation basis are configured.

Normal flow: Scheduler identifies overdue invoices → rule engine evaluates grace period → calculates late fee/interest → validates cap/rounding → posts charge → notifies resident → ageing updates.

Exception / alternate flow: Holiday/cut-off boundary, partial payment, waiver, rate change or calculation correction must use the applicable effective-date rules.

State progression: DUE → GRACE → OVERDUE → CHARGE_CALCULATED → CHARGE_POSTED

Key invariant: Historical invoices must use the rule version effective for the relevant period; charges must be separately traceable.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F10-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

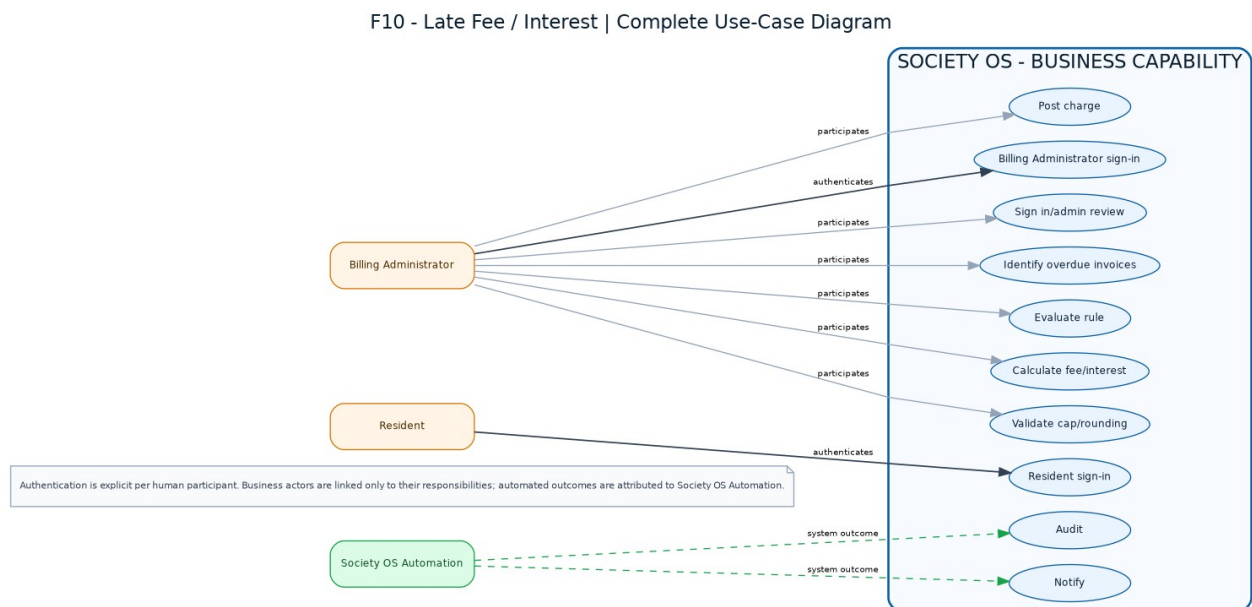
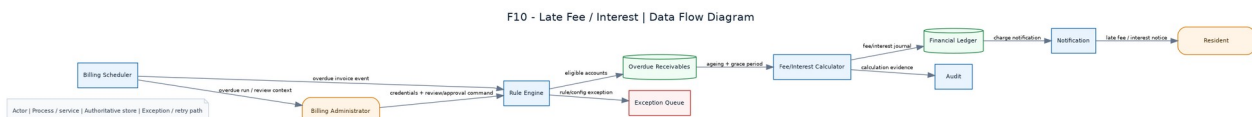


Figure F10-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F11 - Waiver / Discount

Domain: Finance / Adjustments

Primary participants: Resident, Finance Administrator, Committee Approver

Business objective: Reduce an eligible charge through an explicit, auditable waiver or discount process.

Real-world scenario: A resident requests waiver of a late fee due to a documented exceptional event. The approver reviews the request and the system posts the approved reduction.

Preconditions: Charge exists; waiver/discount policy and approval thresholds are configured.

Normal flow: Resident/admin requests → admin signs in → eligibility evaluated → approval chain runs → approved amount posted as adjustment → original charge retained → new balance/statement generated → notification.

Exception / alternate flow: Exceeds threshold, duplicate request, unsupported reason or revoked approval must not alter the ledger.

State progression: REQUESTED → UNDER_REVIEW → APPROVED/REJECTED → POSTED → RECONCILED

Key invariant: Waiver is not a silent edit; it is a separate auditable financial entry linked to the original charge.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F11-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

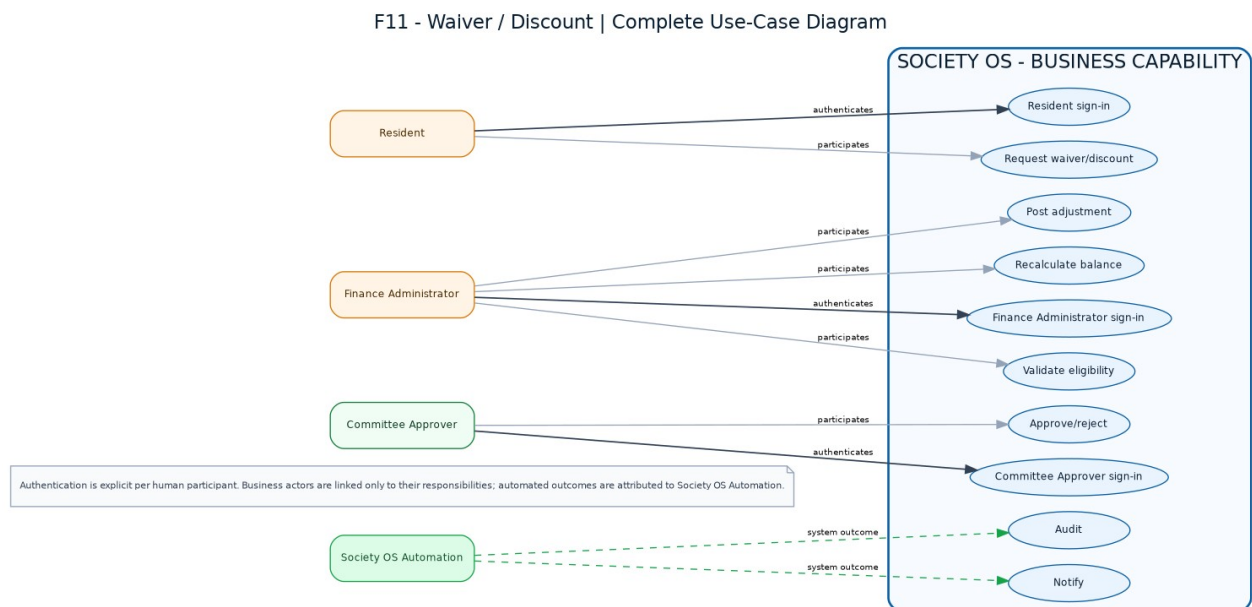
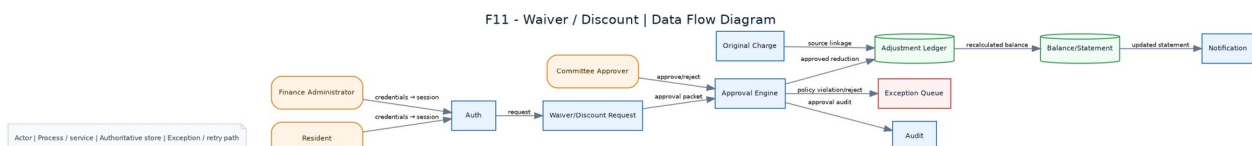


Figure F11-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F12 - Bank Reconciliation

Domain: Finance / Accounting

Primary participants: Finance Administrator, Bank

Business objective: Match bank statement entries against internal transactions and expose unresolved exceptions.

Real-world scenario: The society imports its bank statement. 97% of entries auto-match by amount/reference/date; unmatched entries enter a review queue.

Preconditions: Statement format configured; bank account and ledger mappings exist.

Normal flow: Admin signs in → imports statement → parser validates rows → matcher compares references/amounts → exact matches reconcile → unmatched/ambiguous rows queue for review → admin resolves → reconciliation closes.

Exception / alternate flow: Duplicate statement row, amount mismatch, missing bank reference, reversed payment or timing difference requires exception handling.

State progression: IMPORTED → PARSED → MATCHED/PENDING_REVIEW → RECONCILED or EXCEPTION

Key invariant: Reconciliation marks a relationship between bank evidence and internal transactions; it must not overwrite the underlying ledger.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F12-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

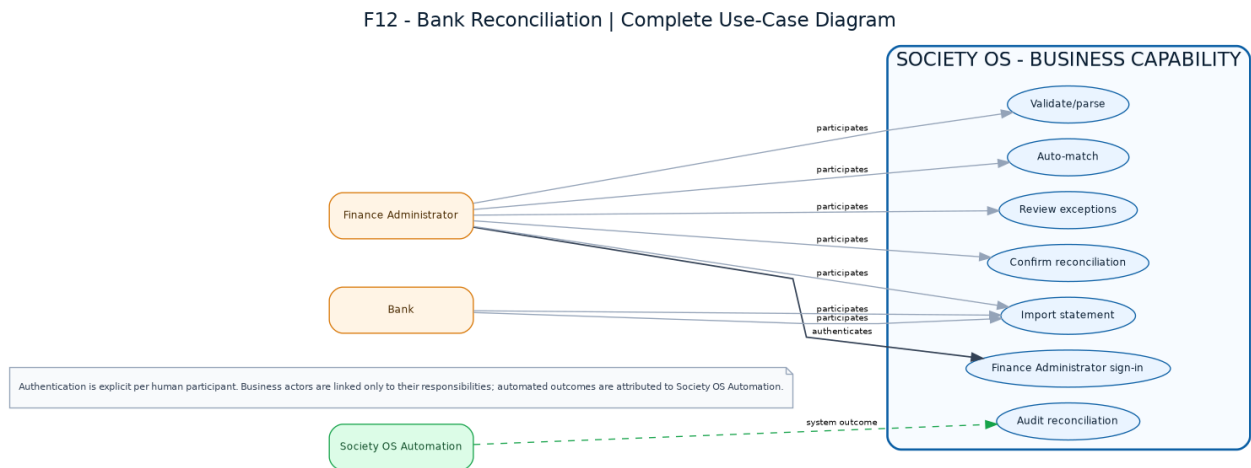
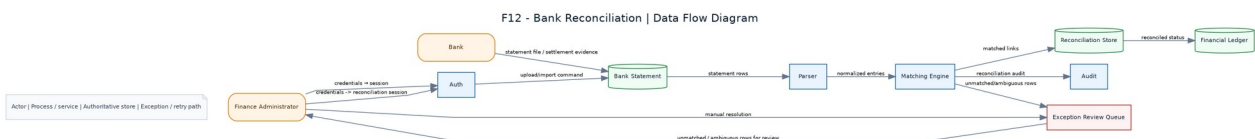


Figure F12-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F13 - Move-Out Final Settlement

Domain: Finance / Move-Out

Primary participants: Resident, Finance Administrator, Society Administrator

Business objective: Compute the final amount payable/receivable at move-out using all authoritative financial obligations.

Real-world scenario: A resident requests move-out while ₹18,000 is outstanding. Final maintenance, parking, utilities, approved damages and credits are consolidated before clearance.

Preconditions: Move-out request exists; occupancy dates and financial records are available.

Normal flow: Resident requests move-out → admin signs in → system freezes relevant period → calculates outstanding + pending charges - credits/deposits → finance reviews → payment/refund/settlement occurs → financial clearance recorded → downstream NOC can proceed.

Exception / alternate flow: Pending invoice, disputed damage, missing meter reading, unapplied credit or refund due creates a clearance exception.

State progression: REQUESTED → CALCULATING → REVIEW → SETTLEMENT_PENDING → CLEARED / EXCEPTION

Key invariant: Final settlement must be reproducible from source transactions and effective-dated rules.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F13-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

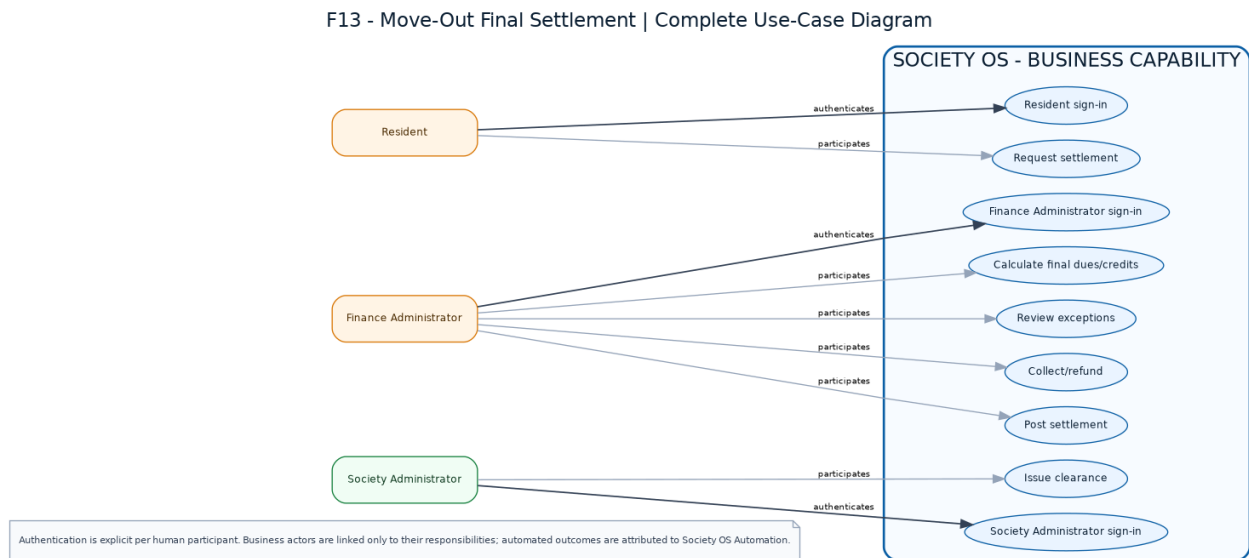
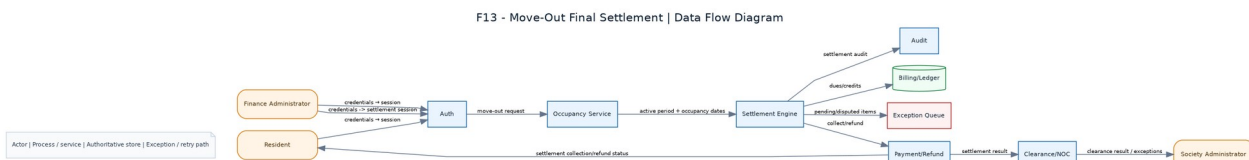


Figure F13-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F14 - Vendor Invoice / Three-Way Match

Domain: Finance / Procurement

Primary participants: Finance Administrator, Procurement Administrator, Vendor

Business objective: Pay vendors only when invoice, purchase order and goods/service receipt agree under configured tolerance.

Real-world scenario: A plumbing vendor invoices 100 valves against a PO for 100, but only 90 were received. The system blocks full payment and exposes the mismatch.

Preconditions: PO and goods receipt exist; vendor is active; invoice uploaded.

Normal flow: Admin signs in → invoice captured → OCR/manual validation → PO matched → receipt matched → tolerance evaluated → approve/reject/hold → approved liability posted → payment workflow continues.

Exception / alternate flow: Quantity mismatch, price variance, duplicate invoice, inactive vendor or missing receipt leads to hold and exception resolution.

State progression: RECEIVED → VALIDATING → MATCHED / EXCEPTION → APPROVED → PAYABLE → PAID

Key invariant: A payable cannot be approved merely because an invoice exists; required matching evidence must satisfy policy.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F14-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

F14 - Vendor Invoice / Three-Way Match | Complete Use-Case Diagram

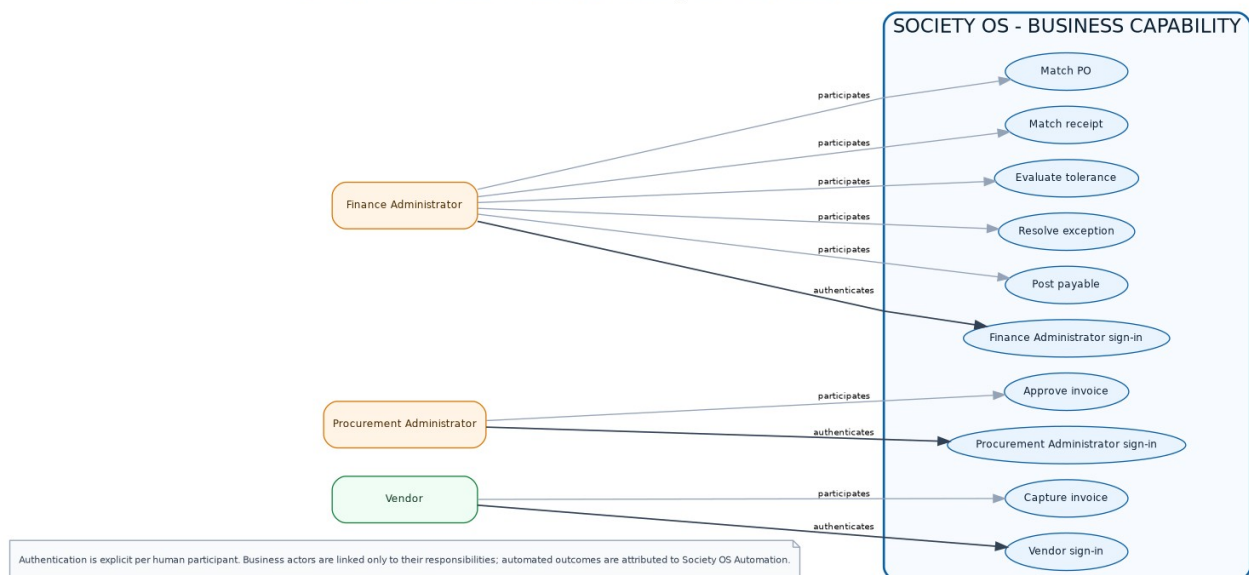
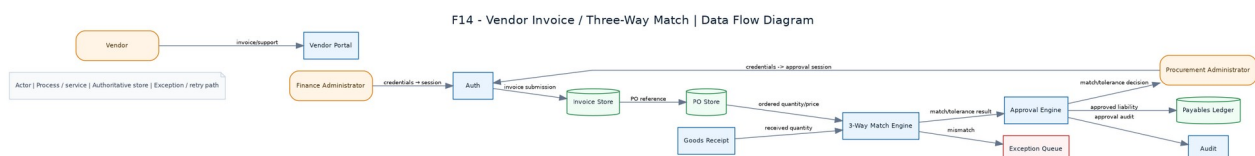


Figure F14-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F15 - Budget vs Actual

Domain: Finance / Planning

Primary participants: Finance Administrator, Committee / Management

Business objective: Compare approved budget against actual committed and paid amounts with drill-down traceability.

Real-world scenario: Management sees that lift AMC spending is 18% above budget. The report traces variance to specific vendor invoices and maintenance work orders.

Preconditions: Budget period and accounts are defined; actuals are posted.

Normal flow: Admin signs in → selects period/account → system aggregates approved budget, commitments and actuals → variance calculated → user drills from summary to source transactions → notes/actions recorded if needed.

Exception / alternate flow: Late postings, reversed invoices, budget revision or closed period must remain transparent.

State progression: BUDGET_DRAFT → APPROVED → TRACKING → VARIANCE_REVIEW → CLOSED

Key invariant: Every reported actual must be traceable to source financial transactions; budget revisions must be versioned.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F15-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

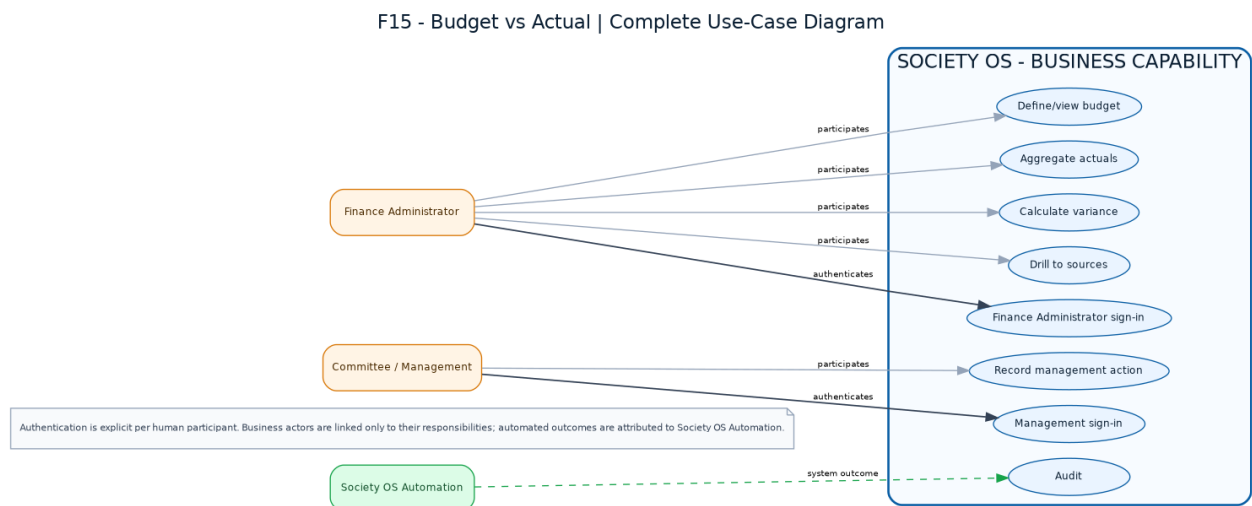
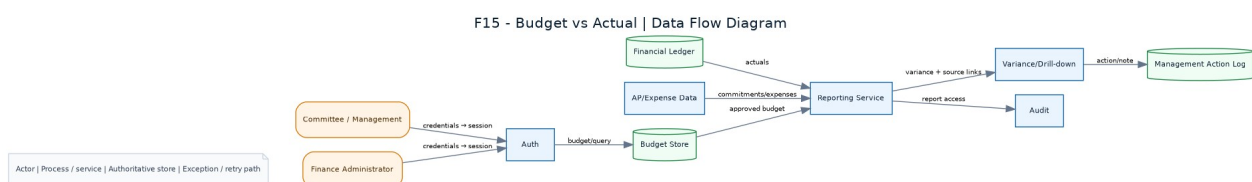


Figure F15-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F16 - Expense Reimbursement

Domain: Finance / Expenses

Primary participants: Employee / Staff, Finance Administrator, Approver

Business objective: Reimburse legitimate employee expenses through an approval and accounting workflow.

Real-world scenario: A site engineer uploads a ₹3,200 emergency plumbing purchase receipt. Finance validates the expense, approver approves it, and the reimbursement becomes payable.

Preconditions: Employee account active; expense policy configured.

Normal flow: Staff signs in → creates claim → attaches evidence → policy checks → admin/approver signs in → approves/rejects → payable posted → payment initiated → reimbursement reconciled.

Exception / alternate flow: Missing receipt, policy limit exceeded, duplicate claim or rejected payment must stop posting/payment at the correct stage.

State progression: DRAFT → SUBMITTED → REVIEW → APPROVED/REJECTED → PAYABLE → PAID/RECONCILED

Key invariant: The expense evidence and approval decision remain linked to the resulting payable.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F16-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

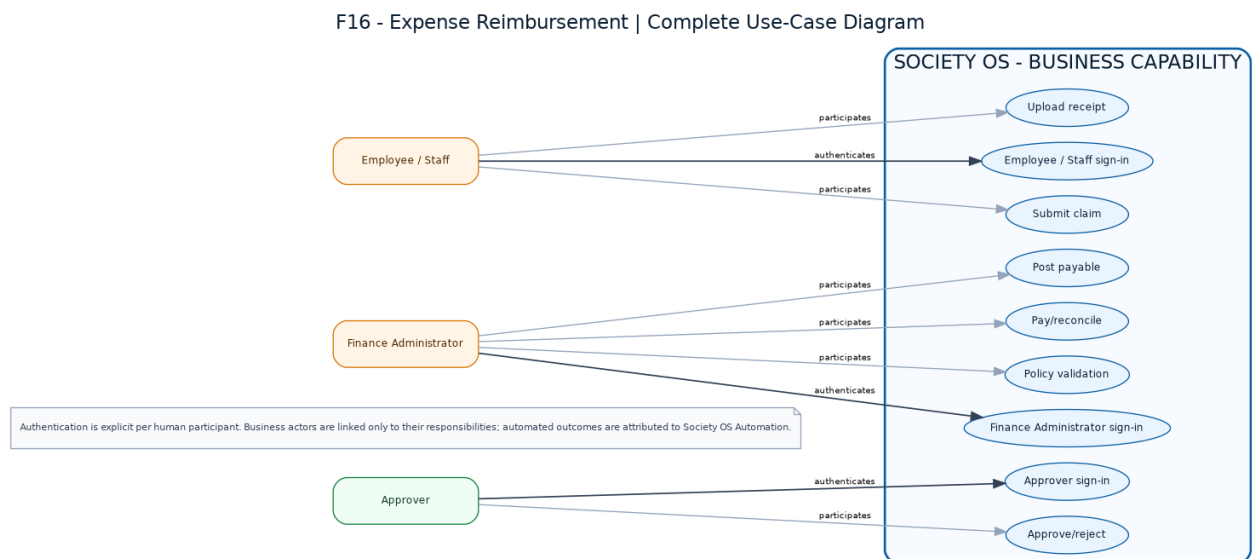
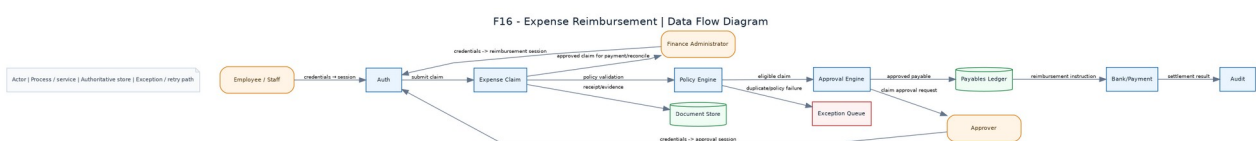


Figure F16-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F17 - Outstanding / Ageing / Collections

Domain: Finance / Collections

Primary participants: Finance Administrator, Resident, Notification Service

Business objective: Turn overdue receivables into a controlled collection workflow without losing financial traceability.

Real-world scenario: The society reviews 0-30, 31-60 and 60+ day ageing buckets, sends reminders, records commitments and escalates long-overdue units.

Preconditions: Invoices and payments are current; ageing rules configured.

Normal flow: Admin signs in → ageing snapshot generated → residents grouped by bucket → notices sent → commitments/disputes recorded → payment updates ageing → unresolved accounts escalate according to policy.

Exception / alternate flow: Disputed charge, legal hold, incorrect allocation or payment pending settlement must be excluded or flagged rather than incorrectly escalated.

State progression: CURRENT → DUE → OVERDUE → REMINDED → ESCALATED → RESOLVED / DISPUTED

Key invariant: Ageing is derived from authoritative receivables, not manually maintained balances.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F17-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

F17 - Outstanding / Ageing / Collections | Complete Use-Case Diagram

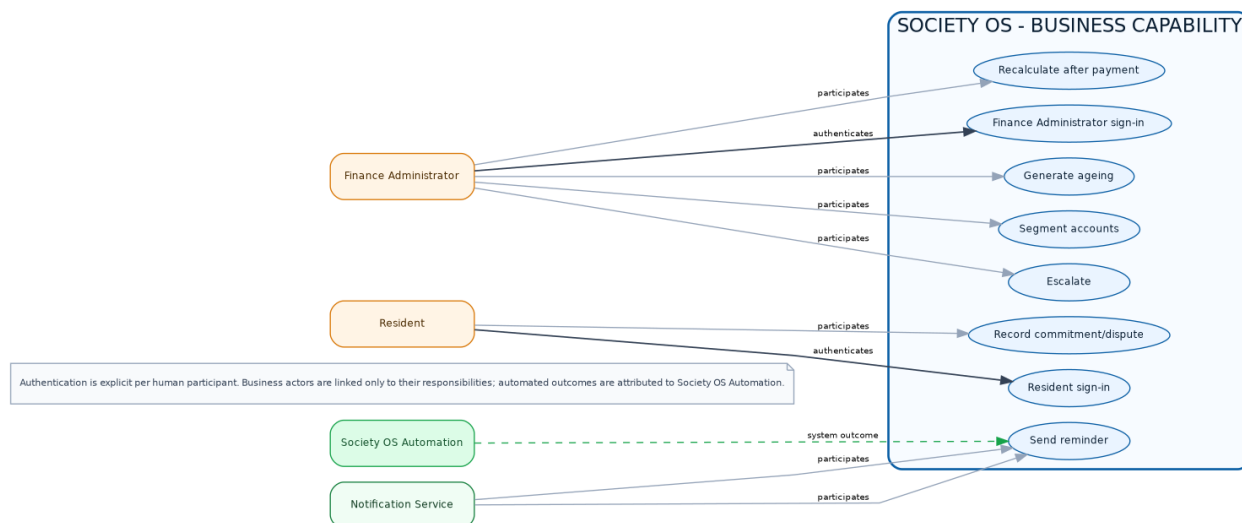
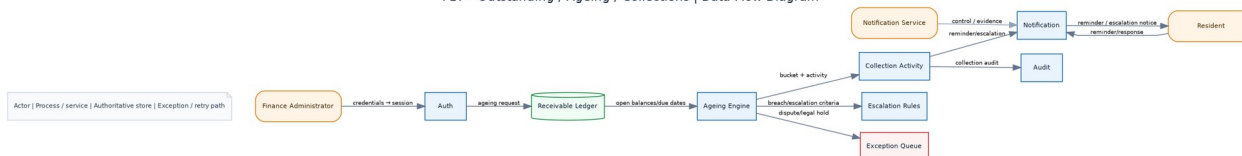


Figure F17-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.

F17 - Outstanding / Ageing / Collections | Data Flow Diagram



F18 - Credit Note / Debit Adjustment

Domain: Finance / Adjustments

Primary participants: Finance Administrator, Approver

Business objective: Correct a bill or account through a linked adjustment transaction rather than rewriting history.

Real-world scenario: A resident was charged ₹1,500 for a service that was cancelled. Finance issues a ₹1,500 credit note against the original charge.

Preconditions: Original invoice/charge exists and adjustment policy is active.

Normal flow: Admin signs in → selects source charge → chooses credit/debit reason → approval runs if required → adjustment posted → balance recalculated → revised statement references original charge and adjustment.

Exception / alternate flow: Source not found, amount exceeds permitted correction, duplicate adjustment or closed period creates exception.

State progression: REQUESTED → REVIEW → APPROVED → POSTED → RECONCILED

Key invariant: Every adjustment carries source linkage, reason and approval evidence as required.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F18-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

F18 - Credit Note / Debit Adjustment | Complete Use-Case Diagram

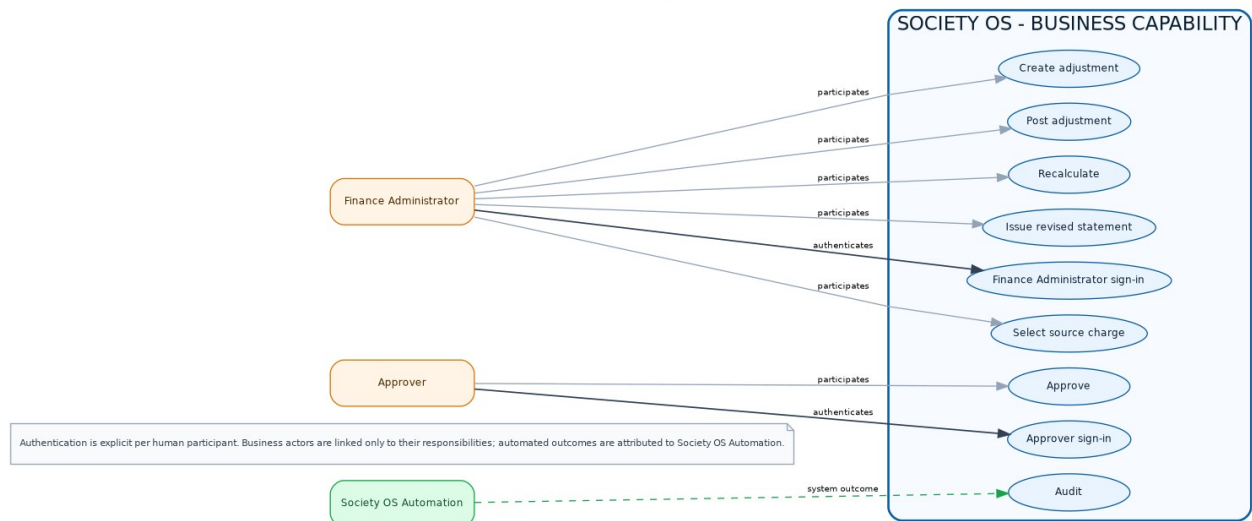
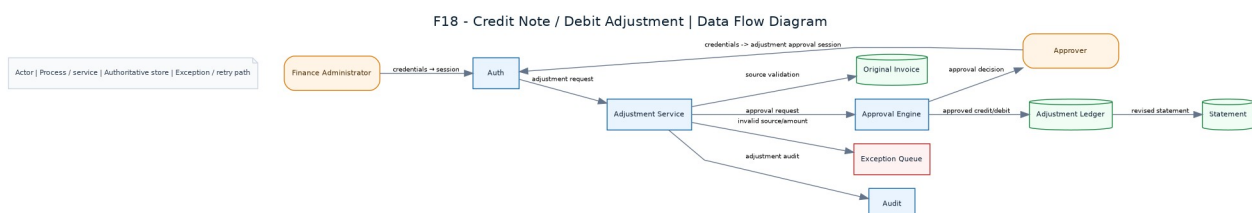


Figure F18-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F19 - Month-End Close

Domain: Finance / Closing

Primary participants: Finance Administrator, Society Management

Business objective: Close a financial period only after defined validation and reconciliation checks pass.

Real-world scenario: At month end, finance verifies bank reconciliation, open payment callbacks, unposted expenses, suspense balances and draft adjustments before closing September.

Preconditions: Period exists and close checklist is configured.

Normal flow: Admin signs in → close checklist generated → exceptions reviewed → reconciliation checks run → pending jobs verified → period locked → closing snapshot recorded → reports generated.

Exception / alternate flow: Open settlement, unresolved bank item, failed batch job or attempted post-close transaction must block or create controlled approval.

State progression: OPEN → CHECKING → EXCEPTIONS_REVIEW → READY_TO_CLOSE → CLOSED

Key invariant: Post-close transactions follow explicit reopening/adjustment policy and never silently rewrite closed data.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F19-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

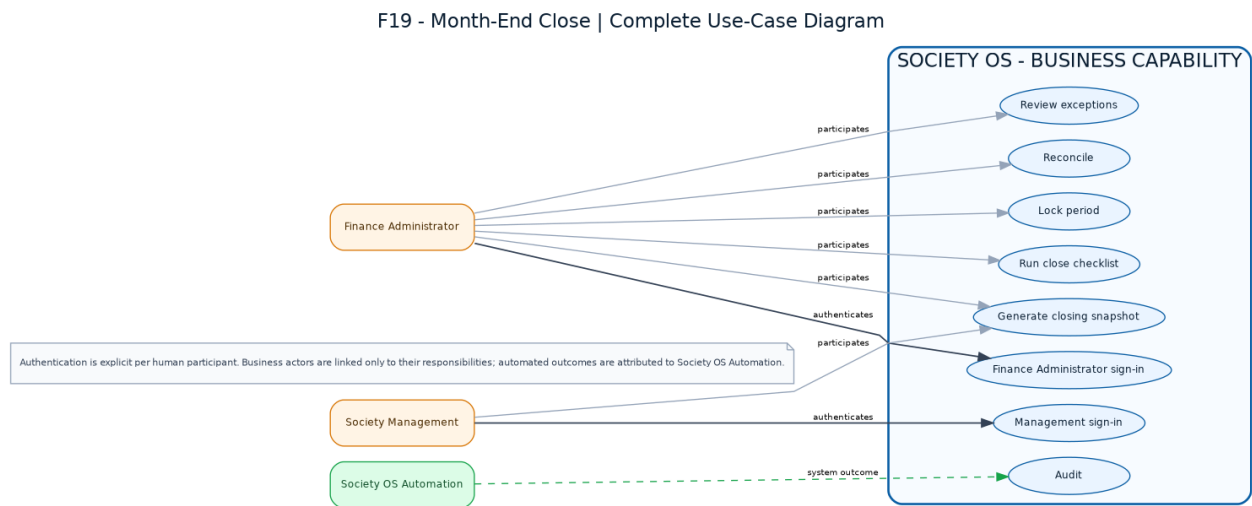
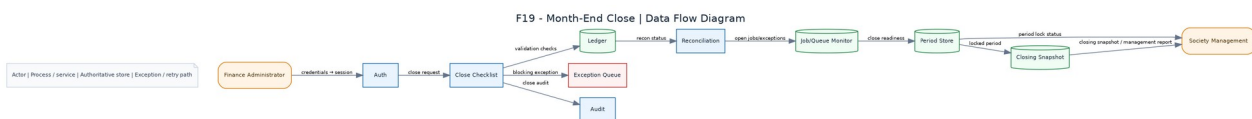


Figure F19-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



F20 - Financial Data Import / Opening Balances

Domain: Finance / Migration

Primary participants: Migration Administrator, Finance Administrator

Business objective: Load legacy financial balances and validate them before becoming opening balances.

Real-world scenario: A new society imports 5,000 resident records and opening balances. 4,912 pass validation and 88 rows require correction before commit.

Preconditions: Import template and mapping are configured; source file available.

Normal flow: Admin signs in → upload → schema validation → duplicate check → account/unit mapping → debit/credit balance validation → preview → confirm → transactional import → exception report → opening balances posted.

Exception / alternate flow: Unmapped unit, duplicate account, unbalanced opening trial, invalid amount or partial import failure requires rollback/compensation.

State progression: UPLOADED → VALIDATED → PREVIEWED → COMMITTING → COMMITTED / FAILED_WITH_ERRORS

Key invariant: Opening balances must balance and remain traceable to the import batch and source row.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure F20-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

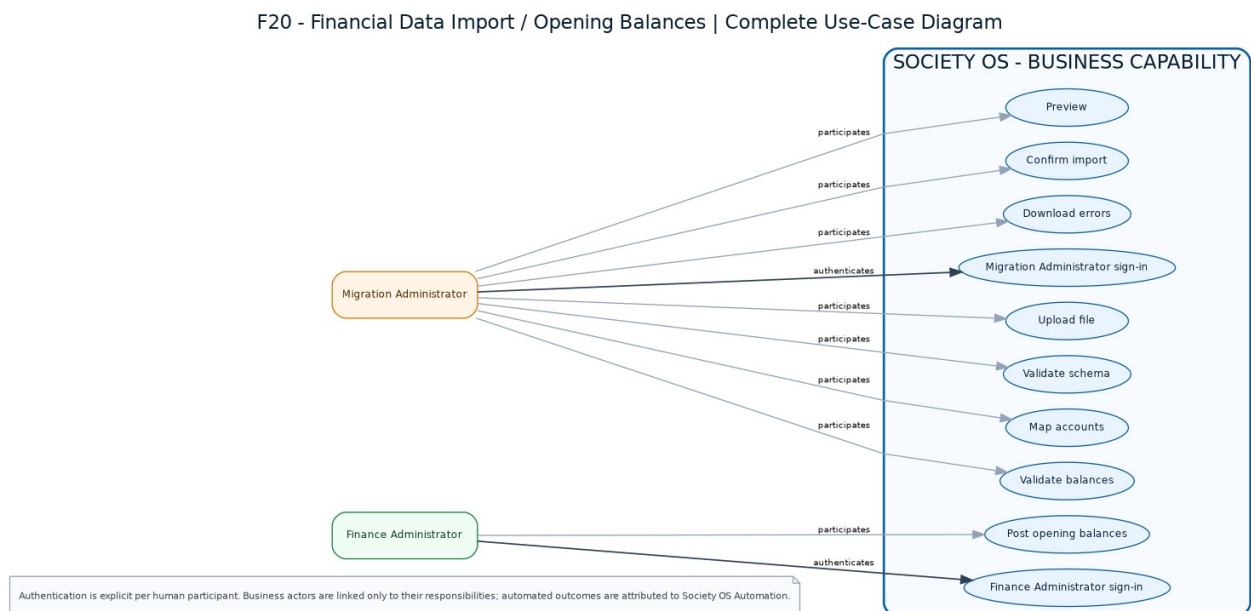
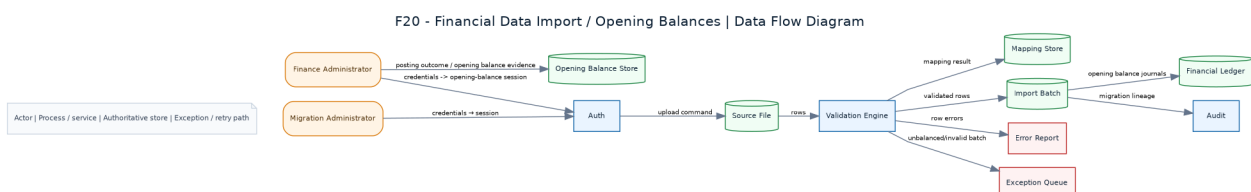


Figure F20-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



001 - Society Onboarding

Domain: Platform / Onboarding

Primary participants: Society Administrator, Platform Administrator

Business objective: Create a fully configured society tenant from legal identity through activation.

Real-world scenario: A six-tower Maharashtra society is created with registration details, IST, FY rules, units, parking, amenities, charge heads and notification defaults before activation.

Preconditions: Authorized onboarding user exists.

Normal flow: Admin signs in → creates society → captures registration/address → selects rule pack/FY/timezone → builds property hierarchy → imports units/parking → configures charge heads and policies → validates readiness → activates tenant.

Exception / alternate flow: Duplicate slug, incomplete property tree, conflicting unit numbers or missing mandatory configuration blocks activation.

State progression: DRAFT → CONFIGURING → VALIDATING → READY → ACTIVE

Key invariant: Tenant cannot be active until mandatory structural and configuration validations pass.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure 001-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

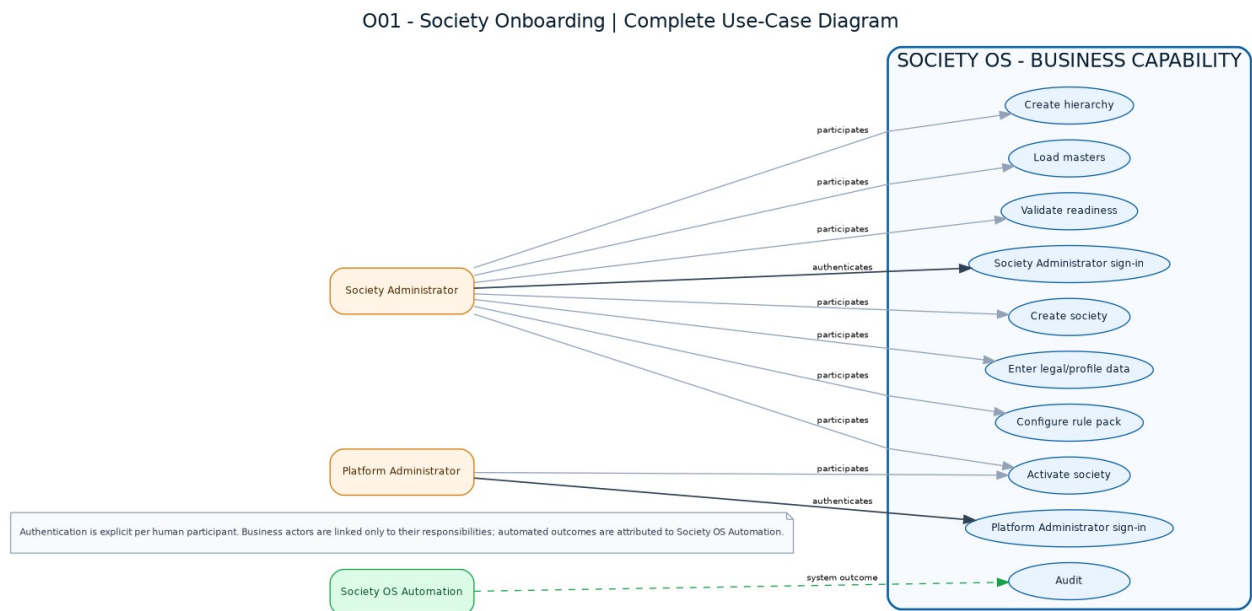
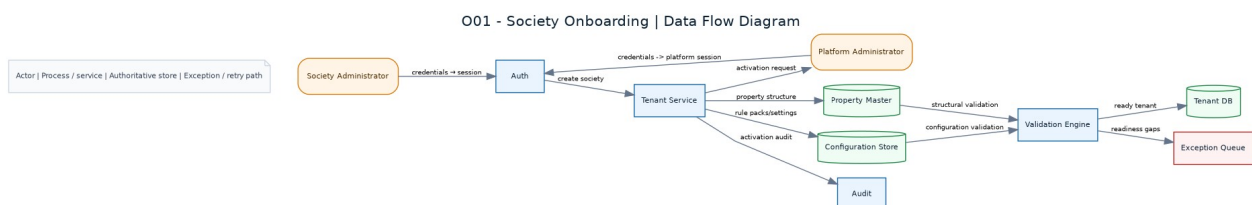


Figure 001-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O02 - Resident Registration and Verification

Domain: Identity / Resident

Primary participants: Resident, Society Administrator, Identity/Email Provider

Business objective: Register a resident, verify identity and obtain society approval where configured.

Real-world scenario: A tenant receives an invitation, verifies mobile by OTP, uploads KYC/agreement, and waits for the admin to approve. Admin must sign in before verification/approval.

Preconditions: Invitation or open-registration policy is enabled; unit exists.

Normal flow: Resident signs in/registers → OTP/email verified → profile completed → unit/relationship selected → documents submitted → admin signs in → admin reviews identity/documents → approve/reject/resubmit → optional admin digital approval/signature → account activates.

Exception / alternate flow: Duplicate phone, wrong unit, missing document, rejected KYC or expired invitation requires resubmission without creating duplicate occupancy.

State progression: INVITED → REGISTERED → IDENTITY_VERIFIED → DOCUMENTS_SUBMITTED → ADMIN_REVIEW → APPROVED/RESUBMIT/REJECTED → ACTIVE

Key invariant: Registration identity state is separate from occupancy; admin approval must be authenticated and auditable.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O02-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

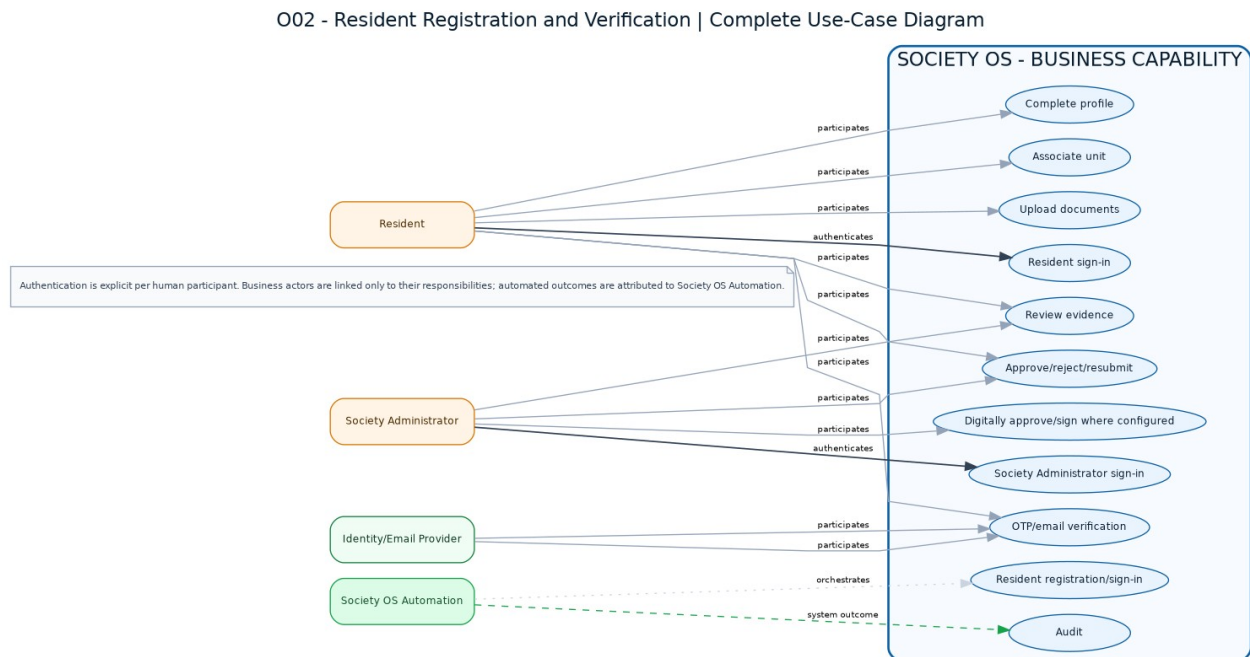
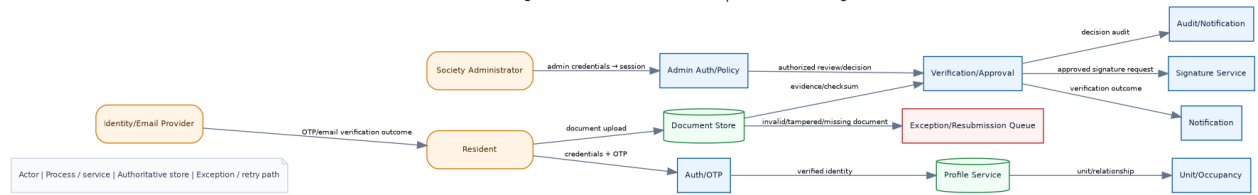


Figure O02-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.

O02 - Resident Registration and Verification | Data Flow Diagram



O03 - Owner/Tenant/Household Lifecycle

Domain: Identity / Occupancy

Primary participants: Resident, Society Administrator

Business objective: Maintain accurate owner, tenant, co-owner and household relationships over time.

Real-world scenario: An owner rents Unit A-1203 to Tenant B, later terminates the tenancy and moves back in. Historical occupancy and household relationships remain intact.

Preconditions: Unit master exists; identity records exist or are being created.

Normal flow: Resident/admin signs in → create/modify relationship → validate overlap dates → approve if configured → update active occupancy/household → close prior relationship at effective date → audit timeline.

Exception / alternate flow: Overlapping occupancies, deleting historical members, orphaned household members or privacy leakage must be prevented.

State progression: REQUESTED → VALIDATED → ACTIVE → ENDED / SUPERSEDED

Key invariant: Historical relationships are immutable facts with effective dates; current active relationships are derived from valid periods.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O03-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

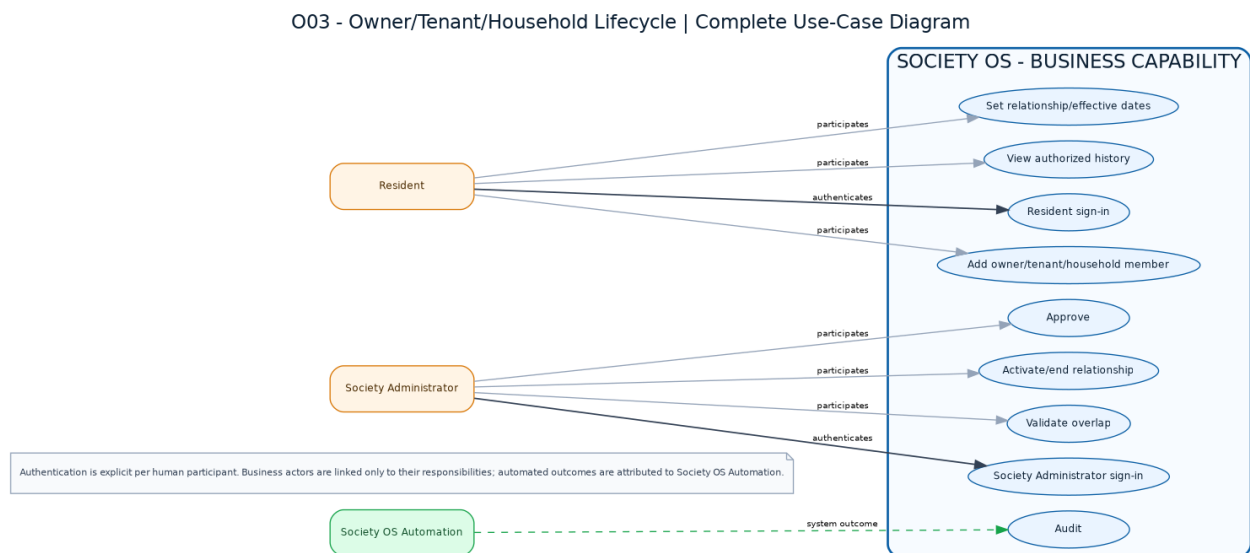
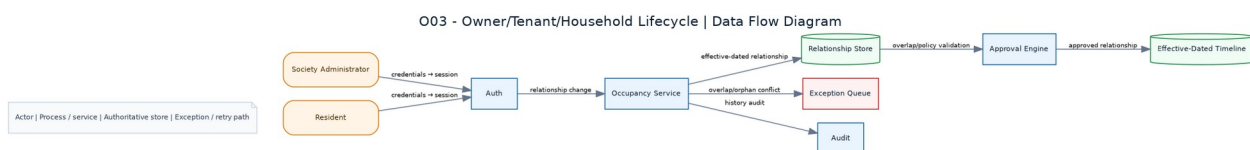


Figure O03-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O04 - Visitor Pre-Approval and Gate Check-In

Domain: Gate / Visitor

Primary participants: Resident, Guard, Gate Device

Business objective: Allow an approved visitor to enter with rapid, traceable gate validation.

Real-world scenario: A resident pre-approves a guest for 7:00-10:00 PM. At 7:35 PM the guard scans the pass, validates identity and records entry.

Preconditions: Visitor pass exists and is within validity window.

Normal flow: Resident signs in → creates invitation → system issues pass → visitor arrives → guard signs in/device authenticates → pass scanned → gate validates unit, window, status and restrictions → entry logged → pass state updates → resident notified.

Exception / alternate flow: Expired/revoked pass, duplicate entry, resident request to deny entry or offline gate operation must be handled safely.

State progression: CREATED → APPROVED → PRESENTED → CHECKED_IN → CHECKED_OUT / EXPIRED / REVOKED

Key invariant: Gate admission is security-sensitive and server-authoritative when current connectivity is available.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O04-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

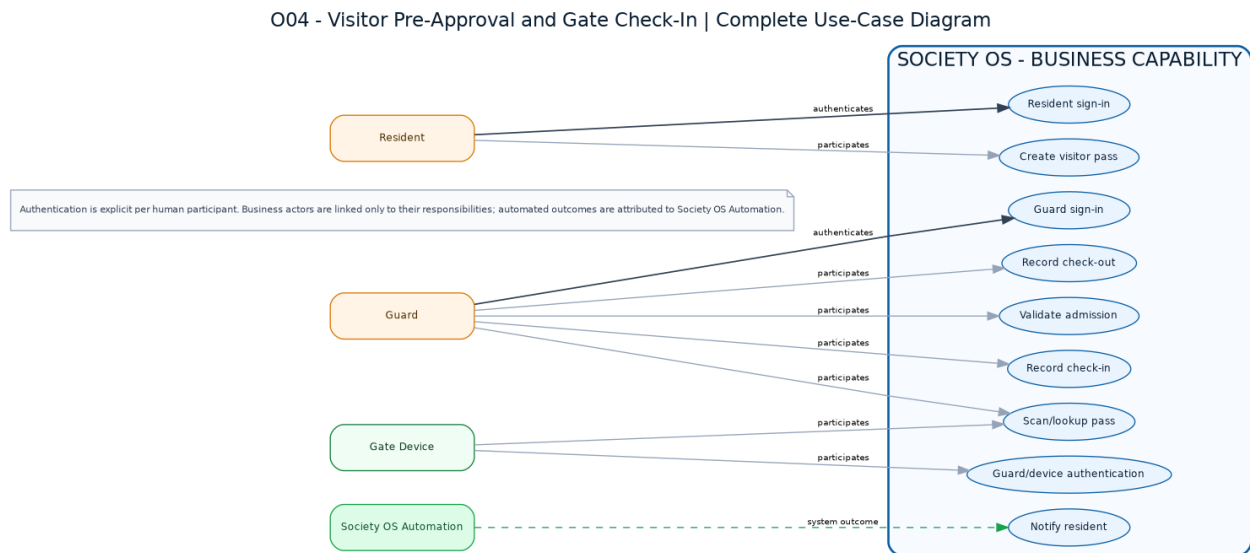
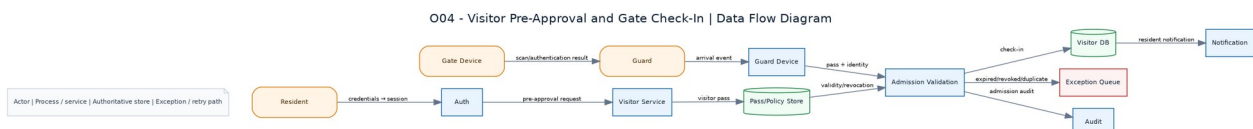


Figure O04-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O05 - Unknown Visitor / Resident Unreachable

Domain: Gate / Security

Primary participants: Guard, Resident, Security Administrator

Business objective: Resolve an unapproved visitor request without converting non-response into automatic approval.

Real-world scenario: A plumber arrives for Unit B-804 without pre-approval; the resident does not answer. The guard follows the configured deny/wait/escalate policy.

Preconditions: Visitor is not pre-approved; emergency override policy may or may not apply.

Normal flow: Guard signs in → searches unit → system shows approval state → calls/request sent → timer starts → resident responds approve/deny or remains unreachable → configured policy decides wait/deny/escalate → decision and reason logged.

Exception / alternate flow: Emergency arrival, false identity, suspicious behaviour or resident response after denial require explicit exception handling.

State progression: PRESENTED → AWAITING_RESIDENT → APPROVED / DENIED / ESCALATED / EMERGENCY_BYPASS

Key invariant: No response is not equivalent to approval.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O05-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

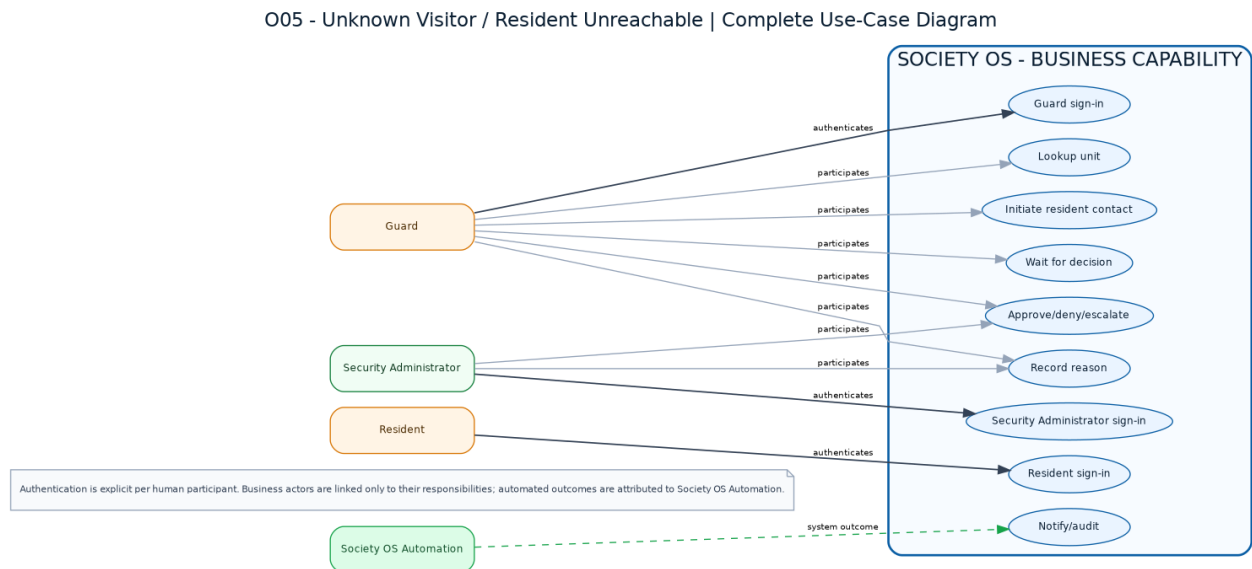
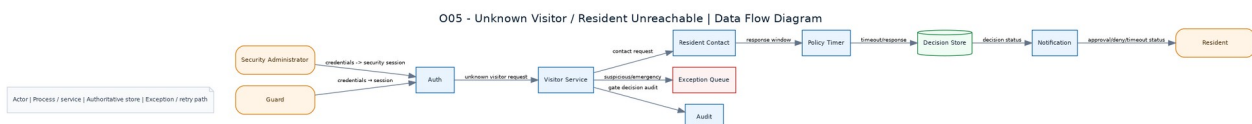


Figure O05-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O06 - Complaint Lifecycle and SLA

Domain: Helpdesk

Primary participants: Resident, Helpdesk Administrator, Technician

Business objective: Manage a complaint from creation to verified resolution with measurable SLA.

Real-world scenario: A resident reports a lift stopping at the 7th floor, adds a video, the ticket is assigned to a technician, placed on hold for a spare, resolved and later confirmed by the resident.

Preconditions: Complaint categories and SLA policies exist.

Normal flow: Resident signs in → creates complaint → classification/SLA calculated → admin/engine assigns → technician acknowledges → work starts → wait/hold as needed → resolution evidence added → resident notified → resident confirms or ticket closes → SLA metrics recorded.

Exception / alternate flow: Duplicate, parent incident, SLA breach, reassignment, reopen or missing resolution evidence must follow explicit rules.

State progression: CREATED → CLASSIFIED → ASSIGNED → ACKNOWLEDGED → IN_PROGRESS → WAITING/HOLD → RESOLVED → REOPENED/CLOSED

Key invariant: SLA clock treatment must be deterministic for hold, pause and reopen semantics.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O06-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

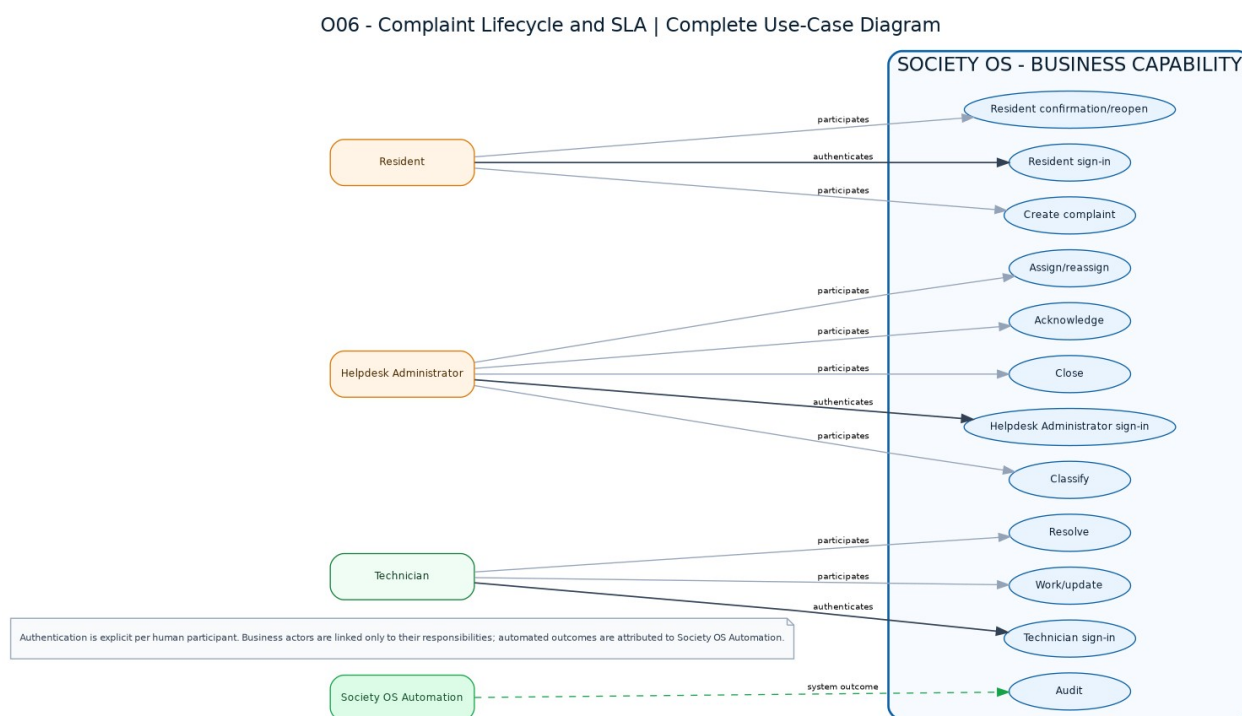
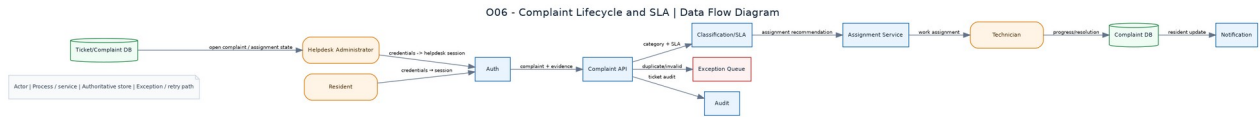


Figure O06-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O07 - Parent Incident / Duplicate Complaints

Domain: Helpdesk / Incidents

Primary participants: Resident, Helpdesk Administrator, Technician

Business objective: Correlate multiple complaints into a parent incident while preserving individual reporters.

Real-world scenario: Ten residents report low water pressure in Tower C within 20 minutes. The platform proposes a parent incident and broadcasts common updates.

Preconditions: Multiple related complaints exist; correlation rules enabled.

Normal flow: Complaints arrive → correlation engine evaluates location/category/time → parent incident suggested/created → admin signs in/reviews → linked complaints retain reporter evidence → common updates distributed → parent resolution closes/reconciles children.

Exception / alternate flow: False correlation, complaint-specific resolution or incident split must be supported.

State progression: CHILD_CREATED → CORRELATION_CANDIDATE → LINKED_PARENT → IN_PROGRESS → RESOLVED → CLOSED

Key invariant: Linking never deletes individual complaint history or reporter identity.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O07-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

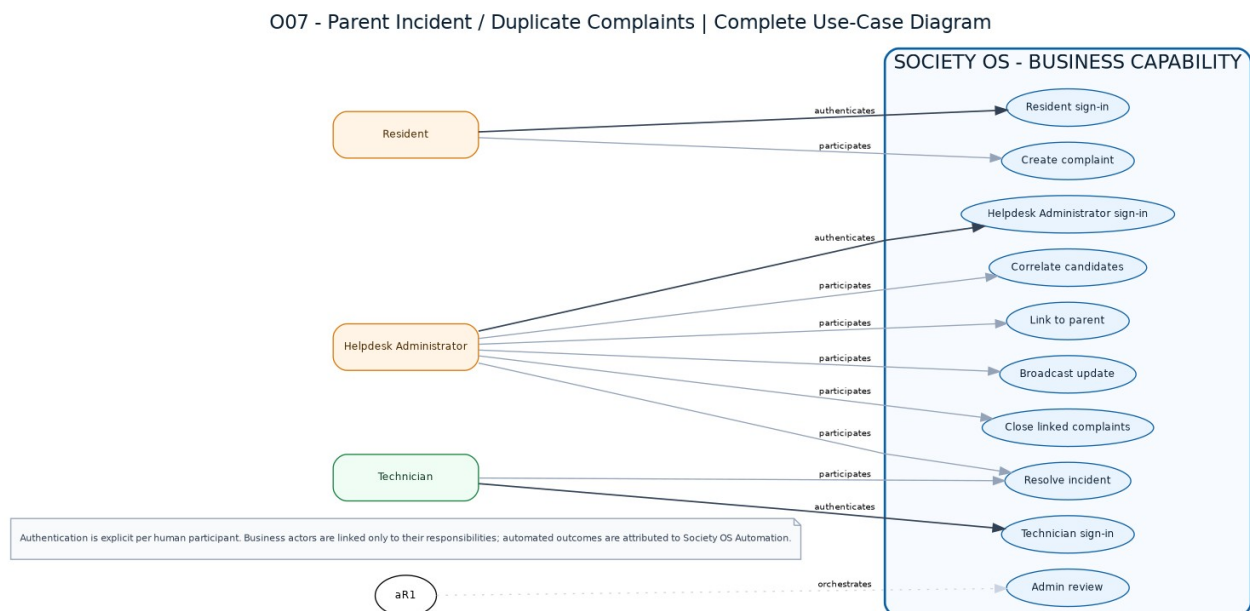
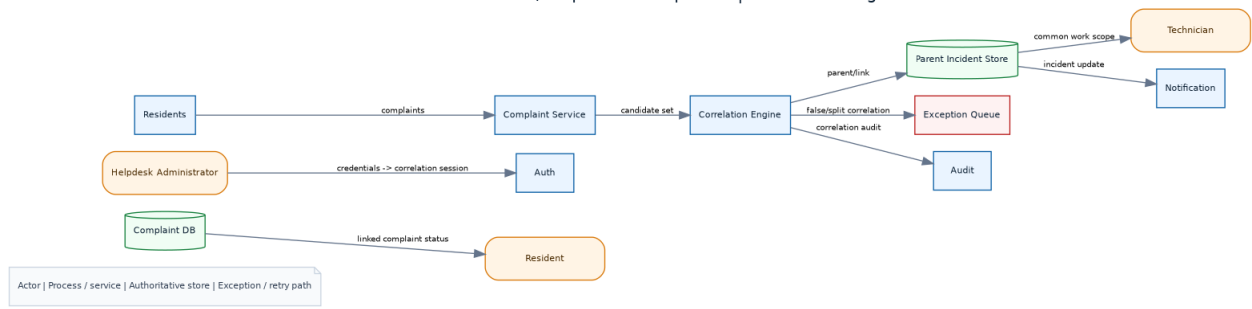


Figure O07-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.

O07 - Parent Incident / Duplicate Complaints | Data Flow Diagram



O08 - Preventive Maintenance

Domain: Maintenance

Primary participants: Maintenance Manager, Technician, Vendor

Business objective: Convert asset maintenance plans into scheduled, auditable work orders.

Real-world scenario: A generator has weekly inspections, monthly test runs and annual servicing. The scheduler creates work orders and advances the next-due dates.

Preconditions: Assets and maintenance plans exist; technician/vendor availability is known.

Normal flow: Manager signs in → defines/edits plan → scheduler generates due work order → assigns technician/vendor → inspection/service performed → parts/cost recorded → completion evidence captured → next due calculated → asset history updated.

Exception / alternate flow: Missed schedule, vendor no-show, failed inspection or parts shortage must create follow-up tasks rather than silently closing work.

State progression: PLAN_ACTIVE → WORK_ORDER_CREATED → ASSIGNED → IN_PROGRESS → COMPLETED / FAILED → NEXT_DUE_SCHEDULED

Key invariant: Every completed maintenance event must remain linked to asset, plan, worker/vendor and evidence.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O08-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

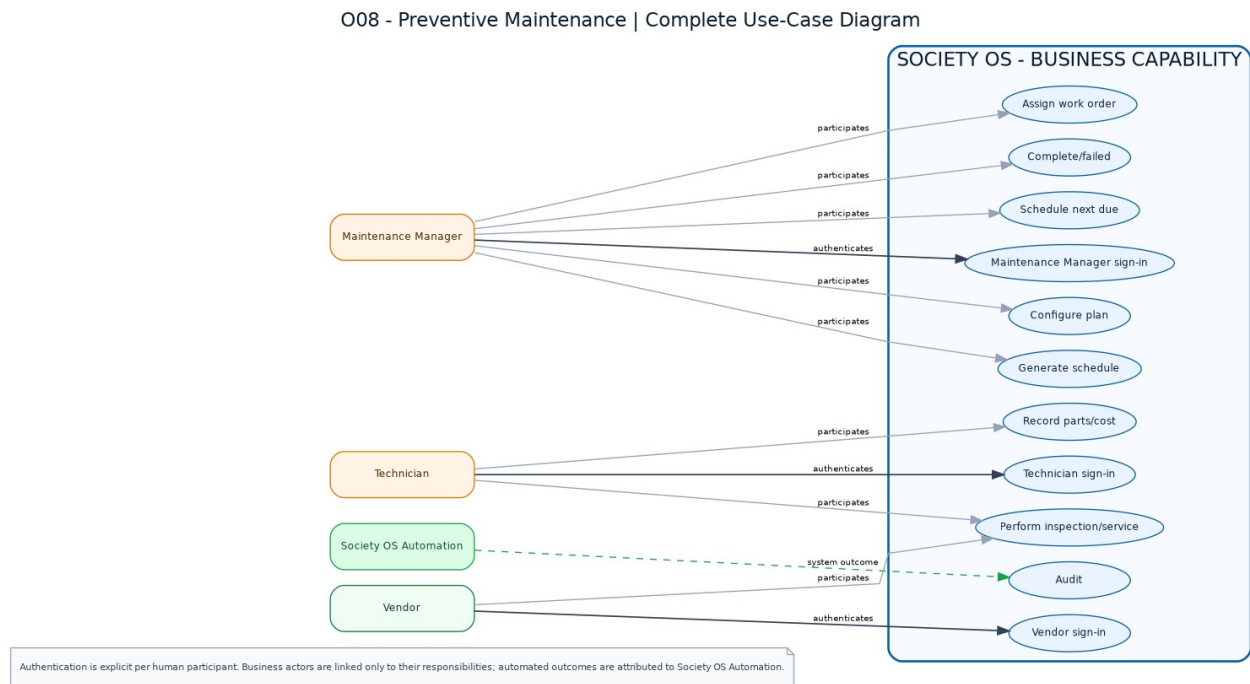
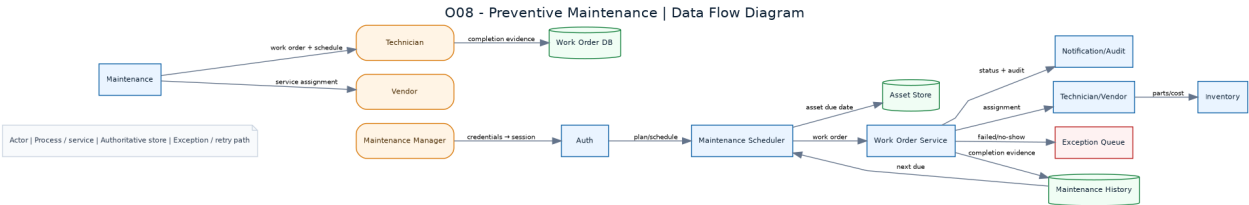


Figure O08-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O09 - Inventory Reorder and Stock Issue

Domain: Inventory / Procurement

Primary participants: Storekeeper, Maintenance Technician, Procurement Administrator

Business objective: Maintain accurate stock and replenish materials based on reorder thresholds.

Real-world scenario: A pump repair consumes the last spare seal below reorder level. The storekeeper issues stock, and procurement raises a purchase request.

Preconditions: Item master, stock locations and reorder levels exist.

Normal flow: Storekeeper signs in → view stock → issue material to work order → stock decremented → reorder rule triggered → purchase request created → approval/PO → receipt → stock incremented.

Exception / alternate flow: Insufficient stock, duplicate receipt, damaged material or serial mismatch must be recorded explicitly.

State progression: AVAILABLE → LOW_STOCK → REQUESTED → ORDERED → RECEIVED → AVAILABLE

Key invariant: Stock balance is derived from controlled movements, not manual overwrites.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O09-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

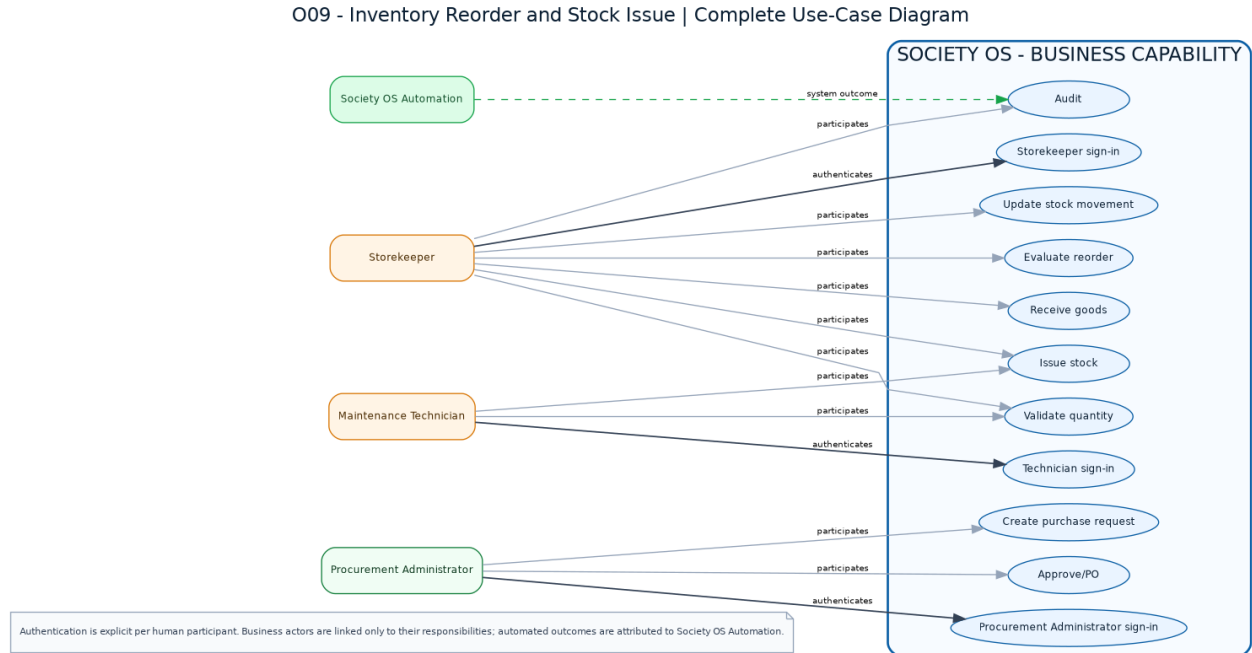
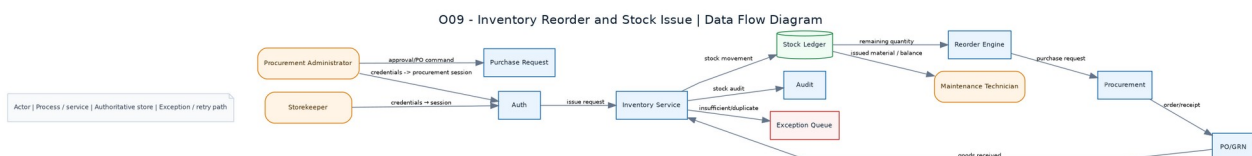


Figure O09-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O10 - Vendor Onboarding and AMC Renewal

Domain: Vendor / Contracts

Primary participants: Society Administrator, Vendor, Finance Administrator

Business objective: Onboard vendors, validate documents and manage AMC lifecycle to renewal/termination.

Real-world scenario: A lift AMC expires on 31 March. The system reminds the society, captures renewal terms and preserves the previous contract version.

Preconditions: Vendor category and required document rules exist.

Normal flow: Admin signs in → creates vendor → collects KYC/GST/PAN/bank/docs → validates → contract/AMC created → service history tracked → renewal reminder → new version/contract approved → expiry/renewal state updated.

Exception / alternate flow: Expired insurance, failed verification, price change or missed renewal must escalate.

State progression: DRAFT → VERIFICATION → ACTIVE → EXPIRING → RENEWED / EXPIRED / TERMINATED

Key invariant: Vendor master changes and contract versions are historical; active contract validity must be unambiguous.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O10-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

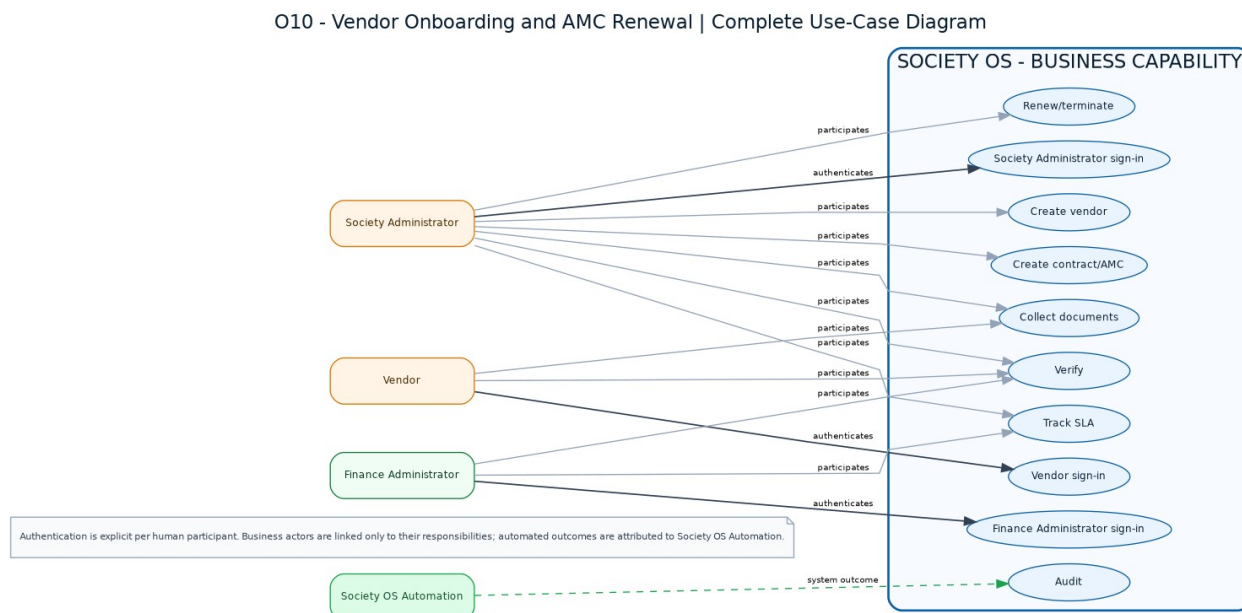
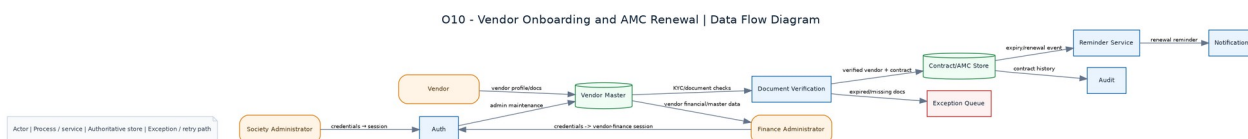


Figure O10-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O11 - Facility Booking Conflict

Domain: Facilities

Primary participants: Resident, Facility Administrator

Business objective: Reserve shared facilities atomically while enforcing time, capacity and blackout rules.

Real-world scenario: A resident requests the clubhouse 6:00-8:00 PM. The system applies a 30-minute cleanup buffer and rejects overlapping confirmed bookings.

Preconditions: Facility hours, booking windows, capacity, blackout and cancellation policies exist.

Normal flow: Resident signs in → selects facility/time → system checks blackout/capacity/conflicts → payment/deposit if configured → reservation atomically created → confirmation sent → check-in/usage tracked → completion or cancellation handled.

Exception / alternate flow: Concurrent booking, blackout, no-show, over-capacity or cancellation-window violation must return deterministic business outcomes.

State progression: REQUESTED → VALIDATING → CONFIRMED / REJECTED → CHECKED_IN → COMPLETED/CANCELLED/NO_SHOW

Key invariant: Confirmed reservations require atomic conflict protection; cleanup buffers are part of conflict evaluation.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O11-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

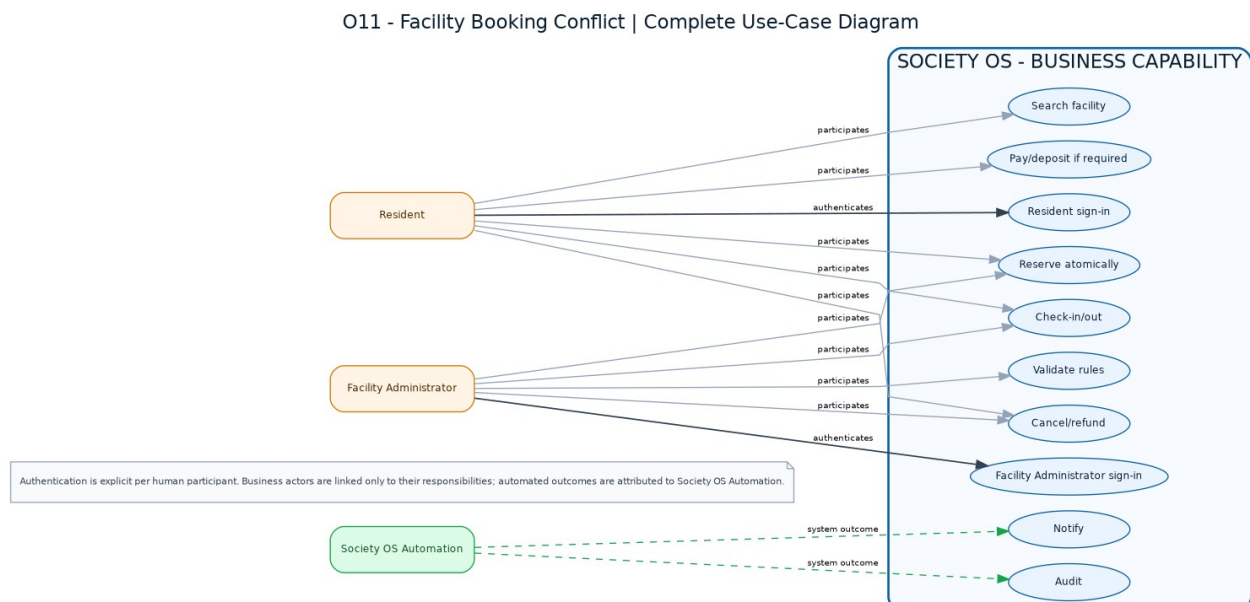
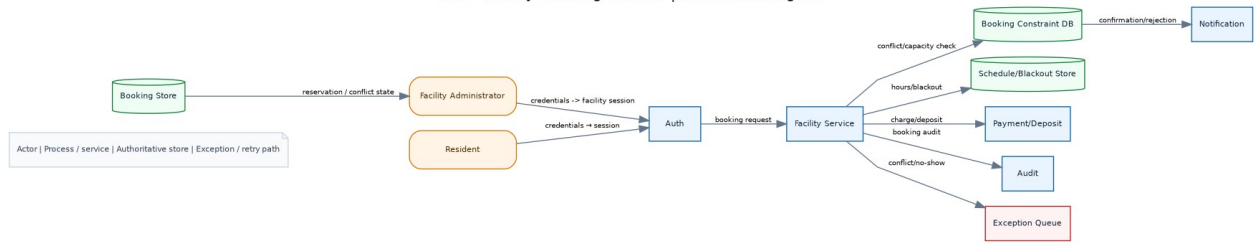


Figure O11-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.

O11 - Facility Booking Conflict | Data Flow Diagram



O12 - Notice and Emergency Broadcast

Domain: Communication

Primary participants: Society Administrator, Resident, Notification Provider

Business objective: Publish targeted notices with priority, schedule, expiry and emergency override semantics.

Real-world scenario: A scheduled water shutdown notice is published at 10:55 AM for a repair beginning at 11:00 AM. A critical fire notice can bypass resident quiet hours under policy.

Preconditions: Notice template/audience and channel configuration exist.

Normal flow: Admin signs in → drafts notice → selects audience/priority/effective window → validates attachments/localization → publishes/schedules → dispatches channels → tracks delivery/acknowledgement → expires/withdraws while preserving revision history.

Exception / alternate flow: Provider throttling, invalid audience, emergency override misuse or withdrawn notice must be auditable.

State progression: DRAFT → SCHEDULED → PUBLISHED → DELIVERING → DELIVERED/PARTIAL → EXPIRED/WITHDRAWN

Key invariant: Emergency override requires an explicit policy reason and audit record.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O12-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

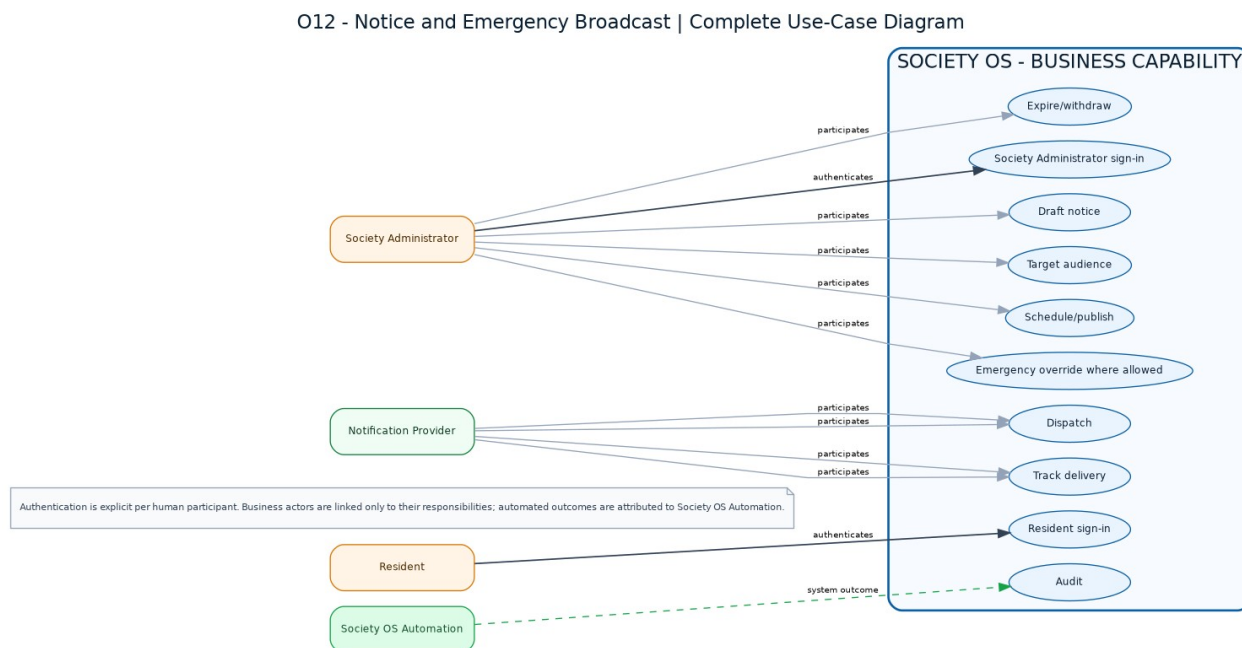
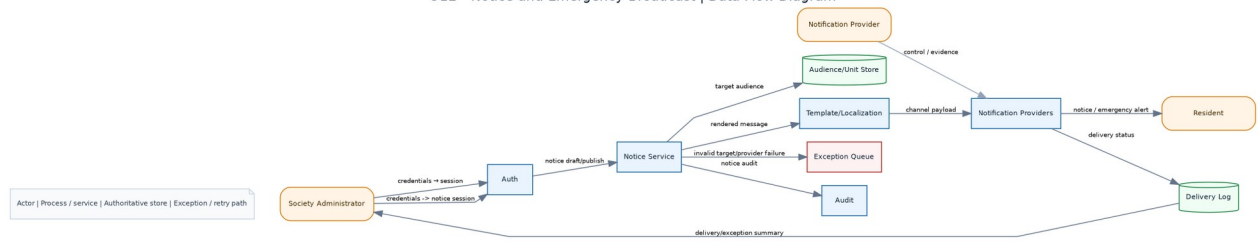


Figure O12-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.

O12 - Notice and Emergency Broadcast | Data Flow Diagram



O13 - Resident Communication Approval

Domain: Communication

Primary participants: Resident, Authorized Contact, Moderator / Administrator

Business objective: Enable controlled resident-to-resident communication without exposing private contact information by default.

Real-world scenario: A committee member wants to contact a resident about a parking issue. The resident accepts the first-contact request before a persistent chat channel opens.

Preconditions: Privacy/contact policy is configured.

Normal flow: Sender signs in → requests contact → system checks privacy policy → recipient receives consent request → recipient accepts/rejects → accepted channel created → messages are append-oriented → report/moderation controls remain active.

Exception / alternate flow: Abuse report, block, revoked consent or message retention policy must close/limit the channel appropriately.

State progression: REQUESTED → PENDING_CONSENT → ACCEPTED/REJECTED → ACTIVE → BLOCKED/CLOSED

Key invariant: Phone numbers remain hidden unless explicitly permitted by policy.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O13-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

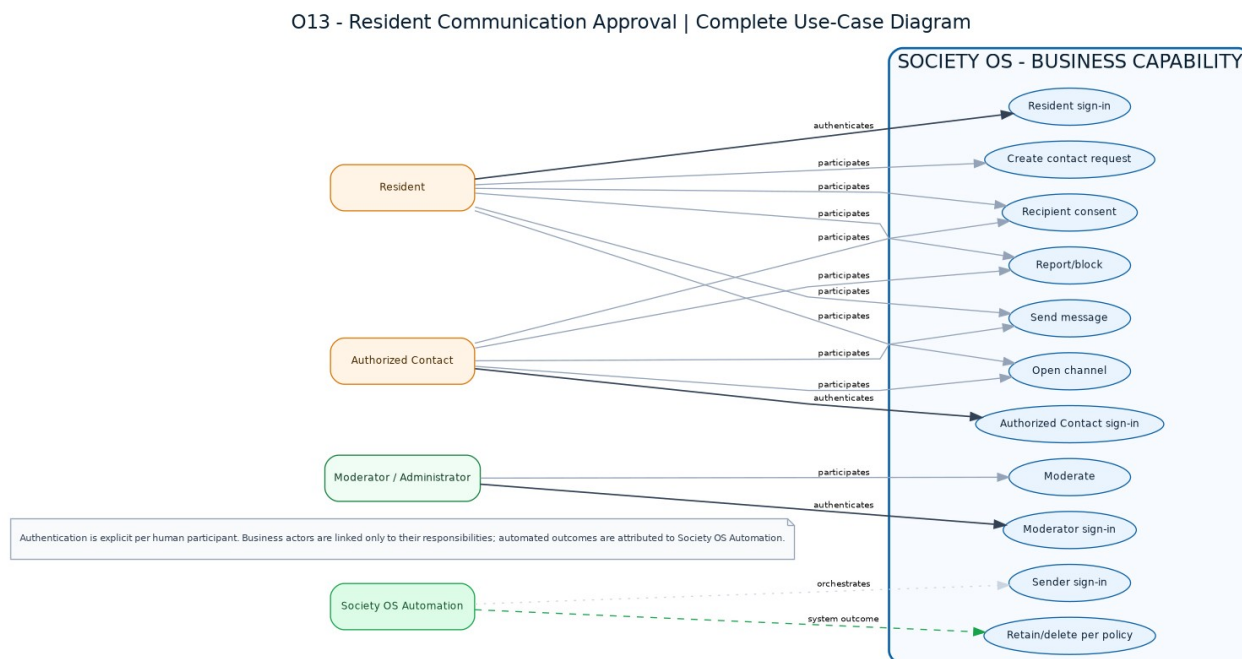
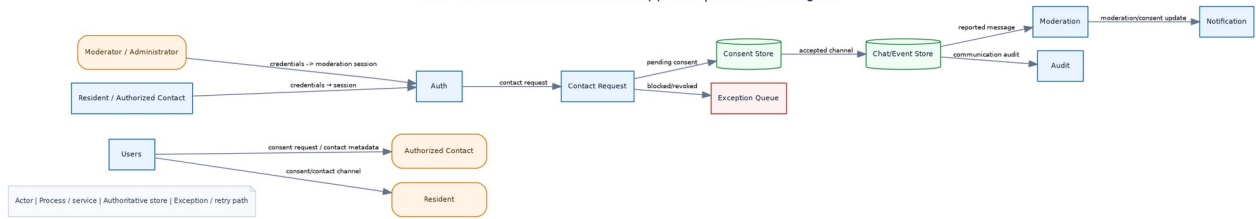


Figure O13-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.

O13 - Resident Communication Approval | Data Flow Diagram



O14 - Move-In Workflow

Domain: Move-In

Primary participants: Resident, Society Administrator, Guard / Facility Team

Business objective: Coordinate a move-in date, access requirements and operational conflicts.

Real-world scenario: A tenant schedules Saturday move-in. Loading access is approved while a lift maintenance block is detected, so the system requests an alternate slot.

Preconditions: Unit/occupancy approval and move-in policy exist.

Normal flow: Resident signs in → submits move-in → admin signs in → verifies occupancy/docs → selects date/time → scheduling engine checks lift/loading conflicts → approvals complete → gate/facility instructions generated → move-in executed → event closed.

Exception / alternate flow: Maintenance conflict, overdue documents, missing deposit or restricted timing must block or route to exception approval.

State progression: REQUESTED → VERIFIED → SCHEDULED → APPROVED → IN_PROGRESS → COMPLETED / CANCELLED

Key invariant: Move-in cannot be marked complete without validated occupancy and required operational clearances.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O14-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

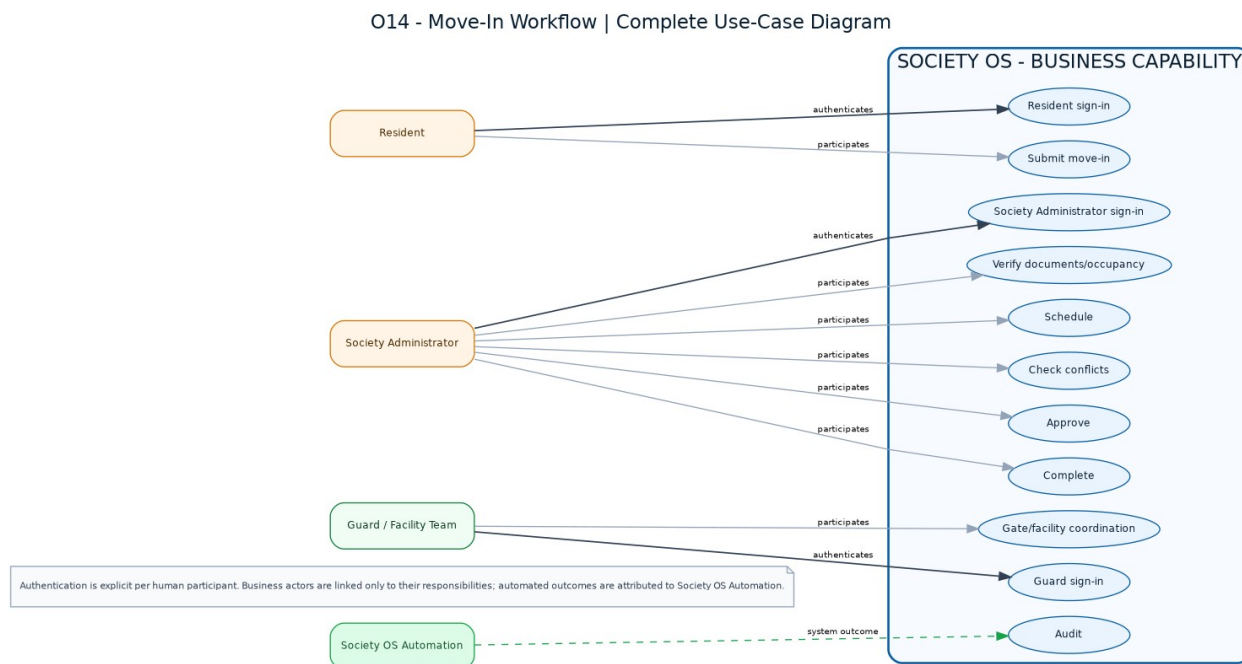
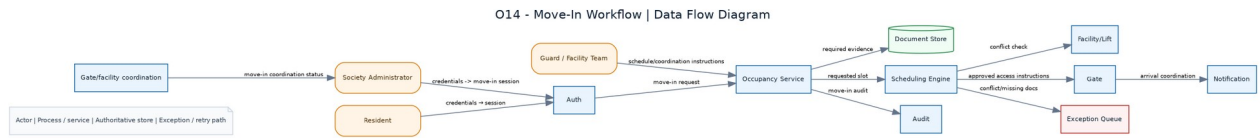


Figure O14-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O15 - Move-Out with NOC

Domain: Move-Out / NOC

Primary participants: Resident, Society Administrator, Finance Administrator, Security Administrator

Business objective: Clear financial, document, parking and access obligations before issuing a verifiable NOC.

Real-world scenario: A resident requests NOC with ₹18,000 outstanding. The system blocks issuance until finance clearance is completed, then admin approves and digitally signs the NOC; access is revoked and history retained.

Preconditions: Move-out request exists; NOC policy/checklist is configured.

Normal flow: Resident signs in → submits move-out/NOC → admin signs in → system evaluates dues, documents, parking, assets and disputes → exceptions resolved → finance clearance → admin approval/signature → NOC PDF/digest generated → gate/credentials revoked → occupancy archived.

Exception / alternate flow: Disputed damage, outstanding dues, active access asset, missing document or failed signature must keep NOC pending.

State progression: REQUESTED → CLEARANCE_CHECK → EXCEPTION/READY → APPROVED → SIGNED → ISSUED → ACCESS_REVOKED → ARCHIVED

Key invariant: NOC issuance must reference the clearance snapshot; revocation must not delete historical records.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O15-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

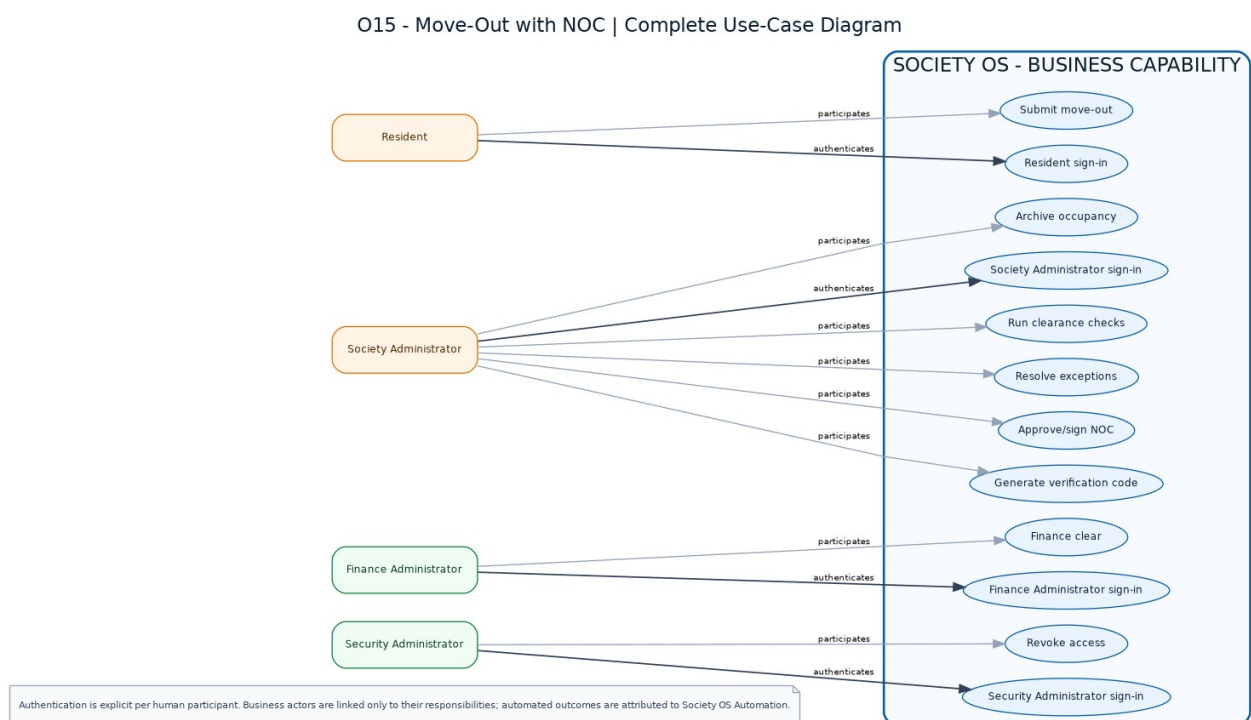
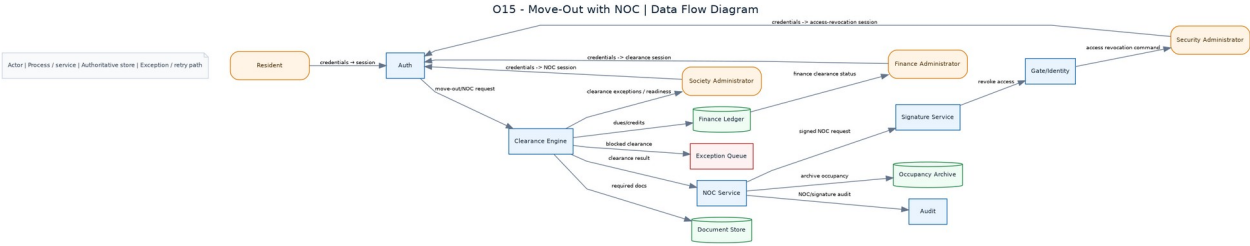


Figure O15-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O16 - Parking Allocation / Transfer

Domain: Parking

Primary participants: Resident, Parking Administrator, Guard

Business objective: Manage parking entitlements, slot allocation, transfers and temporary assignments without conflicting active allocations.

Real-world scenario: A resident sells one vehicle and temporarily receives another slot for two weeks; the old vehicle association ends and the new one becomes active.

Preconditions: Parking inventory and allocation policy exist.

Normal flow: Resident/admin signs in → selects slot → eligibility checked → existing allocation conflict checked → allocation/transfer approved → vehicle association updated → RFID/ANPR mapping updated where configured → guard can validate active assignment.

Exception / alternate flow: Duplicate vehicle, overlapping allocation, reserved/visitor slot misuse or expired temporary allocation must be rejected/expired.

State progression: AVAILABLE → REQUESTED → ALLOCATED → TRANSFERRED/TEMPORARY → ENDED → AVAILABLE

Key invariant: An active physical slot may not have conflicting active resident allocations.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O16-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

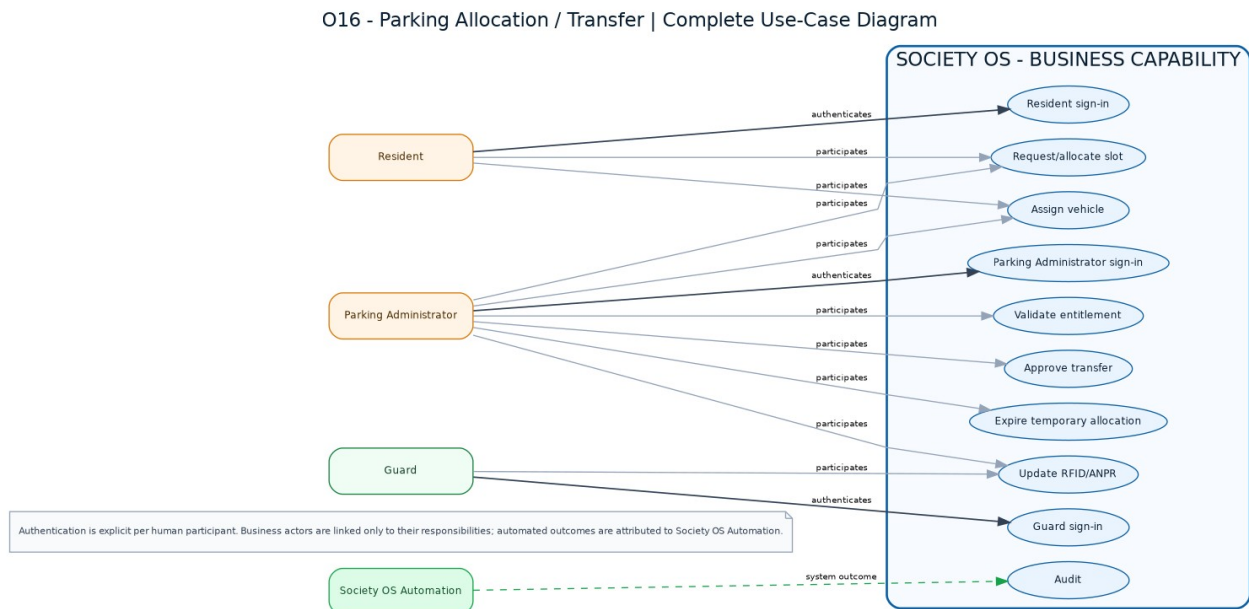
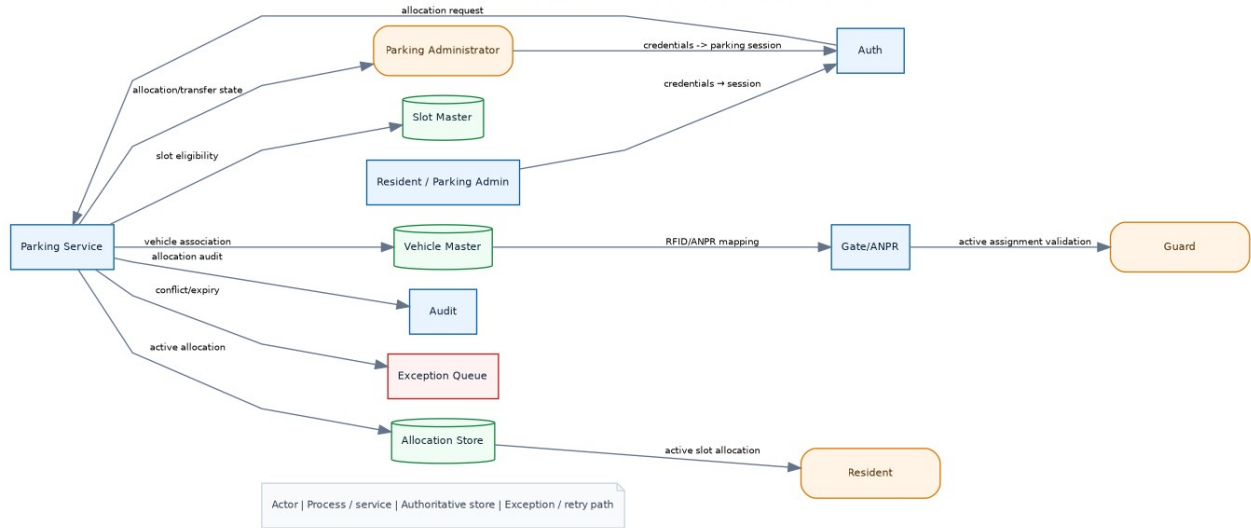


Figure O16-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.

O16 - Parking Allocation / Transfer | Data Flow Diagram



O17 - Asset Lifecycle

Domain: Asset Management

Primary participants: Asset Manager, Technician, Vendor

Business objective: Track an asset from acquisition through allocation, maintenance, impairment and retirement.

Real-world scenario: A generator is received, assigned to the society utility area, serviced monthly, repaired after a failure and eventually retired with its full service history intact.

Preconditions: Asset category and lifecycle statuses exist.

Normal flow: Manager signs in → create/receive asset → capture serial/warranty → assign location → maintenance events update history → incidents link to asset → transfer/return/retire → final history preserved.

Exception / alternate flow: Duplicate serial, transfer to invalid location, warranty mismatch or retirement with open work order requires exception.

State progression: PLANNED → RECEIVED → ACTIVE → UNDER_MAINTENANCE → TRANSFERRED → RETIRED/DISPOSED

Key invariant: Asset identity and historical events remain immutable; current location/owner is derived from the latest valid allocation.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O17-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

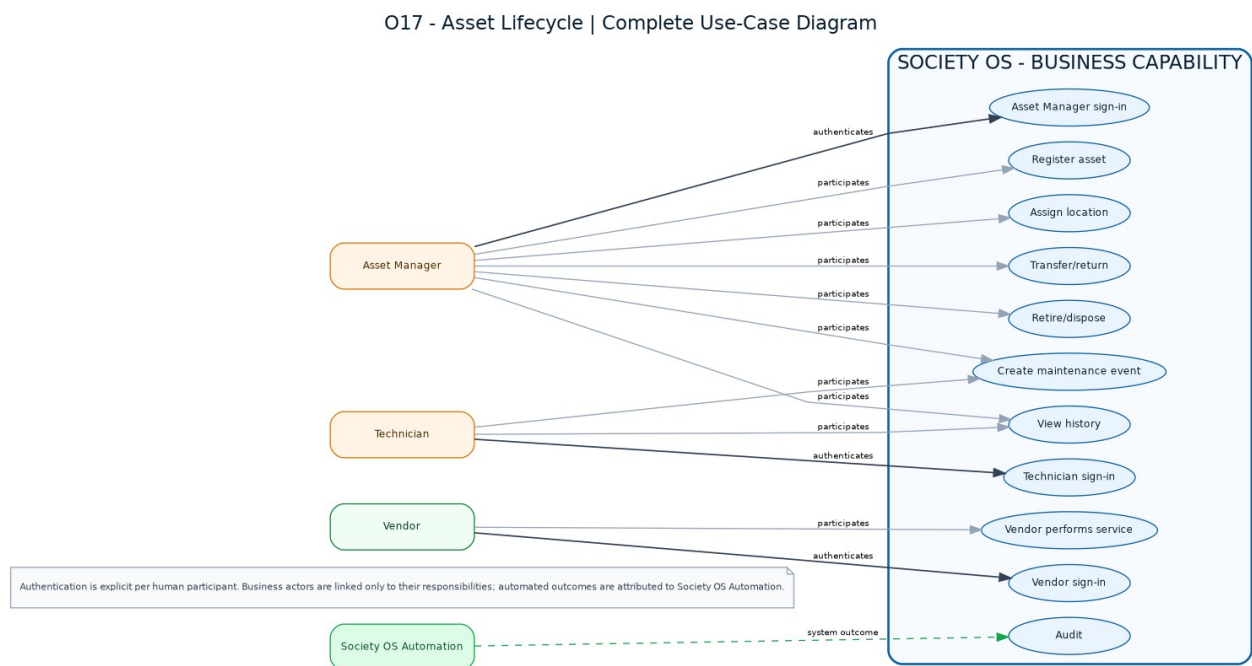
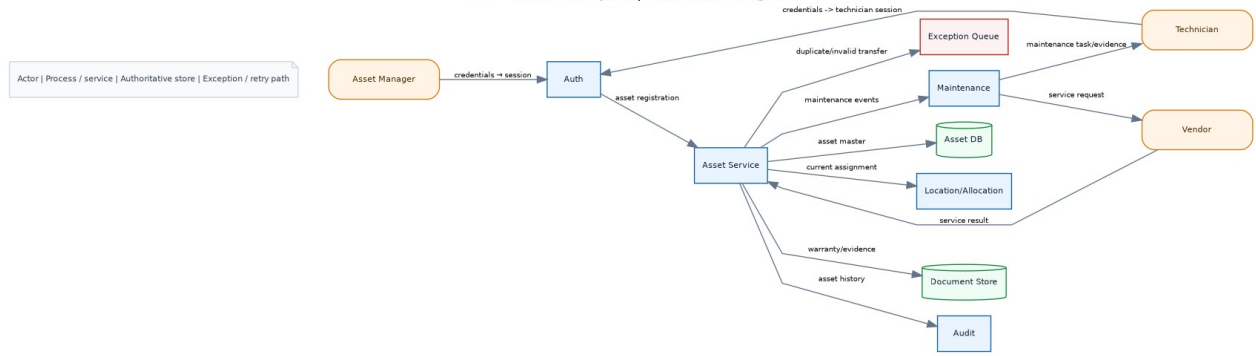


Figure O17-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.

O17 - Asset Lifecycle | Data Flow Diagram



O18 - Document Vault Upload / Version / Secure Access

Domain: Documents

Primary participants: Resident, Society Administrator, Document Service

Business objective: Store sensitive documents privately, version them, verify integrity and expose them only after fresh authorization.

Real-world scenario: A tenant uploads a rental agreement version 2. The previous agreement remains immutable version 1; admin access requires sign-in and authorization. A configured approval can include an admin digital signature.

Preconditions: Document category, owning entity, ACL, retention and size/type rules are configured.

Normal flow: Resident/admin signs in → upload initiated → malware/size/type validation → private object stored → checksum recorded → metadata/version saved → admin signs in for verification/approval where required → secure access issues short-lived retrieval → access is audited.

Exception / alternate flow: Failed upload, duplicate version, unauthorized access, expired document or checksum mismatch must be blocked and logged.

State progression: DRAFT_UPLOAD → VALIDATING → STORED → VERIFIED/REJECTED → ACTIVE_VERSION → SUPERSEDED/EXPIRED

Key invariant: Object storage remains private; every retrieval requires authorization and version integrity remains verifiable.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O18-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

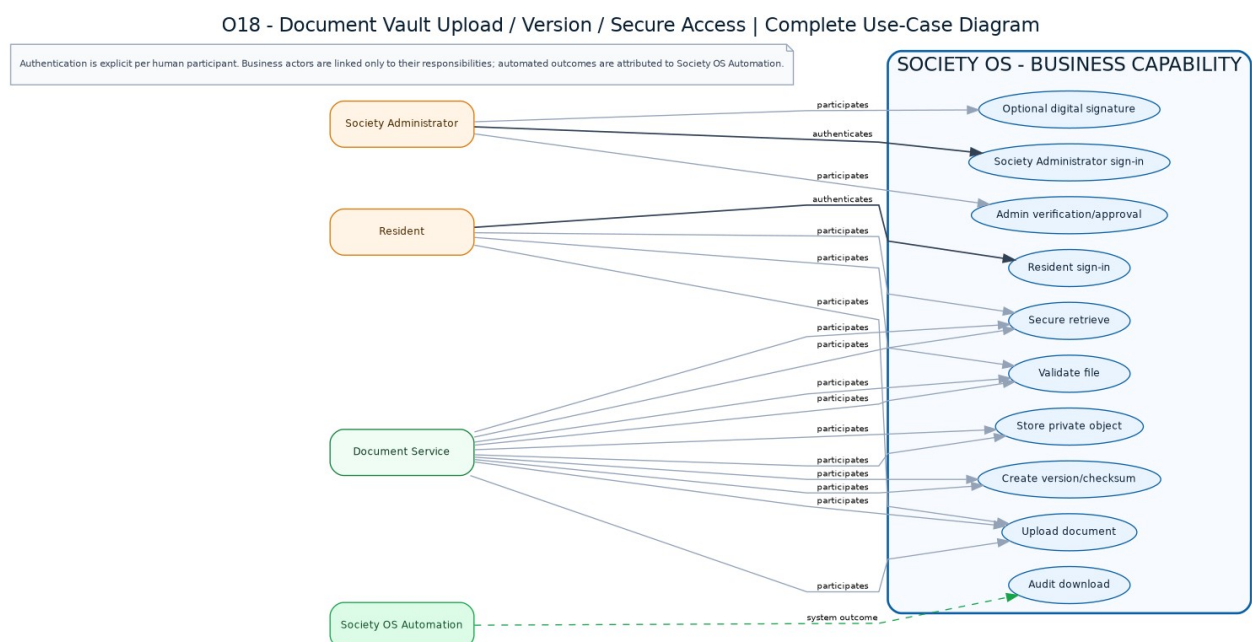
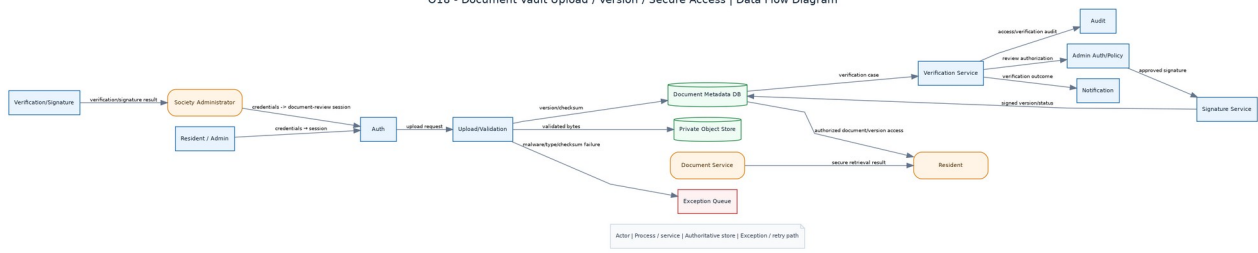


Figure O18-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.

O18 - Document Vault Upload / Version / Secure Access | Data Flow Diagram



O19 - Biometric Attendance Sync / Correction

Domain: Biometric / Attendance

Primary participants: HR / Administrator, Biometric Device, Employee

Business objective: Ingest attendance events idempotently and support authorized corrections without rewriting source punches.

Real-world scenario: A biometric terminal is offline for 20 minutes, stores punches locally, then uploads them. HR later corrects a missed punch with reason and audit.

Preconditions: Device is registered; integration credentials and attendance policy exist.

Normal flow: Device captures punch → local buffer if offline → reconnect → signed/batched sync → server validates device/employee/time → deduplicates → attendance record derived → admin signs in for correction if needed → corrected record versioned/audited.

Exception / alternate flow: Unknown device, duplicate event, clock drift, offline replay or unauthorized correction must be isolated.

State progression: CAPTURED → BUFFERED → SYNCED → VALIDATED → ATTENDANCE_POSTED → CORRECTION_REQUESTED → CORRECTED

Key invariant: Raw biometric templates remain outside Society OS; attendance events are idempotent and correction does not delete source history.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O19-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

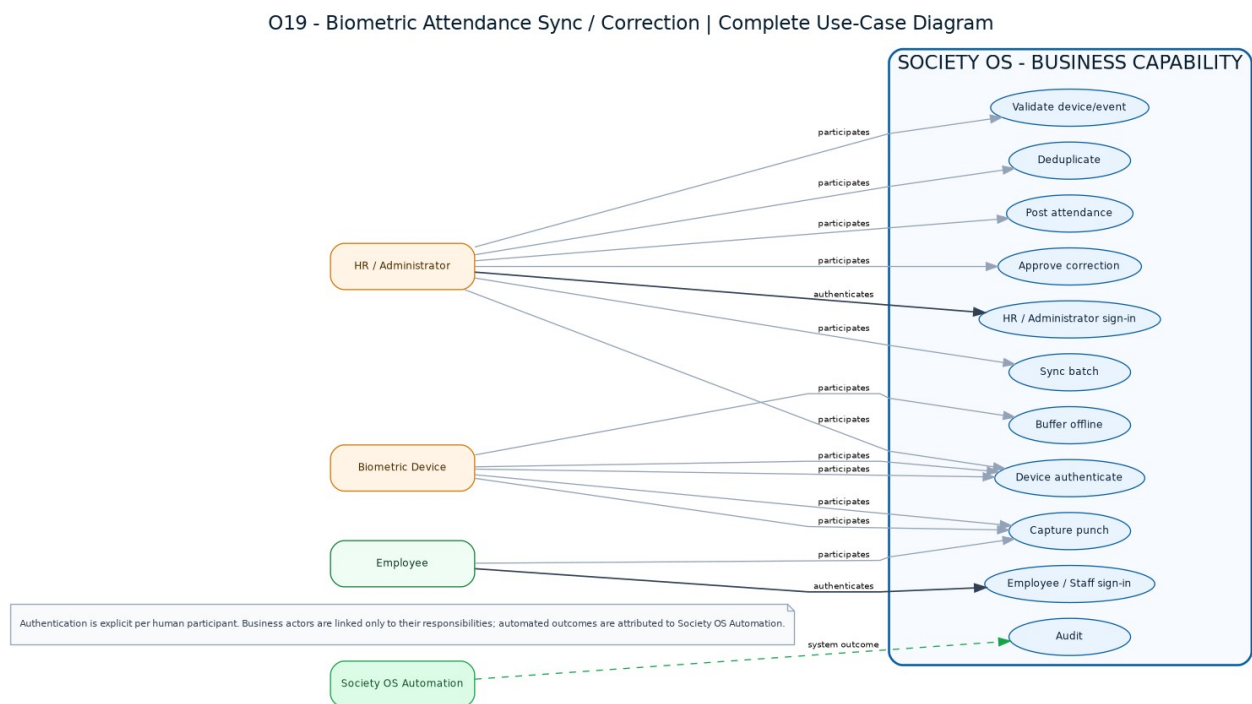
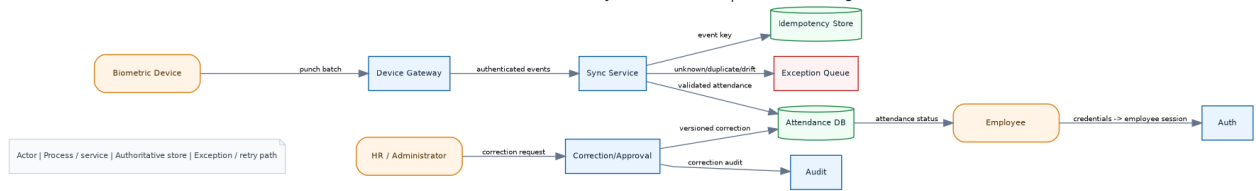


Figure O19-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.

O19 - Biometric Attendance Sync / Correction | Data Flow Diagram



O20 - Import / Export / Migration

Domain: Platform Data

Primary participants: Administrator, Migration Administrator

Business objective: Safely import and export operational masters and transactions with preview, error reporting and rollback/compensation semantics.

Real-world scenario: A society imports residents, vehicles, parking and inventory. 4,912 rows are accepted and 88 fail with downloadable row-level explanations.

Preconditions: Template and mapping rules exist.

Normal flow: Admin signs in → upload → detect schema → validate columns/business rules → duplicate check → preview → confirm → asynchronous commit → success/failure report → history retained; exports apply authorization filters.

Exception / alternate flow: Partial batch failure, duplicate import, invalid foreign key or revoked authorization must not produce silent corruption.

State progression: UPLOADED → VALIDATING → PREVIEWED → COMMITTING → COMPLETED / COMPLETED_WITH_ERRORS / FAILED

Key invariant: Every import has a batch identity and row-level lineage; exports respect privacy and authorization.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O20-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

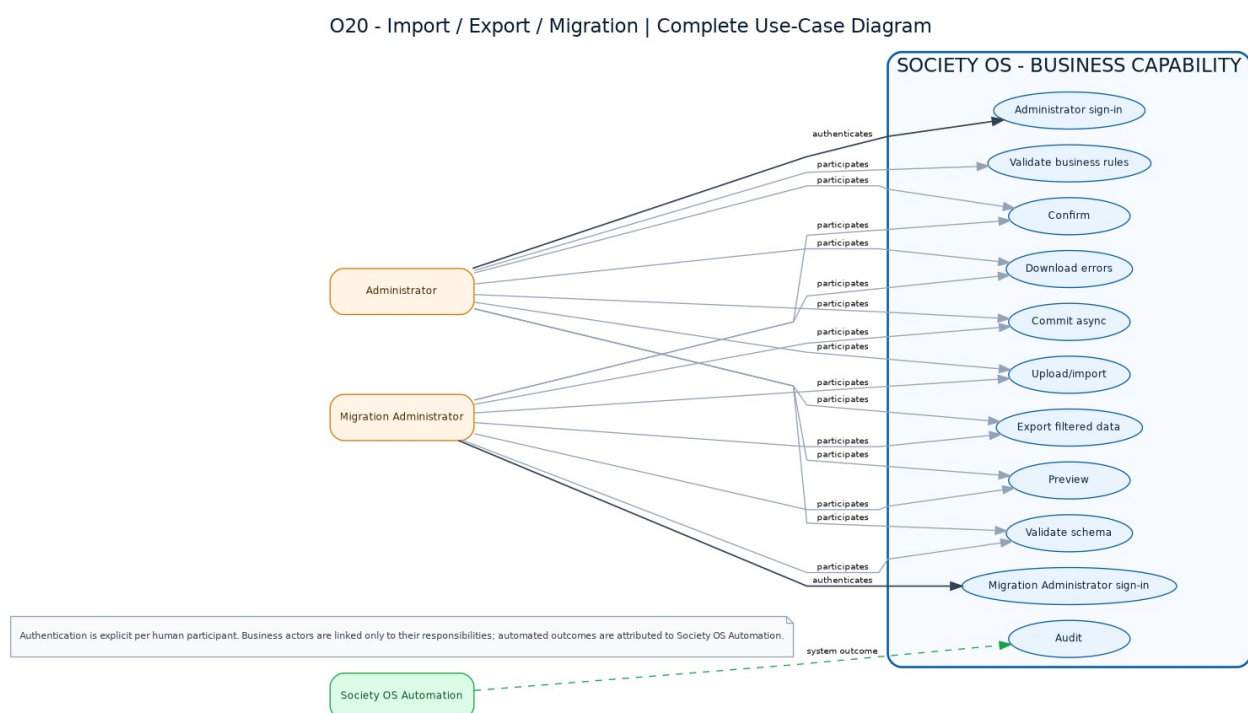
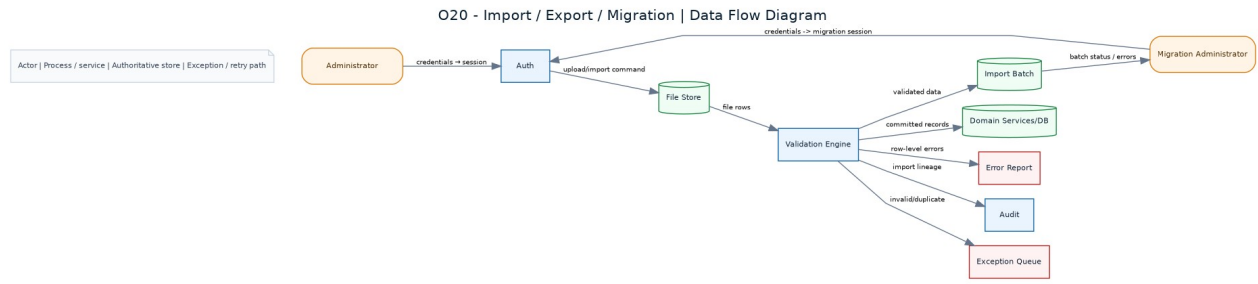


Figure O20-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O21 - Offline Gate Sync / Conflict

Domain: Offline Operations

Primary participants: Guard Device, Resident, Server / Gate Service

Business objective: Continue critical gate operations offline and reconcile safely when connectivity returns.

Real-world scenario: A gate tablet loses connectivity for 20 minutes during a busy period. Guards capture admissible events locally and synchronize when the network returns.

Preconditions: Offline cache and device encryption enabled; device registered.

Normal flow: Guard signs in locally → records event → event queued with client UUID/time/device metadata → connectivity returns → batch sync → server authenticates/device-validates → idempotency/conflict strategy applied → authoritative state returned → local queue marked synced.

Exception / alternate flow: Visitor revocation that predates offline admission, duplicate sync, clock skew or incompatible schema must route to a domain-specific conflict outcome.

State progression: LOCAL_CAPTURED → QUEUED → SYNCING → ACCEPTED / CONFLICT / REJECTED → ACKNOWLEDGED

Key invariant: Conflict strategy is domain-specific; financial ledgers never use generic last-write-wins.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O21-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

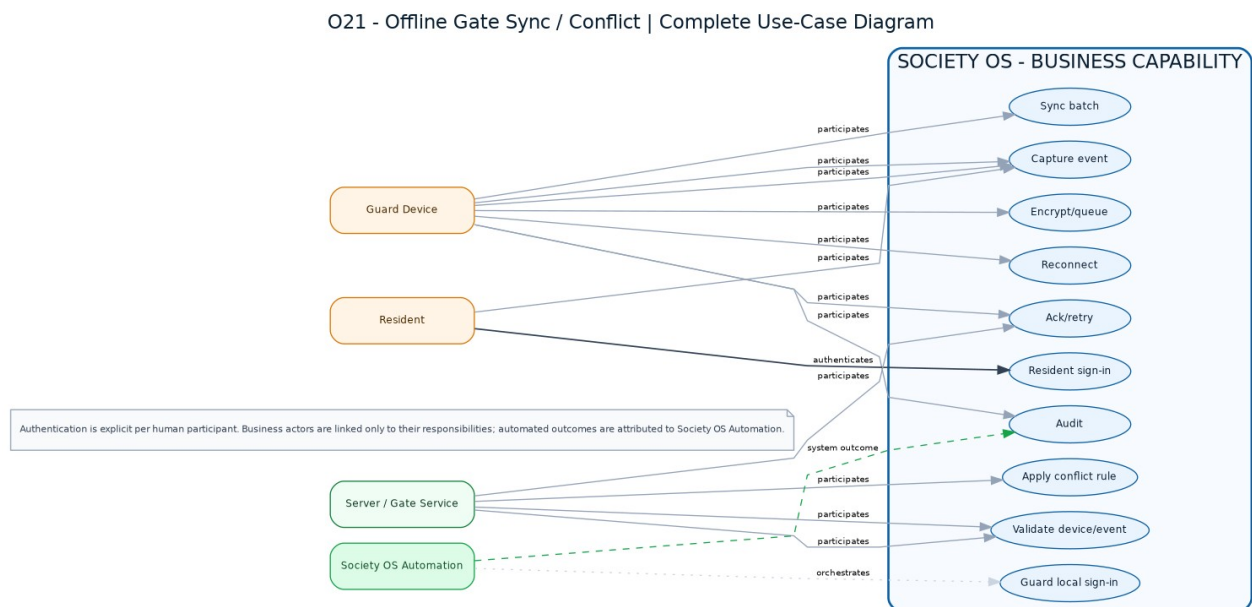
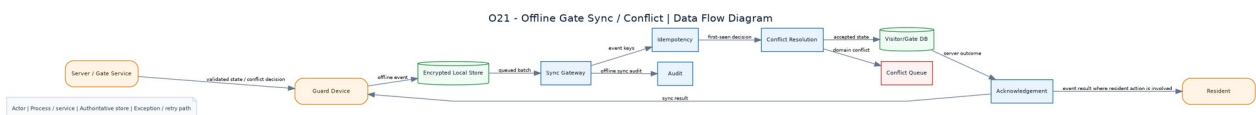


Figure O21-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O22 - Notification Retry / Fallback

Domain: Platform Messaging

Primary participants: Notification Engine, Notification Provider, Administrator

Business objective: Deliver important notifications reliably with retries, fallback channels, templates and observable failure states.

Real-world scenario: Payment reminders are delayed because the push provider throttles requests. The engine retries, then falls back to SMS for residents whose preferences permit it.

Preconditions: Template, user preferences, channel credentials and retry policy exist.

Normal flow: Domain event emitted → notification rule evaluated → template/localization rendered → preferred channel attempted → delivery result recorded → retry/backoff → fallback if configured → final success/DLQ → admin operations can inspect.

Exception / alternate flow: Provider outage, invalid token, quiet hours, critical override or dead-letter condition must be visible and auditable.

State progression: CREATED → QUEUED → SENT → DELIVERED or RETRYING → FALLBACK → DLQ/EXHAUSTED

Key invariant: Notification delivery must not change authoritative business state; it is a downstream effect.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O22-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

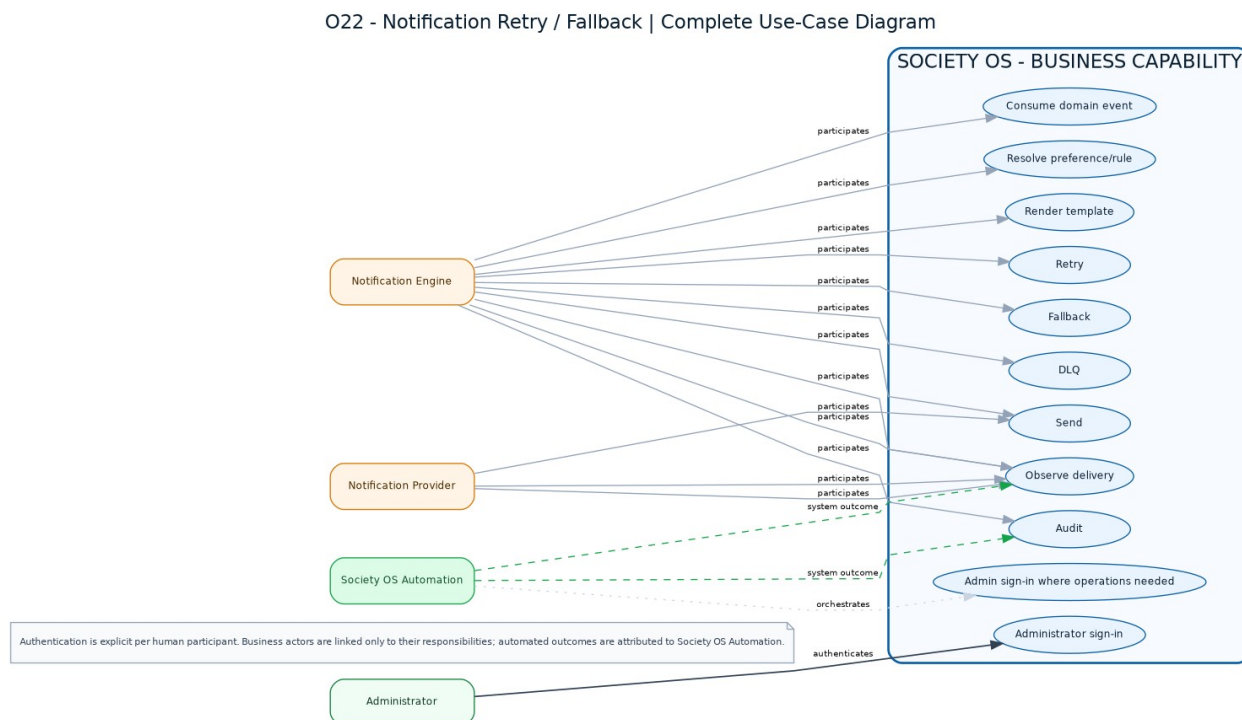
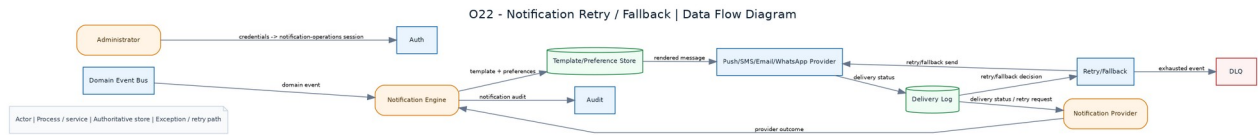


Figure O22-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O23 - Global Search and Privacy Filtering

Domain: Search

Primary participants: Authorized User, Search Service

Business objective: Provide fast cross-domain search without bypassing underlying privacy and tenant boundaries.

Real-world scenario: An administrator searches for “A-1203” and sees the unit, active occupant, vehicle and open complaint permitted by policy, but not a former tenant’s restricted KYC document.

Preconditions: Search indexes and authorization policy are available.

Normal flow: User signs in → enters search → server resolves society context and permissions → searches allowed indexes → filters/sorts/paginates → field-level privacy filters applied → results returned → access may be audited.

Exception / alternate flow: Cross-society query, restricted document, stale index or export attempt must not bypass authorization.

State progression: REQUESTED → AUTHORIZED → SEARCHED → FILTERED → RETURNED / PARTIAL / DENIED

Key invariant: Search is a projection; it must never become a side-channel around source authorization.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O23-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

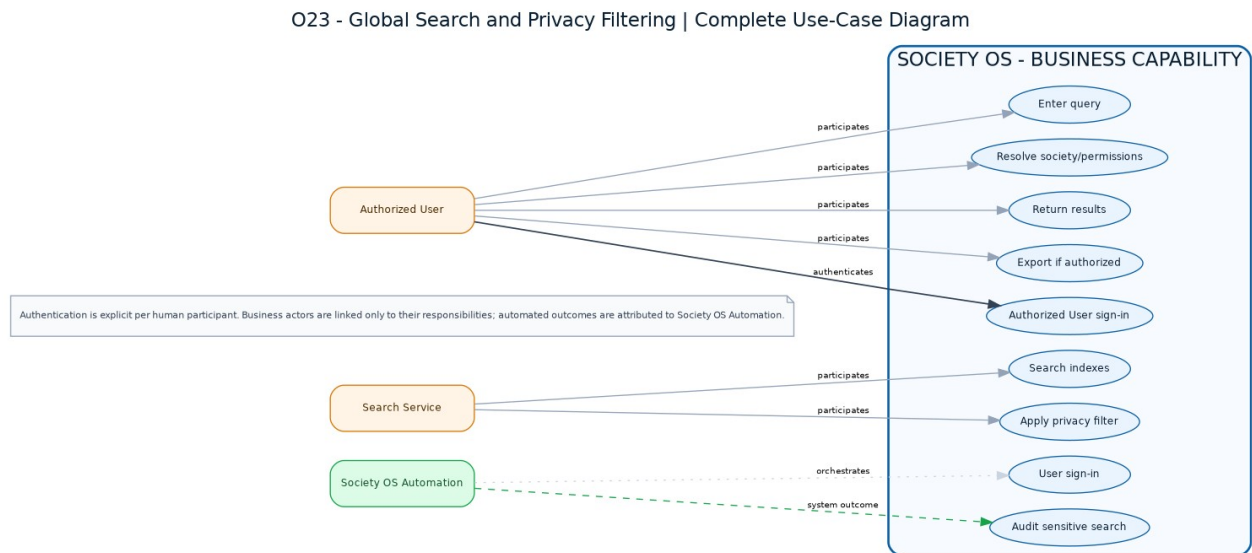
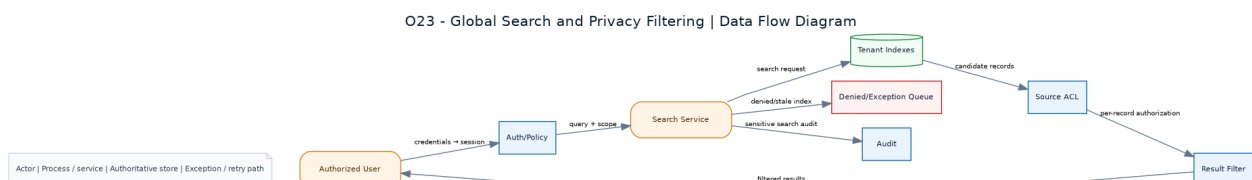


Figure O23-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O24 - Backup Restore / Disaster Recovery

Domain: Platform Operations

Primary participants: Operations Engineer, Storage/Database Services

Business objective: Recover the platform to a validated point while preserving tenant isolation and audit integrity.

Real-world scenario: A deployment corrupts configuration data. Operations restores the affected state to a known point, validates tenant isolation and replays safe operational jobs.

Preconditions: Backups, PITR/logs, restore runbooks and RPO/RTO targets exist.

Normal flow: Engineer signs in → declares incident → selects recovery target → restores database/object data → validates schema/checksum/RLS → replays eligible jobs → reconciles queues/audit → validates application health → closes incident.

Exception / alternate flow: Corrupt backup, partial object restore, missing audit events or replayed financial callbacks require stop/rollback/escalation.

State progression: INCIDENT → RECOVERY_PLANNED → RESTORING → VALIDATING → RECONCILING → VERIFIED → CLOSED

Key invariant: Recovery must preserve tenant boundaries and must not replay financial effects without idempotency checks.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O24-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

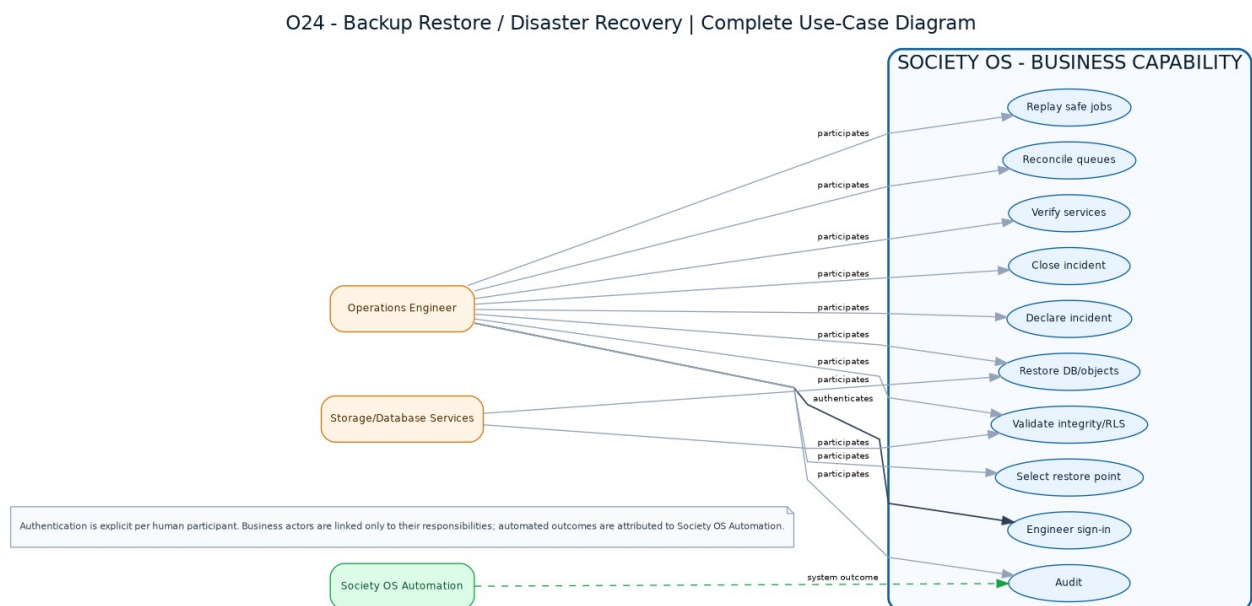
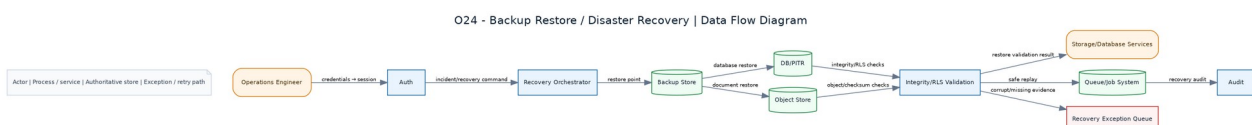


Figure O24-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O25 - AI-Assisted Complaint Classification

Domain: Intelligence

Primary participants: Operator, AI Service, Helpdesk Administrator

Business objective: Use AI to assist classification while keeping final business state deterministic and human-correctable.

Real-world scenario: A complaint text and photo strongly suggest water leakage. AI predicts the category and confidence, but the operator remains able to correct it before assignment.

Preconditions: AI model/version and classification taxonomy exist.

Normal flow: Operator signs in → submits complaint → AI service analyzes text/media → classification and confidence returned → operator reviews/corrects → deterministic category/SLA assigned → audit stores model/version and final human decision.

Exception / alternate flow: Low confidence, model timeout, hallucinated detail or unsafe recommendation must fall back to manual classification.

State progression: CREATED → AI_ANALYZED → REVIEWED → CLASSIFIED → ASSIGNED

Key invariant: AI output is advisory metadata; it never directly authorizes payments, access, NOC or other consequential state.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O25-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

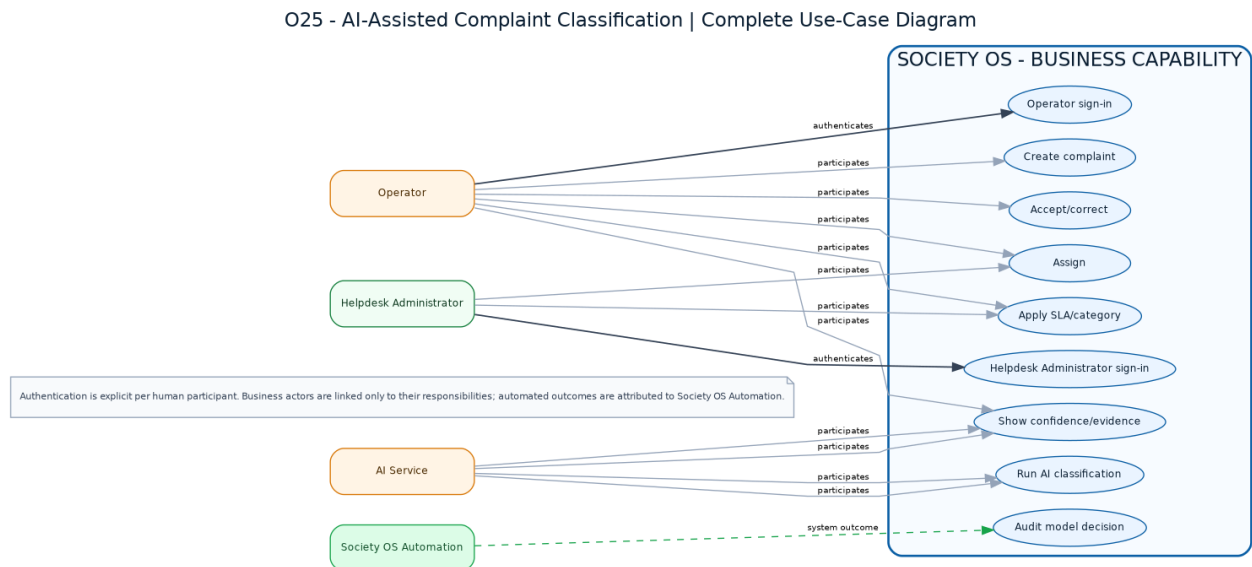
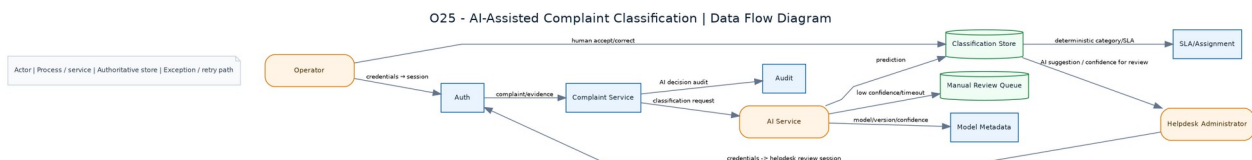


Figure O25-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



O26 - Document Verification / Admin Approval & Digital Signing

Domain: Documents / Verification

Primary participants: Resident, Society Administrator, Verification Service, Signature Service

Business objective: Complete a formal document verification workflow in which the administrator is explicitly authenticated, reviews evidence and signs/approves the outcome when configured.

Real-world scenario: A tenant uploads a rental agreement and identity document. The admin signs in, checks the files and metadata, rejects the agreement for a missing page, the tenant resubmits, and the admin approves and digitally signs the verification outcome.

Preconditions: Document category, verification checklist, approval threshold and signature method are configured.

Normal flow: Resident signs in → uploads document → file validation/checksum → verification case created → admin signs in → opens verification case → reviews document/metadata → accept/reject/resubmit → if approved, admin signs approval/certificate where configured → verification status recorded → resident notified → audit trail sealed.

Exception / alternate flow: Expired credential, tampered file/checksum mismatch, unauthorized verifier, signature failure or resubmission timeout keeps the case pending/rejected and never marks the document verified.

State progression: UPLOADED → VALIDATING → PENDING_ADMIN_REVIEW → RESUBMISSION_REQUIRED or VERIFIED → SIGNATURE_PENDING → SIGNED → ACTIVE

Key invariant: Only an authenticated authorized admin/verifier may approve; verification and signature are separate auditable events.

QA acceptance intent

- Verify the protected human actor signs in before performing privileged operations such as approval, verification, financial adjustment or NOC issuance.
- Verify the happy path creates exactly one authoritative business outcome and downstream notifications are triggered from confirmed state.
- Verify retries, duplicates and provider callbacks cannot create duplicate business effects.
- Verify exception states are visible, recoverable and auditable, without corrupting historical records.

Figure O26-A. Complete use-case diagram - participants, authenticated entry points and business interactions.

O26 - Document Verification / Admin Approval & Digital Signing | Complete Use-Case Diagram

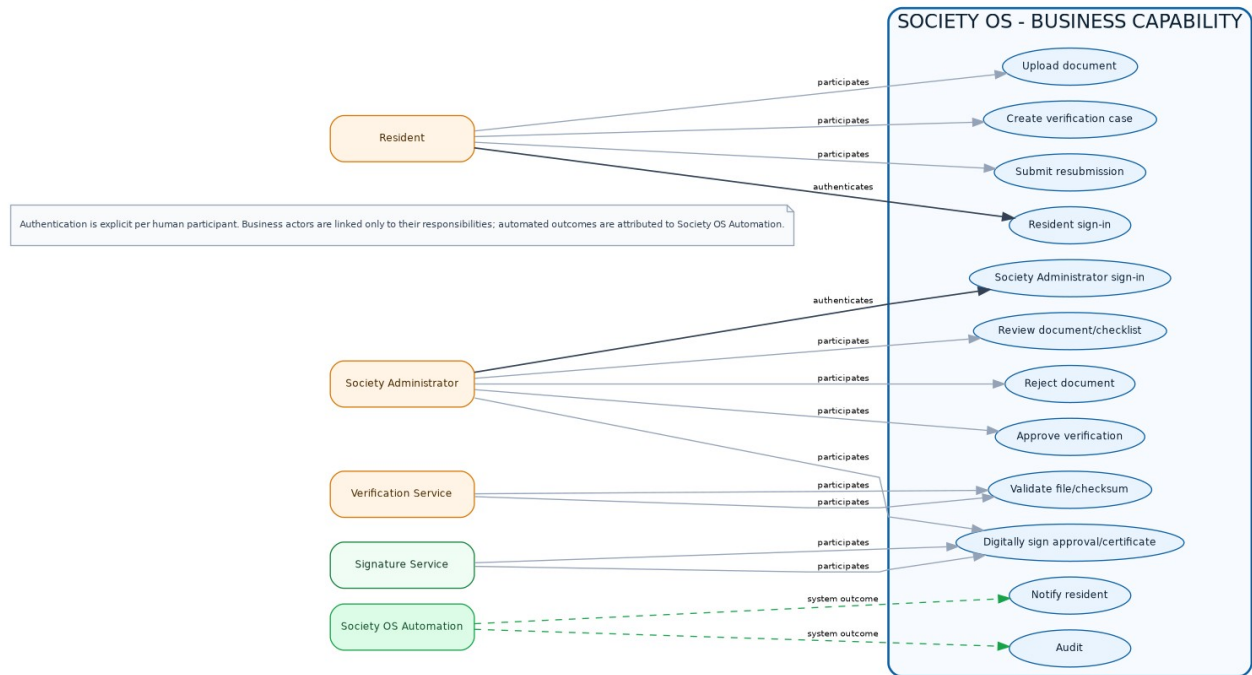
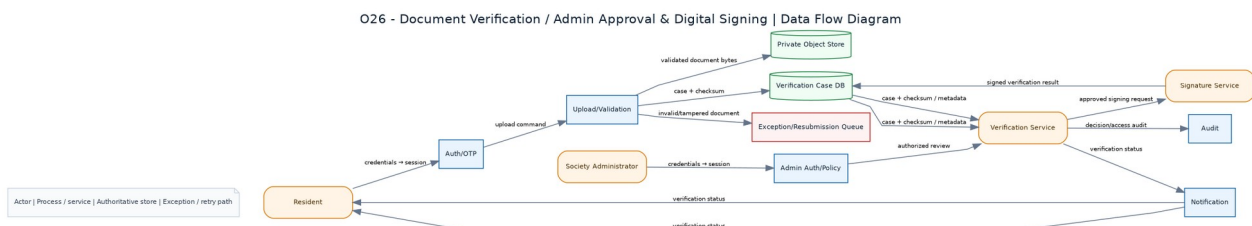


Figure O26-B. Complete data flow diagram - actors, services, authoritative stores, external boundaries and exception path.



19.11 Financial Scenario Completion Standard

- Finance is the hero product domain and must be treated as a controlled accounting system, not simply a collection of payment screens or gateway integrations.
- Every financial scenario explicitly identifies source transaction, calculation/matching rule, approval boundary, ledger effect, reconciliation state, correction/reversal path, notification effect and audit evidence.
- Every posted journal is balanced; posted history is append-only; corrections are linked reversal/adjustment transactions; refunds reference originating payments; duplicate callbacks are idempotent; reconciliation never edits authoritative ledger history.
- Every financial reportable number is traceable to source transactions and effective-dated rules so that historical periods remain reproducible.
- Financial exception states must end in a deterministic operator action: retry, wait for provider, reconcile, approve, reverse, refund, dispute, escalate or close.

19.12 Explicit Document Verification Control

Mandatory rule: Document verification is not complete merely because a document was uploaded. The administrator/verifier must authenticate into the system before opening the verification case, perform the configured checklist, record approve/reject/resubmit, and - where digital approval/signature is configured - execute the signature action as a separate auditable event.

Why this matters: This prevents the common diagram gap in which a document jumps directly from resident upload to VERIFIED without showing the authenticated administrative decision, the verification service, the approval/signature boundary and the resulting audit evidence.